



Nottingham Trent
University

Department of Computer Science

MAJOR PROJECT COMP40311

**“A SEMANTICALLY DEVELOPED KNOWLEDGE-BASED SYSTEM FOR
DEPLOYING KALI LINUX TOOLS”**

By

EFFIONG OKON ANTAI (N1280528)

Supervisor: DR. RICHARD OTUKA

Date: 29 August, 2025

In submitting this work I confirm that I am aware of, and am abiding by, the University's expectations for proof-reading. In accordance with NTU policy, I hereby declare that the above stated assessment is my own work and that it has not previously been submitted for assessment to another University or for any other qualification.

Student Signature:

Date: 29 August, 2025

Acknowledgement

I would like to express my deepest gratitude to my supervisor Dr Richard Otuka for their guidance, encouragement, and constructive feedback throughout this research project. Their support was invaluable in shaping the direction of my work and ensuring its successful completion.

I am also grateful to the lecturers and staff of Nottingham Trent University for providing the knowledge, tools, and resources that made this project possible. Special thanks go to my colleagues and classmates for their collaboration, discussions, and helpful insights.

I owe heartfelt appreciation to my family for their unwavering love, prayers, and encouragement during this journey. Their belief in me gave me the strength to stay focused and motivated.

Finally, I would like to thank God Almighty for giving me the wisdom, health, and perseverance to complete this work.

Index (Table of Contents)

Abstract	7
Executive Summary	8
Chapter 1: Introduction	
1.1 Background.....	10
1.2 Problem Statement.....	11
1.3 Aim and Objectives.....	12
1.4 Research Questions.....	12
1.5 Scope of Study.....	13
1.6 Project Significance.....	15
1.7 Project Organisation.....	15
1.8 Overview of Methodology.....	16
1.9 Gantt Chart.....	16
1.10 Ethical and Legal Concerns.....	20
1.11 Structure of the Report.....	22
Chapter 2: Literature Review	
2.1 Introduction.....	24
2.2 Knowledge-Based Systems (KBS).....	24
2.3 Ontologies and the Semantic Web.....	24
2.4 Protégé as an Ontology Tool.....	25
2.5 Kali Linux and Its Challenges.....	26
2.6 Existing Research and Gaps.....	26
2.7 Web-Based and Static Solutions.....	29
2.8 Summary of Gaps.....	32
2.9 Conclusion.....	34
Chapter 3: New Ideas and Approach	
3.1 Introduction.....	36
3.2 System Architecture Overview.....	36
3.3 System Workflow.....	38

3.4 Ontology and Taxonomy.....	45
3.5 UML Class Diagram.....	49
3.6 Professional, Social, Ethical and Legal (PSEL) Considerations.....	51

Chapter 4: Implementation

4.1 Introduction.....	52
4.2 Ontology Class Hierarchy.....	52
4.3 Category Class Individuals.....	59
4.4 Object Property Linking.....	60
4.5 Data Properties.....	62
4.6 Reasoning and SWRL Rules Implementation.....	63
4.7 DL Queries.....	67
4.8 SPARQL Query Templates for Tool Recommendation.....	68
4.9 Integration of Reasoning & SPARQL in the Recommendation Pipeline.....	73
4.10 Ethical Guardrails and Annotation Integration.....	74
4.11 Summary.....	77

Chapter 5: Testing, Evaluation, and Validation

5.1 Introduction.....	79
5.2 Test Method.....	79
5.3 Scenarios and Results.....	80
5.4 Summary.....	91
5.5 Validation Against Objectives.....	93
5.6 Userbility Feedback.....	95
5.7 Error Handling & Robustness.....	95
5.8 Limitations.....	96
5.9 Reproducibility Steps.....	97
5.10 User Testing and Feedback.....	98
5.11 Chapter Summary.....	100

Chapter 6: Conclusion and Future Work

6.1 Introduction.....	101
6.2 Summary of Achievements.....	101
6.3 Limitations.....	102
6.4 Evaluation of Protégé Against Alternatives and Prior Work.....	102
6.5 Future Work.....	104
6.6 Final Reflection.....	104
6.7 Thesis Summary.....	105
References.....	107

List of Figures

Figure 1: Project Gantt Chart by Weeks.....	17
Figure 2: Overview Diagram of KBS for Kali Linux Tools Selection.....	37
Figure 3: Flowchart of the Runtime Tool Recommendation Pipeline.....	38
Figure 4: UML Class Diagram of the Ontology Structure.....	50
Figure 5: Class hierarchy of the implemented ontology in Protégé.....	53
Figure 6: "Category" Class hierarchy of the implemented ontology.....	54
Figure 7: "SuggestedTool" Class hierarchy of the implemented ontology.....	55
Figure 8: "Task" Class hierarchy of the implemented ontology.....	56
Figure 9: "Tool" Class hierarchy of the implemented ontology.....	57
Figure 10: "UserLevel" Class hierarchy of the implemented ontology.....	59
Figure 11: "SuggestedToolChain" Class hierarchy of the implemented ontology.....	60
Figure 12: Object property "isUsedFor" linking Tool to Task.....	61
Figure 12: Object property "isFollowedBy" linking tools in sequence.....	62
Figure 13: Object property "belongsToCategory" linking Tool to Category.....	63
Figure 14: Data Property Assertions in Protégé.....	63
Figure 15: SWRL Rule Inferring Beginner Tools.....	64
Figure 16: SWRL Rule Inferring Password Cracking Tools.....	65
Figure 17: Example Reasoning Output.....	66
Figure 18: Example DL queries in Protégé with Pellet.....	68
Figure 19: DL Query for retrieving tools by Task label.....	69

Figure 20: Querying Suggested Tools After Reasoning.....	70
Figure 22: Filtering by User Level.....	71
Figure 23: Querying Tool Sequences (isFollwedBy Property).....	72
Figure 24: Integration in the Runtime System.....	73
Figure 25: Ethical Guardrails as Ontology Annotations.....	75
Figure 26: Annotation Properties (Ethics-as-Metadata).....	76
Figure 27: Annotation Properties (Ethics-as-Metadata).....	76
Figure 28: Global Ontology Graph of Kali Linux Tools showing Tasks,.....	80
Figure 29: Ontology Graph of CrackPassword Task and Associated Tools.....	81
Figure 30: Task-based Retrieval (CrackPassword).....	81
Figure 31: Skill-Level Retrieval	83
Figure 32: Tool Chain Retrieval (Suggested Sequences).....	84
Figure 33: Skill-Level Retrieval.....	85
Figure 34: Negative Test	87
Figure 35: Performance Test.....	88
Figure 36: Beginner Tools-Tools and requireSkillLever.....	89
Figure 37: Advanced Wireless Tool- Tool and BelongsToCategory.....	90
Figure 38: Tools that lead to OpenVAS.....	91

List of Tables

Table 1: Project Risks and Mitigation Plan.....	15
Table 2: Summary of Gaps.....	33
Table 3: FollowedBy Tool Sequence.....	43
Table 4: FollowedBy Tool Sequence.....	44
Table 5: Tool Taxonomy and Command Integration.....	48
Table 6: Task-based Retrieval (CrackPassword).....	82
Table 7: Skill-level Retrieval (Beginner Tools).....	83
Table 8: Tool Chain Retrieval (Suggested Sequences).....	84
Table 9: Advanced Tools Retrieval.....	86
Table 10: Complex DL Queries.....	89

Abstract

This project presents the design and implementation of a semantically developed knowledge-based system for recommending Kali Linux tools. The motivation was to help learners and penetration testers navigate the large number of tools in Kali Linux by providing context-aware, task-driven, and skill-level appropriate suggestions.

The ontology was developed in Protégé using OWL 2, with a structured hierarchy of classes including Tool, Task, Category, SuggestedTool, and UserLevel. Object properties such as `isUsedFor`, `isFollowedBy`, and `requiresSkillLevel` captured semantic relationships, while data properties such as `hasCommand` and `hasDescription` provided tool-specific details. SWRL rules and the Pellet reasoner were used to dynamically infer SuggestedTool instances, enabling automatic classification and sequential workflow guidance.

SPARQL query templates were designed to support retrieval by task, skill level, and tool sequences, with practical tests demonstrating accuracy and adaptability. Ethical and legal safeguards were embedded through annotation properties, ensuring recommendations are delivered with reminders of authorisation, risk, and responsible use.

Evaluation was conducted through six test scenarios covering task retrieval, skill-level filtering, workflow chains, a negative test, and performance checks. Results showed that the ontology was consistent, efficient, and able to generate reliable, user-aware recommendations.

The system contributes a reusable semantic model for cybersecurity training and practice, offering both technical guidance and ethical safeguards. It provides a foundation for further integration into interactive learning platforms and real-world penetration testing environments.

Executive Summary

This thesis develops a knowledge-based system (KBS) that recommends Kali Linux tools based on the user's task and skill level. The core of the system is an OWL 2 ontology built in Protégé, with key classes (Tool, Task, Category, UserLevel, SuggestedTool), object properties (isUsedFor, belongsToCategory, isFollowedBy, requiresSkillLevel), and data properties (hasCommand, hasDescription). SWRL rules and the Pellet reasoner infer SuggestedTool items and short tool chains, while SPARQL queries return practical outputs (tool names, short descriptions, and example commands).

Motivation: Kali Linux includes hundreds of tools, which can overwhelm beginners and slow down experts. Static lists and keyword search do not explain when, why, or in what order tools should be used. This project addresses that gap with a semantic, task-aware model that supports simple workflows (e.g., Nmap -> Nikto -> OpenVAS).

Method.

Model the domain (tools, tasks, categories, user levels) in OWL 2.

Add commands and descriptions as data properties.

Write SWRL rules for skill tags, category checks, and basic sequence logic.

Use DL Queries (Manchester syntax) in Protégé for fast checks and SPARQL for final retrieval.

Evaluate with scenario tests, negative checks, timing, and user feedback.

Implementation highlights.

Reasoning: Pellet + SWRL infer SuggestedTool and simple chains; DL Queries confirm behaviour quickly inside Protégé.

Queries: SPARQL templates support task-based retrieval, skill filtering, and property-path chain discovery (e.g., tools that reach OpenVAS in ≤ 2 steps).

Pipeline: Task normalisation $\rightarrow$ reasoning $\rightarrow$ SPARQL retrieval $\rightarrow$ output with descriptions and commands.

Evaluation. Six scenarios tested task retrieval (e.g., CrackPassword), beginner/advanced filtering, chain suggestions, a negative test (to block wrong mappings), performance checks, and user testing. Results showed consistent inferences, correct mappings (e.g., Nmap/Zenmap for PortScanner), and useful next-step guidance.

Ethics. The system embeds ethical/legal notes (e.g., authorised scope, risk cues, preferred phase), shown alongside recommendations to promote responsible use.

Contributions.

A reusable OWL ontology of Kali tools linked to tasks, skill levels, and simple workflows.

A reasoning layer (SWRL + Pellet) that creates SuggestedTool and short tool chains.

A set of DL and SPARQL query templates that turn the model into actionable recommendations.

Limitations. Manual population (limited coverage), Protégé's template saving constraints, and lack of a production API.

Future work. Automate tool ingestion; publish the model to a SPARQL/OWL server (e.g., Fuseki/GraphDB) for live use; add WebProtégé for collaborative curation; extend workflows and natural-language input.

Chapter 1: Introduction

1.1 Background

In today's digital world, cybersecurity is very important. People, companies, and governments use the internet for banking, shopping, saving private data, and running important services. As technology grows, cyber threats are also increasing. Hackers find new ways to break into systems, steal information, and cause damage (Tavani 2016; ENISA 2022).

One way to stop these attacks is through penetration testing. This means testing a system to find weak points before hackers do (NIST 2008). To do this, cybersecurity professionals use special tools.

Kali Linux is a popular system that has many tools for ethical hacking, digital forensics, and penetration testing. It is free, open-source, and based on Linux. It includes more than 600 tools, such as Nmap (used for network scanning), Wireshark (used to check traffic), and Metasploit (used to test for weaknesses) (Sharma, Rathore & Mathur 2019).

However, Kali Linux can be hard to use, especially for beginners. Many users do not know which tool to choose for a specific task. There are many tools with similar purposes, and it is not always clear which one to use (Alshaikh 2020). For example, someone trying to test a Wi-Fi network may see tools like Aircrack-ng, Reaver, and Wifite, but they may not know which is best or how to use them.

The Kali Linux website and many online guides only give short tool descriptions. They do not explain how tools are related, when to use them, or how they can be used together. This makes learning difficult, especially for students and junior cybersecurity workers (Kaur & Kaur 2017).

This project also aims to support learning. By helping users connect tasks to tools clearly, the system can build confidence, improve understanding, and make

tool selection easier. Beginners can learn how and why tools are used for specific cybersecurity activities, which helps improve their skills over time

To solve this problem, a knowledge-based system (KBS) can be helpful. A KBS uses smart ways of organizing knowledge, such as Ontology, which shows how tools are connected and what tasks they can perform (Gruber 1993; Studer, Benjamins & Fensel, 1998). For example, if a user says, "I want to scan a network," the system can suggest tools like Nmap and explain how to use them. This project will use Protege, free software used to build ontologies (Musen 2015), to create a smart system that helps users choose the right Kali Linux tools for their tasks.

1.2 Problem Statement

Cybersecurity problems are becoming harder to solve because attackers use smarter methods. Defenders need to use the right tools to stay ahead. Kali Linux is a popular system that has many useful tools (Sharma, Rathore & Mathur 2019), but many users especially beginners, find it hard to know which tool to use and how to use it.

The biggest problem is that Kali Linux lists the tools without much detail. The website shows only tool names and broad categories, like "Information Gathering" or "Exploitation" (Offensive Security 2024). It does not explain which tools work best for which tasks, or how they are related.

Some guides and lists try to help (Bhandari & Jain 2019), but they are often out of date and not very interactive. Most do not explain how tools work together or give suggestions based on what the user wants to do (Alshaikh 2020).

Other cybersecurity models like MITRE ATT&CK or the ontology by Oltramari et al. (2014) focus on describing attacks and threats, not on helping users choose tools.

Because of this, there is a need for a system that can:

- ✓ Suggest tools based on what the user wants to do,

- ✓ Explain how each tool works,
- ✓ Show how tools connect and work in order.

This project will build a system like this using Protege to create an ontology. It will help users find the best tools for tasks like scanning a network or testing Wi-Fi security.

1.3 Aim and Objectives

Aims:

To build a simple and smart system that helps users, especially beginners, choose and understand Kali Linux tools for different cybersecurity tasks.

To build an intelligent system that could aid in the selection of Kali Linux tools for different cyber security attacks thereby addressing the issue of complexity in Kali Linux tool selection for both penetration testing experts and non-experts.

Objectives:

To conduct an extensive literature review in a view to determine the-state-of-the-art in regard to Kali Linux tool selection models.

To design a system structure using UML diagrams for ensuring clarity to penetration testing experts, non-experts and future developers.

To develop a knowledge-based system that is equipped with intelligent reasoning capabilities for suggesting Kali Linux tools for defined cybersecurity attacks in real time.

To carry out system testing (functional and non-functional) by experts and non-experts.

To carry out an evaluation between the proposed knowledge-based system and the current models identified in objective 2 to justify the contribution of this research.

1.4 Project Scope

This project will focus on building an intelligent knowledge-based system that aids both experts and non-experts in selecting Kali Linux tools cybersecurity actions. It would aid students, experts and non-experts in selecting Kali Linux Tools in cyberspace.

Project deliverables will include:

- ✓ Study of at least 30 commonly used Kali Linux tools
- ✓ Grouping tools based on what they do.
- ✓ Creating an ontology using Protégé and OWL.
- ✓ Using smart reasoning to suggest tools for different tasks.
- ✓ Testing the system with cybersecurity learners.
- ✓ Writing and presenting a full report on the project.
- ✓ Not Included in the Project:
- ✓ Creating new cybersecurity tools.
- ✓ Running real-time penetration tests.
- ✓ Supporting tools outside Kali Linux.

1.5 Risks and Mitigation

Like any technical project, this work carries potential risks that could affect delivery, quality, or usefulness. Identifying these risks early helps in creating strategies to reduce their likelihood or impact. Risks for this project are categorised into technical, resource-related, and user adoption challenges.

Key risk factors include:

Limited availability of high-quality, up-to-date information on Kali Linux tools, which may affect ontology accuracy.

Difficulty in designing rules that accurately reflect complex tool-task relationships, leading to incorrect recommendations.

Technical compatibility issues between Protégé, OWL reasoning, and SPARQL queries.

Time constraints due to balancing academic deadlines and other commitments.

Low user participation during testing, limiting evaluation quality.

Mitigation strategies focus on early planning, incremental testing, and backup measures to ensure continuous progress.

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R1	Inaccurate or outdated tool data affecting recommendations	Medium	High	Use multiple up-to-date sources, including official Kali Linux documentation and active cybersecurity forums, and validate data with domain experts.
R2	Complex SWRL rule creation leading to incorrect reasoning results	Medium	High	Test rules incrementally, verify against known task-tool mappings, and use small datasets for debugging before scaling.
R3	Technical incompatibility between ontology components and reasoner	Low	High	Select stable versions of Protégé, OWL API, and Pellet reasoner; test in a controlled environment before full deployment.
R4	Limited user feedback during evaluation phase	Medium	Medium	Recruit testers early from cybersecurity student communities and online ethical hacking

				forums.
R5	Time constraints affecting quality and scope	Medium	High	Develop a detailed timeline with milestones; prioritise core ontology and reasoning features before adding enhancements.
R6	Loss of project files due to hardware or software failure	Low	High	Maintain regular backups on secure cloud storage and external drives.
R7	Ethical or legal issues in describing penetration testing tools	Low	Medium	Use academic and ethical guidelines to ensure descriptions focus on authorised and educational usage only.

Table 1: Project Risks and Mitigation Plan

1.6 Research Questions

The main question of this project is:

What is the-state-of-the-art in Kali Linux tool selection models / solutions?

How can a semantically developed knowledge-based system aid users in selecting Kali Linux tools in real time?

1.7 Target Users and Tool Categories

This project is for students, experts and non-experts in cybersecurity, penetration testers or ethical hackers and Teachers or trainers.

The system will include tools from the following areas:

Information Gathering

Exploitation Password

Cracking Wireless

Attacks Vulnerability

Scanning Forensics

1.8 Overview of Methodology

The project will use Protege to create an ontology that links tools to tasks. This ontology will be used to build a knowledge-based system. The steps include:

Studying and grouping tools based on what they are used for.

Building an ontology that shows tool names, categories, tasks, and relationships.

Adding reasoning rules to the system, so it can suggest the right tools when a user describes a task.

Testing the system with real users who will give feedback.

Improving the system based on what users say.

This method will help create a system that is simple, smart, and useful for those learning cybersecurity.

1.9 Gantt Chart

Project Management

Effective planning and scheduling are critical to ensuring the project's success within the seven-week window between **13 July 2025** and **29 August 2025**.

The Gantt chart (illustrates the timeline, sequencing, and overlap of tasks to ensure efficient use of time while allowing flexibility to manage unforeseen challenges.

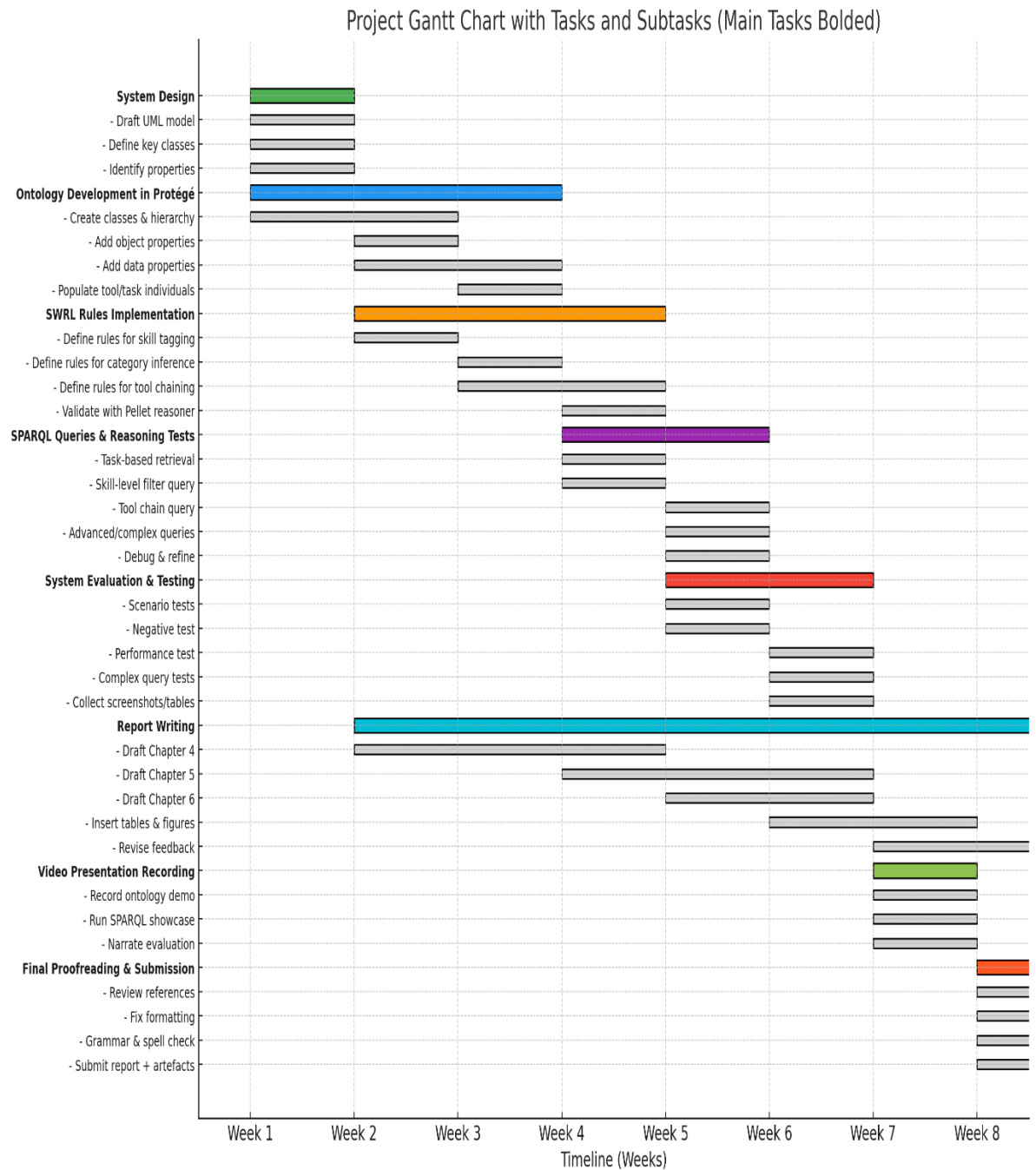


Figure 1: Project Gantt Chart by Weeks

Project Timeline Explanation

The schedule covers seven weeks, with tasks sequenced to build the knowledge-based system incrementally and in parallel with documentation:

Week 1 (13–19 July)

System Design

Draft UML model for ontology structure

Define key classes: Tool, Task, Category, UserLevel, SuggestedTool

Identify core object/data properties

Weeks 1–3 (13–31 July)

Ontology Development in Protégé

Create classes & hierarchy in OWL

Add object properties (isUsedFor, isFollowedBy, belongsToCategory)

Add data properties (hasCommand, hasDescription, requiresSkillLevel)

Populate initial tool/task individuals

Weeks 2–4 (20 July – 2 Aug)

SWRL Rules Implementation

Define rules for skill tagging

Define rules for category inference

Define rules for tool chaining (isFollowedBy → SuggestedToolChain)

Validate with Pellet reasoner

Weeks 4–5 (3–9 Aug)

SPARQL Queries & Reasoning Tests

Implement Task-based retrieval template

Implement Skill-level filter query

Implement Tool chain query

Implement Advanced/Complex queries (multi-constraint + ethics-aware)

Debug & refine query results

Weeks 5–6 (10–16 Aug)

System Evaluation & Testing

Run scenario tests (5.3.1–5.3.6)

Negative test (invalid mapping)

Performance test (query timing)

Complex query tests (multi-constraint, ethics-aware)

Collect screenshots/tables

User Testing & Feedback

Recruit beginner, intermediate, and advanced users

Provide 3 practical penetration testing tasks (network scanning, password cracking, vulnerability scanning)

Collect feedback on usability, clarity, and tool relevance

Weeks 2–8 (20 July – 29 Aug)

Report Writing

Draft Chapter 4 (Implementation)

Draft Chapter 5 (Evaluation & Testing)

Draft Chapter 6 (Conclusion & Future Work)

Insert tables, figures, screenshots

Revise based on feedback

Week 7 (17–23 Aug)

Video Presentation Recording

Record ontology overview demo

Run live SPARQL queries for showcase

Narrate evaluation highlights

Week 8 (24–29 Aug)

Final Proofreading & Submission

Review references (Harvard style)

Fix formatting (tables, figures, captions)

Final grammar/spell-check

Submit report + artefacts (ontology file, queries, video)

Linking Risks to Timeline

The project schedule has been designed to directly address the risks outlined in

Table 1: Project Risks and Mitigation Plan

R1 (Outdated tool data): Literature review and tool classification are embedded in early weeks, allowing time for validation with multiple sources.

R2 (Complex SWRL rules): Rule creation starts in Week 3, with incremental testing in Weeks 4–5 to minimise logic errors before final integration.

R3 (Technical incompatibility): Ontology development and reasoner tests begin early, enabling identification and resolution of compatibility issues before the evaluation phase.

R4 (Limited user feedback): Evaluation is scheduled for Weeks 5–6, giving time to recruit participants during earlier development stages.

R5 (Time constraints): Core ontology and reasoning functions are prioritised in the first five weeks, with enhancements only if time allows.

R6 (Data loss): Backups are planned after each major milestone (end of Weeks 2, 4, and 6).

R7 (Ethical/legal issues): Ethical and legal checks are embedded in system design and tool documentation from Week 1 onward, avoiding rework later.

This integration of risk mitigation into the timeline ensures that potential challenges are proactively addressed rather than reacted to at the last minute

1.10 Ethical and Legal Concerns

Using cybersecurity tools, especially those embedded in platforms like Kali Linux, requires strict adherence to ethical practices and legal regulations. While penetration testing (also known as ethical hacking) is a critical component of cybersecurity risk assessment, its execution must be governed by structured standards and clear boundaries. Without the proper legal framework or ethical guidelines, even well-intentioned testing can result in legal liabilities or professional misconduct.

Ethical Foundations

Ethical hacking operates on the principle of “do no harm.” It involves simulating cyberattacks to identify system vulnerabilities before malicious actors can exploit

them. However, even ethical hackers must operate within clearly defined rules to ensure their activities are not mistaken for malicious actions (Tavani, 2016).

To ensure ethical alignment, penetration testers should:

- ✓ Obtain informed consent.
- ✓ Define a clear scope.
- ✓ Document findings transparently.

Professional bodies such as (ISC)², ISACA, and EC-Council promote ethical codes of conduct. For instance, the EC-Council's Code of Ethical Conduct emphasises integrity, legality, and responsible use of technology (EC-Council, 2023).

Legal Frameworks and Standards

Laws governing penetration testing differ between countries but generally aim to prevent unauthorised access, protect private data, and respect digital ownership.

United Kingdom:

- Computer Misuse Act 1990: Criminalises unauthorised system access (Legislation.gov.uk, 1990).
- Data Protection Act 2018 / UK GDPR: Protects personal data during testing (ICO, 2023).
- Telecommunications Regulations 2000: Allows lawful system monitoring under defined conditions (Legislation.gov.uk, 2000).

International Examples:

- CFAA (USA): Prohibits unauthorised access to systems (U.S. Code, 2020).
- GDPR (EU): Sets strict rules on personal data processing (EU Parliament, 2016).

Before testing, written Rules of Engagement (RoE) must be agreed upon to define scope, permissions, and boundaries.

Technical Standards and Best Practices

Penetration testers should follow established standards, including:

- OWASP Testing Guide – Framework for secure web application testing.
- NIST SP 800-115 – US federal guidelines for security assessments (NIST, 2008).
- OSSTMM – Comprehensive security testing methodology (ISECOM, 2010).
- PTES – Covers preparation through to reporting (PTES, 2021).
- CREST Framework – UK certification and ethical testing standards.

Common Ethical and Legal Pitfalls

- Testing without written authorisation.
- Running tests on live systems without isolation.
- Collecting excessive or unnecessary personal data.
- Failing to report vulnerabilities transparently.

Powerful tools like Metasploit, Wireshark, and Hydra must be handled with care to prevent misuse.

Building Ethical Awareness into the System

This project embeds ethical and legal awareness into the knowledge-based system by:

- ✓ Adding ethical warnings and consent alerts.
- ✓ Mapping tools to approved use cases.
- ✓ Providing legal reminders on relevant regulations.
- ✓ Supporting optional logging and traceability.

Penetration testing is not only a technical discipline but also an ethical and legal responsibility. This project integrates safeguards to ensure that tool recommendations promote responsible cybersecurity practices aligned with the law.

1.11 Structure of the Report

This report is divided into the following chapters:

Chapter 2: Literature Review: Reviews existing work on Kali Linux tools,

knowledge-based systems, and ontologies in cybersecurity.

Chapter 3: Methodology: Describes the methods used to design and develop the ontology and knowledge-based system.

Chapter 4: System Design and Implementation: Covers the design process, development tools, and features of the proposed system.

Chapter 5: Testing and Evaluation: Presents the results of user testing and feedback analysis.

Chapter 6: Conclusion and Future Work: Summarizes the findings and suggests improvements for future development.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews what other researchers and professionals have done in the fields of cybersecurity tools, knowledge-based systems (KBS), and ontologies. It also examines how Semantic Web technologies can be used to improve the selection and understanding of Kali Linux tools. Many existing studies focus on threat detection, digital forensics, or penetration testing but do not provide intelligent guidance to help users especially beginners choose the right tools for specific cybersecurity tasks. This project addresses that gap by proposing an ontology-based, semantic system that links Kali Linux tools to real-world tasks.

2.2 Knowledge-Based Systems (KBS)

A Knowledge-Based System (KBS) is a computer program that uses expert knowledge to solve problems and support decision-making. According to Harmon and King (1985), a good KBS acts like a human expert, providing advice even in complex domains. In cybersecurity, KBS can help users, especially those new to the field, choose the correct tools for specific actions such as scanning a network or analyzing a breach. Older KBS systems were rule-based and difficult to update. However, the integration of semantic technologies has improved their flexibility and intelligence. As Studer et al. (1998) pointed out, ontologies allow KBS systems to adapt to new information and provide more accurate guidance.

2.3 Ontologies and the Semantic Web

Ontologies are structured representations of knowledge that define concepts and relationships. Gruber (1993) described an ontology as "a formal specification of

a conceptualization,” allowing both humans and computers to understand how concepts are related. In cybersecurity, ontologies can group tools based on their functions (e.g., password cracking or exploitation) and establish relationships between tools, tasks, and protocols. The Semantic Web extends this concept by allowing machines to interpret the meaning of information. Berners-Lee et al. (2001) describe it as a system that connects data meaningfully across the web.

Reasoning extends the ontology’s intelligence. By integrating semantic rule languages such as SWRL and reasoners like Pellet or HermiT, the system can infer hidden relationships, validate logic, and suggest tool chains.

These reasoners apply rules to check if the task makes sense and to find matching tools. For example, if a user says “scan the network and then test for weaknesses,” the system might recommend Nmap followed by Metasploit. This kind of reasoning helps users make better choices by linking tasks to tool combinations that match what they want to do.

In this project, Semantic Web principles are used to create smart tool recommendations based on user intent, not just keywords.

2.4 Protege as an Ontology Tool

Protege is an open-source tool developed by Stanford University for building ontologies using Web Ontology Language (OWL). Musen (2015) notes that Protégé helps structure domain knowledge using classes, subclasses, object properties, and individuals. In this project, it allows for the creation of a taxonomy of Kali Linux tools, connecting tools like “Nmap” to tasks such as “Network Scanning.”

While Protege is powerful for structuring data, it does not include built-in reasoning or user interaction features. As Horridge et al. (2004) suggest, adding reasoning capabilities and a user interface layer is necessary to make the system interactive and practical.

2.5 Kali Linux and Its Challenges

Kali Linux is a Debian-based operating system widely used by penetration testers and cybersecurity analysts. It includes over 600 tools for tasks such as network scanning, password cracking, and forensic analysis. However, as Kaur and Kaur (2017) note, the large number of tools can be overwhelming for new users who may not know which tool fits which task. Most online resources simply list tools with brief descriptions, which is not sufficient for effective tool selection. On top of everything, Kali Linux tools often lack user-friendly design. Beginners face a hard time because many tools are command-line only, menus are not clear, and there isn't much feedback when something goes wrong. This makes the learning process harder and slower. To fix this, it's important to think about how users interact with the system. Simple changes interact stand prompts, guided messages, and useful help tips can make a big difference. These User Experience (UX) improvements would help learners feel more confident and stay focused on the task without getting frustrated.

This project aims to improve this by building a task-based, intelligent guidance system for selecting Kali tools.

2.6 Existing Research and Gaps

Alghamdi et al. (2021) developed a comprehensive cybersecurity ontology using the Web Ontology Language (OWL). Their work focused on defining cyber threats, vulnerabilities, assets, and attack behaviors, helping both humans and machines interpret cybersecurity incidents more systematically. The model included terms such as "phishing attack," "malware injection," "ransomware," and "data exfiltration." The goal of their ontology was to improve threat detection, risk assessment, and situational awareness.

Deeper Critical Evaluation: While the ontology created by Alghamdi et al. (2021) offers strong theoretical value and supports threat classification, it does not address the practical needs of cybersecurity practitioners who must choose specific tools for different stages of penetration testing or digital forensics. The model is designed more for strategic analysis and documentation than operational use. It lacks links to tools like Nmap, Wireshark, or Metasploit, and provides no suggestions for how such tools might be selected based on a task such as network scanning or vulnerability exploitation. This makes it more suitable for research environments than practical, task-based tool selection. In contrast, the system proposed in this project will directly map user defined tasks to tool recommendations, supported by reasoning and real-world usage patterns.

Bhandari and Jain (2019) focused specifically on the Kali Linux ecosystem. Their work aimed to classify the tools bundled in Kali Linux into high-level categories. These included "Information Gathering," "Password Attacks," "Wireless Attacks," "Exploitation Tools," and "Forensics." Their approach provided structure to the overwhelming number of tools available in Kali Linux and helped reduce some of the complexity for new users.

Deeper Critical Evaluation: While the categorization system proposed by Bhandari and Jain (2019) was useful for organizing tools, it fell short in terms of functionality and adaptability. The classification was static, meaning it did not update dynamically as tools evolved or as new tools were added to Kali Linux. More importantly, it did not account for task sequences or tool relationships. For example, a user conducting a penetration test might need to use tools in a specific order, starting with reconnaissance and ending with reporting, but their system did not support this workflow. Furthermore, it did not consider user experience levels, environments (e.g., IoT, web apps), or provide explanations for why one tool might be preferred over another. The proposed ontology based system in this project addresses these limitations by using structured knowledge

and semantic relationships to suggest tools dynamically based on the user's task, context, and workflow stage.

Sharma et al. (2019) developed a web-based tool recommender system. The system allows users to enter keywords, and it returns a list of tools that might match. This approach is useful for intermediate users who already know the terminology or tool names. However, the recommender is limited by its reliance on keyword search, which does not consider the semantics of cybersecurity tasks. It does not support reasoning, such as suggesting a tool sequence based on task type or recognizing tool compatibility. Moreover, it cannot differentiate between tools with similar names but vastly different purposes or advise based on user experience level. In short, while the system offers a basic recommendation engine, it does not go beyond superficial matching and lacks task-awareness or contextual filtering.

Deeper Critical Evaluation: Sharma et al. (2019) attempted to simplify tool selection by introducing a web-based recommender system that accepts keyword inputs. While this approach is useful for users familiar with specific tool names, it has several limitations. Firstly, keyword-based searches assume prior knowledge, making the system unsuitable for novices who may not know the technical terms. Secondly, it lacks the ability to interpret user goals or context — for instance, it cannot distinguish whether the user is scanning a local network or testing a cloud environment. Furthermore, the system does not model the logical flow between tools (e.g., scanning before exploitation) or suggest tool chains. In comparison to the dynamic, context-aware systems seen in other AI-driven domains (like medical diagnosis or intelligent tutoring), Sharma et al.'s approach remains shallow in intelligence and scope. Therefore, while it provides a starting point, it does not bridge the task-tool-user gap needed in real-world cybersecurity training or practice.

Parasram (2023) contributed through his book titled "Digital Forensics with Kali Linux," which serves as a hands-on instructional guide for cybersecurity

learners. The book walks readers through practical examples involving popular tools like Autopsy, Volatility, Wireshark, and Sleuth Kit. It explains how these tools are used to investigate digital evidence such as hard drives, memory images, and network traffic.

Deeper Critical Evaluation: Parasram's guide is highly valuable for teaching practical usage of tools, especially for learners performing structured forensic tasks. However, it is limited in terms of automation, intelligence, and adaptability. It provides one-way guidance without feedback or personalization. The content is static, lacking a reasoning engine to suggest alternative tools, explain why one tool is better in a specific case, or build a tool chain for a complex investigation. Furthermore, it assumes the reader already knows the general flow of digital forensics and does not support dynamic decision-making during an investigation. Unlike Parasram's static guide, the system proposed in this project will use ontological knowledge and task awareness to make suggestions tailored to the user's context and evolving task needs.

2.7 Web-Based and Static Solutions

In addition to academic studies, several practical tools and web-based platforms have been developed to support users working with Kali Linux. These include documentation repositories, categorized lists of tools, video tutorials, blogs, and interactive cybersecurity training environments. Although these platforms serve as valuable resources for learners and professionals, they typically lack intelligent features that understand user intent or provide adaptive guidance based on real-world tasks. This results in a learning process that often depends on trial and error rather than structured knowledge and tailored recommendations.

The official tool listing website for Kali Linux, www.tools.kali.org, presents users with a categorized directory of tools bundled with the operating system. These tools are organized under categories such as "Information Gathering,"

"Vulnerability Analysis," "Wireless Attacks," "Web Applications," "Forensics," and others. Each tool entry typically includes the tool name, a short description, version information, and sometimes the terminal command to execute the tool. While this directory serves as a central reference point for tool availability, its utility is limited to browsing and does not support intelligent selection, comparison, or task-based filtering.

One significant limitation of such a static resource is its inability to understand the user's context or needs. For example, a beginner interested in finding live devices on a wireless network may not know which category to start with or how to differentiate between similar tools like Nmap, Netdiscover, or Angry IP Scanner. The list format requires prior knowledge of what each tool does and offers no support for comparing tools or recommending one over the other. It also fails to show tool compatibility, sequencing, or relevance based on task complexity or operational environment.

Many existing platforms do not include modern UX features. They list tools in static ways without guiding users step-by-step or adapting based on experience level or task. This makes learning harder, especially for beginners who need help to know what to do next.

At the same time, new technologies like OpenC2 and STIX show how cybersecurity is becoming smarter and more connected. These frameworks are mostly used for sharing threat information, but they show that using structured knowledge and automation is becoming more common. This supports the idea that a smart tool assistant—like the one in this project—is both needed and timely.

Interactive platforms such as TryHackMe and Hack The Box offer scenario-based cybersecurity learning environments. These platforms are structured around practical labs, challenges, and gamified exercises that simulate real-world cybersecurity attacks and defences. Users are guided through different cybersecurity phases such as reconnaissance, privilege escalation, and post-

exploitation using pre-configured virtual machines and scenarios. These platforms are effective in demonstrating how tools are used in practice and help users build hands-on skills within safe environments.

However, these platforms also have limitations when it comes to tool selection and guided decision-making. While users are introduced to various tools during the challenges, there is no built-in intelligence that recommends tools based on a described task. Users are expected to either know which tool to use or learn through trial and error. Additionally, these environments do not explain why one tool may be preferred over another in a given situation or how tool outputs can be combined in multi-step operations. This lack of semantic guidance can slow down the learning curve, especially for beginners.

Another issue with static resources is the lack of reasoning or workflow modelling. Cybersecurity tasks are often multi-staged and interdependent. For instance, a full penetration test might involve tools for reconnaissance, followed by scanning, exploitation, credential harvesting, and reporting. Each phase may rely on the outcome of the previous step, and choosing the wrong tool at any stage could reduce the effectiveness of the entire test. However, websites like tools.kali.org and many online tutorials treat each tool in isolation, without providing guidance on how they fit into a logical testing workflow or how they complement one another in practice.

Moreover, most web-based resources do not consider user-specific factors such as skill level, the nature of the target system (e.g., IoT, web application, or enterprise network), or the user's operating environment. A static list cannot distinguish between a student conducting a lab exercise and a professional carrying out an advanced network test. Without such contextual filtering, beginners may be overwhelmed with irrelevant or overly complex tools, while advanced users may find the recommendations too basic or incomplete for their use case.

In contrast, the knowledge-based system proposed in this project will leverage ontologies and semantic reasoning to overcome the shortcomings of these static and web-based solutions. It will allow users to describe their goals in natural terms (e.g., “scan for live devices,” “test Wi-Fi security”) and receive tool suggestions based on structured knowledge and defined relationships between tasks and tools. Furthermore, the system can include logic to suggest tool sequences (e.g., using Nmap for discovery before launching an exploit via Metasploit) and offer suggestions based on the user’s level of experience and the environment in which the task is being performed.

In summary, while platforms like tools.kali.org, TryHackMe, and Hack The Box offer valuable learning content and tool exposure, they are not sufficient for intelligent, user-centered tool selection. Their limitations include:

- No task-based guidance or adaptive recommendation;
- No semantic understanding of user goals or context;
- No explanation of tool compatibility or sequencing;
- No filtering based on user experience or environment;
- Minimal integration of structured cybersecurity knowledge and reasoning.

This reinforces the need for a dynamic, intelligent system like the one proposed in this project, which integrates task awareness, semantic relationships, and user guidance to make Kali Linux tool usage more accessible and effective—particularly for new learners and non-experts in cybersecurity.

2.8 Summary of Gaps

Many researchers and professionals have tried to help users better understand and use cybersecurity tools, especially those in Kali Linux. However, after reviewing both academic and practical works, they all have certain limitations. These limitations make it difficult for users especially beginners to choose the right

t tool for a specific task. This section summarizes the key gaps found in the reviewed work.

A simple table below shows what each study or platform focused on, and what important features they are missing.

Author / Project	Focus Area	Main Gap Identified
Alghamdi et al. (2021)	Cyber threat ontology using OWL	Describes threats and attacks but does not help users choose or use Kali Linux tools
Bhandari & Jain (2019)	Grouping of Kali tools	Organizes tools by category but does not show how tools are related or how to choose them for tasks
Sharma et al. (2019)	Web tool recommender based on keywords	Helps with tool search, but does not understand tasks, context, or tool relationships
Parasram (2023)	Practical guide for forensic tools	Good for learning, but has no smart system to guide tool selection based on tasks or reasoning
tools.kali.org	Official tool directory	Gives tool names and categories but no help with selection, task guidance, or tool comparison

Table 2: Summary of Gaps

Each of these sources does something useful:

1. Alghamdi et al. (2021) created a detailed model for understanding cyber threats. But it doesn't tell users what tools to use in practice.
2. Bhandari & Jain (2019) grouped tools to make it easier to find them, but they did not add explanations or reasoning about when and why to use one tool over another.
3. Sharma et al. (2019) built a search tool, but it works mostly on keywords. It cannot understand if a user is trying to scan a wireless network, test passwords, or collect logs. It just gives a list without smart suggestions.
4. Parasram (2023) wrote a very helpful book for students, but users still need to know which tool to use. The book doesn't include any automated way of choosing tools based on goals or tasks.
5. www.tools.kali.org, the official tool list, shows all the tools, but it does not give help to choose one. It does not give task-focused suggestions or explain how tools are related.

All these works focus on either teaching, listing, or describing tools. But none of them gives a smart, helpful way for users to say what they want to do and then get the right tool suggestions. They do not:

Understand user goals (e.g., "I want to test Wi-Fi security");

Show relationships between tools (e.g., scan with Nmap, then exploit with Metasploit);

Provide tool sequences for complete workflows.

Filter or explain tools based on skill level (beginner, intermediate, expert);

Adapt to changes or updates in Kali Linux over time.

These gaps show the need for a new solution. The proposed system in this project will fill these gaps using a knowledge-based system and ontology. The system will help users find tools based on tasks, explain how the tools work, and show how they connect in a process. This will make it easier, especially for beginners, to learn and use Kali Linux tools in the right way.

2.9 Conclusion

This literature review has explored a range of existing studies and practical resources related to cybersecurity tool selection, knowledge-based systems, and the use of ontologies in technical domains. The reviewed work highlights considerable progress in threat modeling, tool classification, and educational support. However, a detailed critical analysis has shown that most current solutions fall short in delivering a dynamic, intelligent, and user-centered tool selection system.

Academic works like those by Alghamdi et al. (2021) provide strong ontological foundations for describing cybersecurity threats but lack operational linkage to tools and real-world workflows. Efforts such as Bhandari and Jain (2019) help categorize Kali Linux tools but are limited by their static structure and lack of adaptive guidance. Sharma et al. (2019) contribute a searchable tool recommender, but its keyword-based method does not support semantic understanding or user-specific filtering. Practical instructional guides like Parasram (2023) offer valuable hands-on training but do not include intelligent or interactive support for tool decision-making.

In summary, there is a clear gap in the availability of a system that combines semantic reasoning, task-based tool matching, and adaptive guidance within the Kali Linux ecosystem. The proposed project addresses this by developing an ontology-driven knowledge-based system in Protégé. This system will connect user goals to tool functions using defined relationships, context awareness, and reasoning logic.

The next chapter outlines how this proposed approach will be designed and implemented, focusing on the ontology structure, reasoning mechanism, and the way it will support cybersecurity learners and professionals alike.

Chapter 3: New Ideas and Approach

3.1 Introduction

This chapter explains how the ontology was designed for the Kali Linux Tool Recommendation System. The ontology is the backbone of the system because it connects tasks (e.g., scan a network) with tools (e.g., Nmap, Netdiscover) and provides reasoning that helps beginners and experts choose tools in the right order.

The design was done in Protégé, where classes, properties, and rules were modeled step by step. The goal was to make the ontology simple, reusable, and powerful enough to suggest tools and tool sequences.

3.2 System Architecture Overview

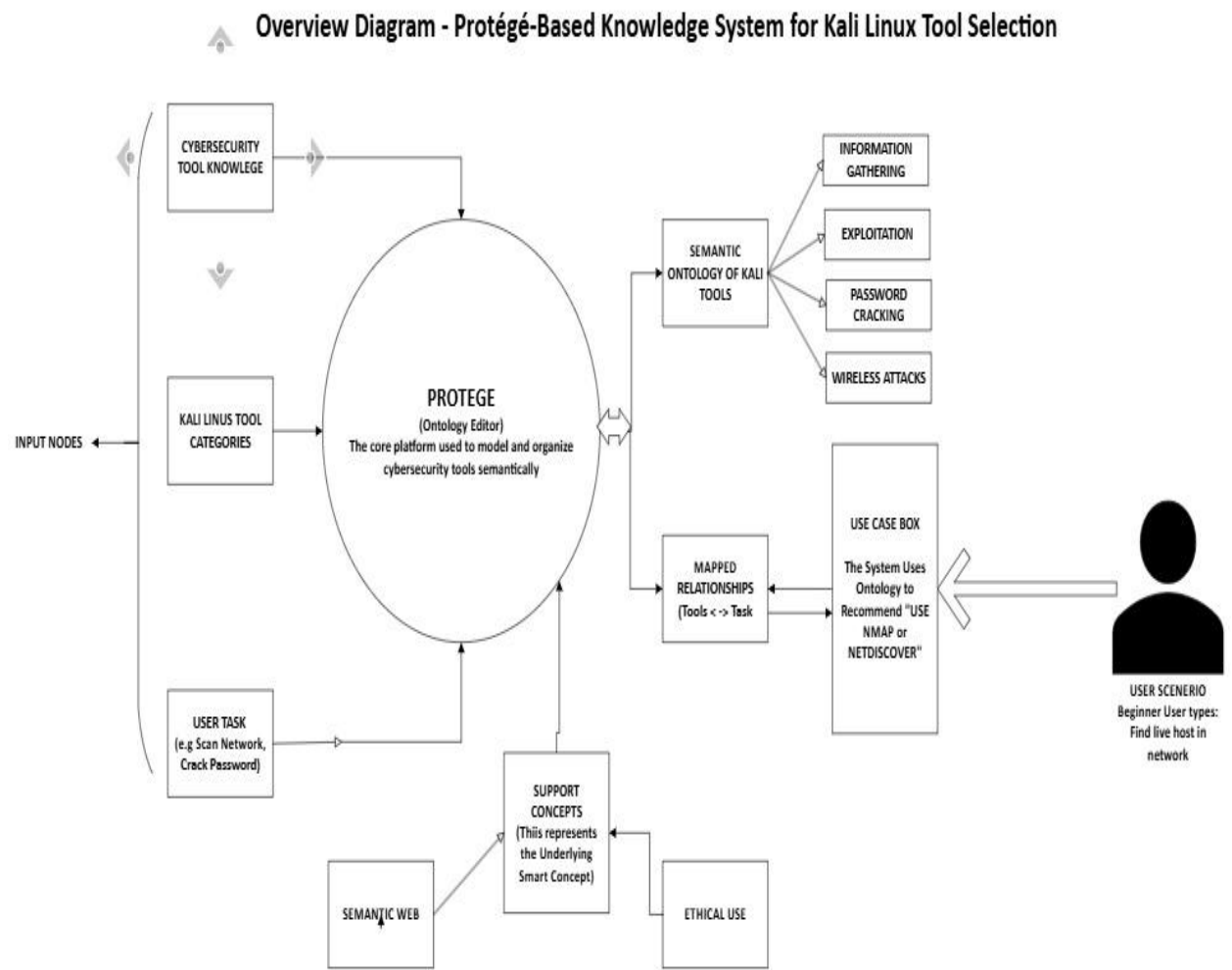


Figure 2: Overview Diagram of KBS for Kali Linux Tools Selection.

The system is built from several main components:

1. Input Sources: Tasks and tools from Kali Linux documentation and real penetration testing workflows.
2. Ontology Development in Protégé: Where tasks, tools, and commands were formally represented.
3. Tool-Task Mapping: Tools are linked to the tasks they perform. Example: Nmap -> NetworkScanning.
4. Reasoner: Uses rules and logic to suggest tools automatically.
5. Ethical Guardrails: Each tool includes warnings to remind users about legal and safe usage.

User Scenarios: For example:

- User says "find live hosts" -> Ontology suggests Nmap and Netdiscover.
- User says "crack WPA2 password" -> Ontology suggests Aircrack-ng and Reaver.

3.3 System Workflow

The operational process through which the proposed knowledge-based system (KBS) generates Kali Linux tool recommendations is illustrated in Figure 1: (System Architecture Overview) and detailed step-by-step in Figure 2: (Flowchart: Runtime Tool Recommendation Pipeline).

Figure 3.2 presents the high-level architecture, showing the key modules: Input Sources, Ontology Development in Protégé, Tool-Task Mapping, Reasoning Engine, and Ethical Guardrails and their interconnections. Building upon this architectural foundation, Figure 3.3 visualises the runtime sequence that occurs when a user submits a task request.

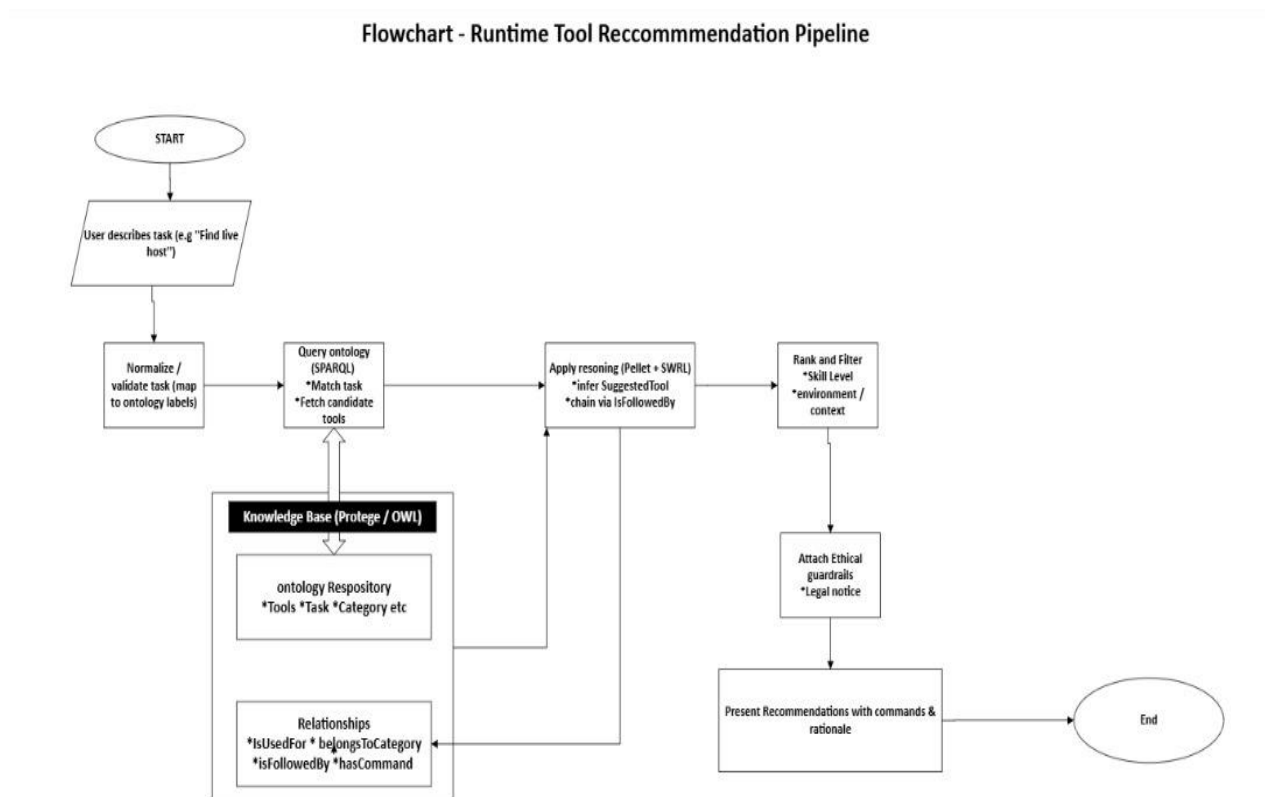


Figure 3: Flowchart of the Runtime Tool Recommendation Pipeline

The system follows a step-by-step process from user input to final recommendations:

- **User Task Input:** The process starts when a user (beginner or advanced) types a task in plain language, such as "Find live hosts" or "Crack password." This matches the Input Sources shown in Figure 1.
- **Task Normalisation and Validation:** The system standardises the user input and matches it to defined task labels in the ontology. For example, "Find live hosts" is mapped to the standard task Network Scanning. This step relates to the Ontology Development in Protégé phase in Figure 1.
- **Ontology Query (SPARQL):** The normalised task is used to query the ontology repository. SPARQL retrieves all tools connected to that task in the Tool–Task Mapping. For example, Network Scanning may return tools like Nmap and Netdiscover.
- **Reasoning (Pellet + SWRL):** The reasoning engine applies OWL reasoning and SWRL rules to find the most suitable tools. It can also suggest tool sequences using the `isFollowedBy` property, such as Run Nmap, then Nikto. This matches the Reasoning Engine block in Figure 1.
- **Ranking and Filtering:** The list of candidate tools is ranked and filtered based on factors like user skill level (Beginner, Intermediate, Advanced), environment (wired or wireless), and context. This ensures the final list is both relevant and practical.
- **Ethical Guardrails:** Before results are shown, the system attaches ethical and legal notices — reminding the user to only run tools in authorised environments and comply with laws like the UK Computer Misuse Act 1990.

Recommendation Output

Finally, the system presents the recommended tools with example commands and short explanations. For example:

Recommended: Nmap -> Command: nmap -sP 192.168.1.0/24. Reason:

Efficient host discovery.

By linking the system architecture (Figure 2) with the runtime workflow (Figure 3), this section gives both a design view and a step-by-step explanation of how the system works in practice.

3.3.1 Ontology Overview

The ontology forms the core knowledge representation framework for the proposed Kali Linux Tool Recommendation System. It models cybersecurity tools, tasks, categories, and user skill levels in a structured and machine-readable way, enabling reasoning and intelligent recommendations. The ontology is developed in OWL using Protégé, ensuring compliance with semantic web standards and supporting interoperability with other knowledge-based systems.

The top-level classes include Tool, Task, Category, UserLevel, and SuggestedTool.

1. **Tool** represents the actual cybersecurity tools available in Kali Linux, organised into functional subclasses such as NetworkScanningTool, PasswordCrackingTool, and VulnerabilityAnalysisTool.
2. **Task** defines the penetration testing or security assessment activities, such as Find Live Hosts or Crack WPA2 Password.
3. **Category** provides high-level classification for grouping tools according to their purpose.
4. **UserLevel** captures the skill level required to operate a tool effectively, categorised into Beginner, Intermediate, and Advanced.
5. **SuggestedTool** represents tool groupings or sequences recommended by the reasoning engine, often inferred using SWRL rules and object properties like isFollowedBy and SuggestedToolChain.

Semantic relationships between classes are defined using object properties such as `isUsedFor`, `belongsToCategory`, `requiresSkillLevel`, and `isFollowedBy`. Data properties such as `hasCommand` and `hasDescription` store executable commands and explanatory notes for each tool.

This ontology design supports advanced reasoning by combining explicit knowledge (manually entered facts) with inferred knowledge (generated through SWRL rules and Pellet reasoning). The structure ensures the system can dynamically suggest tools based on task descriptions, user skill levels, and logical tool sequences in penetration testing workflows.

3.3.2 Ontology Design Best Practices

To make the ontology consistent and easy to maintain, the following best practices were applied:

- **Disjoint Classes:** Each tool category is separate to avoid confusion.
- **Modularity:** Tools, tasks, and commands are grouped in modules so they can be reused in other systems.
- **Annotations:** Human-readable notes (e.g., `rdfs:comment`) were added for clarity.
- **OWL Restrictions:** Classes like `BeginnerTool` were defined using restrictions (e.g., tools that require beginner skill level).

This makes the ontology both machine-readable and user-friendly.

3.3.3 Components

The ontology is made up of several connected parts that together give a full picture of the Kali Linux tool ecosystem. These parts make the knowledge base both easy for humans to understand and usable by machines for reasoning and recommendations.

(a) Classes

Classes are the main categories of things in the ontology:

- Tool: Represents cybersecurity tools in Kali Linux. Tools can also be grouped into subclasses such as NetworkScanningTool or PasswordCrackingTool.
- Task: Represents activities in penetration testing, like Scan Network or Exploit Web Application.
- Category: Groups tools into broader areas, such as Information Gathering or Vulnerability Analysis.
- UserLevel: Shows what skill level (Beginner, Intermediate, Advanced) is needed to use the tool.
- SuggestedTool: Represents tools that are recommended in certain situations, often based on reasoning rules.

(b) Object Properties

These describe relationships between classes:

- isUsedFor: Connects a tool to the task it performs.
- belongsToCategory: Links a tool to its category.
- requiresSkillLevel: States what level of user can use the tool.
- isFollowedBy: Shows tool sequences, i.e., which tool logically comes after another.
- SuggestedToolChain – Suggests the next recommended tool in a workflow.

(c) Data Properties

These describe details about tools:

- hasCommand: Stores example commands for running a tool (e.g., command-line syntax).
- hasDescription: Human-readable explanation of what the tool does.
- hasEthicalNote (Literal): Brief responsible-use reminder (e.g., “use only on authorised scope”).
- hasLegalNote (Literal): legal caution or reference (e.g., GDPR/consent reminders).

- riskLevel (Literal): simple risk tag (e.g., Low, Medium, High).
- preferredPhase (Literal): recommended stage/phase where the tool is most appropriate (e.g., Reconnaissance, Vulnerability Analysis, Reporting).

(d) Individuals

These are the actual examples (instances) of the classes:

- Tools: Nmap, Netdiscover, Aircrack-ng.
- Tasks: Find Live Hosts, Crack WPA2 Password.
- Categories: Network Scanning, Password Cracking.

(e) Annotations

Annotations make the ontology easier to read and maintain:

rdfs:comment: Provides explanations or notes.

owl:versionInfo: Records version history.

All these components work together to make the ontology well-structured, extendable, and suitable for reasoning. This ensures that the system can recommend tools automatically in a way that is both logical and easy to follow.

3.3.4 Semantic Relationships and Tool Sequences

In real-world penetration testing, tools are often used one after another in a logical order. To represent this, the ontology uses two special links: isFollowedBy and SuggestedToolChain. These links show how one tool can lead to another, helping to model the actual workflow of security testing.

	From	isFollowedBy	Justification
1	Nmap	Netdiscover	Nmap scans hosts/services; Netdiscover refines this with ARP-based network discovery.
2	Nikto	OpenVAS	Nikto does fast web vulnerability scanning; OpenVAS performs deep assessments.
3	OpenVAS	Lynis	After OpenVAS, Lynis conducts host-level auditing for

			compliance and hardening.
4	Aircrackng	Airmonng	Aircrack-ng requires monitor mode, initialized with Airmon-ng.
5	Reaver	Wifite	Wifite automates the Reaver, Aircrack-ng process.

Table 3: FollowedBy Tool Sequence

In the ontology for Kali Linux tools, the SuggestedToolChain property is very important. It connects one tool to another that is usually recommended as the next step in penetration testing. This makes the system smarter, more automated, and easier to follow.

The idea reflects how security testing is done in real life, where tools are rarely used alone. Instead, they are combined in sequences for example, starting with network discovery, then moving to vulnerability scanning, and finally into exploitation or reporting.

The table below shows some examples of these logical progressions using the SuggestedToolChain property.

	FromTool	SuggestedToolChain	Justification
1	Nmap	Netdiscover	Nmap scans ports; Netdiscover can refine host discovery via ARP.
2	Nikto	OpenVAS	Nikto gives fast scans; OpenVAS runs deep scans.
3	OpenVAS	Lynis	Lynis audits hosts discovered vulnerable by OpenVAS.
4	Aircrackng	Airmonng	Airmon-ng is needed to set monitor mode for Aircrack-ng.
5	Reaver	Wifite	Wifite automates WPS and WPA attacks, including Reaver's.

6	Wireshark	Tcpdump	Wireshark provides visual inspection; tcpdump gives CLI granularity.
7	Metasploit	Sqlmap	Metasploit exploits services, leading to SQL injections with Sqlmap.
8	Sqlmap	Armitage	Armitage visualizes ongoing exploits and payloads.

Table 4: FollowedBy Tool Sequence

3.4 Ontology and Taxonomy Design in Protege

The main part of the proposed system is a taxonomy and ontology built with the Web Ontology Language (OWL) in Protégé. The taxonomy works as a hierarchical classification system. It groups Kali Linux tools into categories such as:

- Information Gathering
- Vulnerability Analysis
- Password Attacks
- Wireless Attacks
- Exploitation
- Forensics
- Sniffing & Spoofing
- Reporting

Ontology Elements:

Building on this taxonomy, the ontology defines relationships between tools, tasks, categories, and user levels. It includes:

Classes: Tool, Task, Category, UserLevel

Object Properties: "isUsedFor," "requiresSkillLevel," "belongsToCategory," and "isFollowedBy"

Individuals: Nmap, Hydra, Autopsy, etc.

Data Properties: "hasCommand," "hasDescription,"

Example:

Nmap isUsedFor NetworkScanning

Hydra isUsedFor PasswordCracking

Autopsy isUsedFor DigitalForensics

DigitalForensics requiresSkillLevel Intermediate

3.4.1 Tool Taxonomy and Command Integration

To support the ontology, a detailed taxonomy was developed. This taxonomy groups tools by their functions, adds short descriptions, and provides example commands. The design helps in two ways: it allows semantic reasoning in the system and also gives users practical guidance on how to run the tools from the command line.

This table is included in Chapter 3 because it is part of the knowledge modelling process. The focus here is on how the tools are organised and represented in the ontology. The actual system implementation is covered later in Chapter 4.

	Individual	Tool	Rdfs:Comment	Command
1	Lynis	VulnerabilityAnalysisTool	Security auditing tool	lynis audit system
2	OpenVAS	VulnerabilityAnalysisTool	Vulnerability scanner	openvas-start
3	Nikto	VulnerabilityAnalysisTool	Web server scanner	nikto -h http://example.com
4	Wireshark	SniffingSpoofingTool	GUI packet analyzer	Wireshark
5	Ettercap	SniffingSpoofingTool	MITM attacks and sniffing	ettercap -G

6	MITMf	SniffingSpoofing Tool	MITM attack framework	mitmf --arp --spoo --gateway x.x.x.x
7	MagicTree	ReportingTool	Pentesting data aggregation	java -jar magictree.jar
8	Dradis	ReportingTool	Reporting and collaboration platform	Dradis
9	Nmap	NetworkScanningTool	Scan for open ports	nmap -sS <target>
10	Netdiscover	NetworkScanningTool	Discover live hosts in network	netdiscover -r <IP_range>
11	Unicornscan	NetworkScanningTool	Asynchronous port scanner	unicornscan -Iv <target> :a
12	Airmon-ng	WirelessAttackTool	Enables monitor mode	airmon-ng start wlan0
13	Airodump-ng	WirelessAttackTool	Packet capture tool	airodump-ng wlan0mon
14	Reaver	WirelessAttackTool	Brute-force WPS PINs	reaver -i wlan0mon -b BSSID -vv
15	Cowpatty	WirelessAttackTool	WPA password cracking	cowpatty -r capture.pcap -f wordlist.txt
16	Aircrack-ng	WirelessAttackTool	Cracks WEP/WPA keys from captured packets	aircrack-ng capture.cap
17	Kismet	WirelessAttackTool	Wireless network detector and sniffer	Kismet
18	Binwalk	ForensicsTool	Firmware analysis tool	binwalk firmware.img
19	Volatility	ForensicsTool	RAM analysis	volatility -f

			framework	memdump.raw -- profile=Win7SP1x64 pslist
20	Autopsy	ForensicsTool	GUI-based forensic analysis platform	Autopsy
21	Sqlmap	ExploitationTool	Automates SQL injection exploitation	sqlmap -u "http://pasite.com" -dbs
22	Searchsploit	ExploitationTool	Offline exploit database search	searchsploit Apache
23	OWASPZAP	ExploitationTool	Web app security scanner	zap.sh
24	MSFVenom	ExploitationTool	Payload generation tool	msfvenom -p windows/meterpreter/ reverse_tcp LHOST=... -f exe > shell.exe
25	Armitage	ExploitationTool	GUI front-end for Metasploit	Armitage
26	BurpSuite	ExploitationTool	Web vulnerability scanner and proxy tool	Burpsuite
27	Metasploit	ExploitationTool	Comprehensive exploitation framework	Msfconsole
28	Xhydra	BeginnerTool	GUI-based password cracking	(GUI)
29	Wifite	BeginnerTool	Automated wireless attack tool	Wifite
30	Zenmap	BeginnerTool	GUI frontend for Nmap	Zenmap

Table 5: Tool Taxonomy and Command Integration

This pattern is repeated across other categories such as Password Cracking, Wireless Attacks, Web Application Testing, Exploitation, Forensics, Sniffing & Spoofing, and Reporting. In each case, the table connects a tool with its task and includes the command to run it. This makes the information useful both for beginners learning the tools and for professionals who need quick reference while working.

Ethical Note:

The ontology and taxonomy are designed strictly for lawful penetration testing and cybersecurity training. All examples assume that tools are used within authorised environments.

3.5 UML Class Diagram of the Ontology Structure

Figure 3 shows the UML Class Diagram of the ontology structure. It presents the main concepts Tool, Task, Category, UserProfile, and SuggestedTool and how they are connected through object and data properties. The diagram also includes the system parts that work with the ontology, such as the Reasoner, Query Service, and Ontology Repository. Together, these allow the system to run SWRL rules, execute SPARQL queries, and generate personalised tool

recommendations.

UML Class Diagram of the Ontology Structure for kali Linux Tool Recommendation

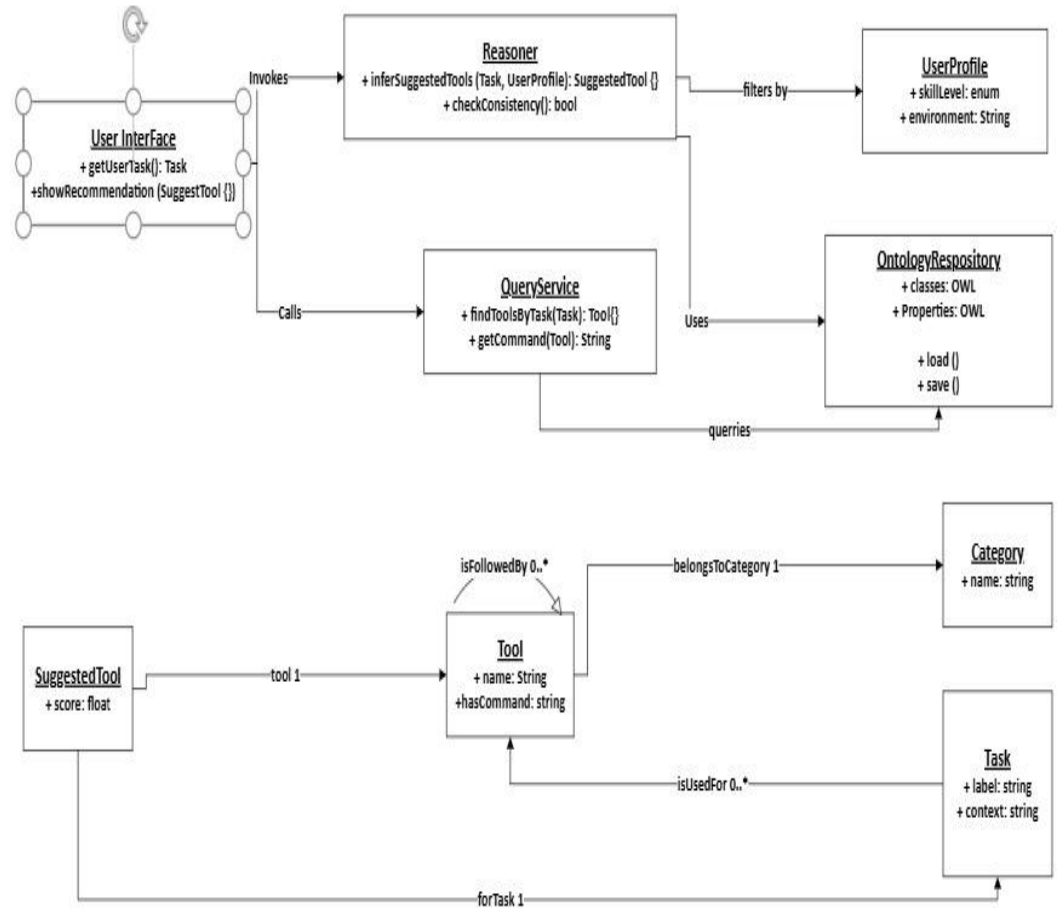


Figure 4: UML Class Diagram of the Ontology Structure

The UML class diagram shows the conceptual structure of the ontology. It highlights the main classes (Tool, Task, Category, UserProfile, SuggestedTool), the object properties (isUsedFor, belongsToCategory, isFollowedBy), and the data properties (hasCommand, hasDescription). It also illustrates how the Reasoner and Query Service work with the Ontology Repository to give ranked tool suggestions based on the user's task and skill level.

3.7 Professional, Social, Ethical and Legal (PSEL) Considerations

This project is shaped by professional, social, ethical and legal responsibilities, not just technical work. The ontology was structured around recognised penetration testing phases such as scanning, exploitation and post-exploitation. This reflects professional practice and makes the system relevant for learners and practitioners. A user skill level property was added so that beginners are not recommended advanced or harmful tools, which helps create a safe and guided learning path. SWRL rules were also written within the scope of authorised penetration testing tasks, avoiding any promotion of illegal or harmful activities. All commands and tool information were taken from open Kali Linux documentation, with no use of personal or sensitive data. The project was carried out in line with the UK Computer Misuse Act (1990) and GDPR. Linking these PSEL aspects to the design shows the system is both technically sound and responsibly developed.

Chapter 4: System Implementation

4.1 Introduction

This chapter details the practical implementation of the proposed Knowledge-Based System (KBS) for Kali Linux tool recommendations. The ontology was developed using Protege 5.x and implemented in OWL (Web Ontology Language) to capture relationships between tasks, tools, categories, and user skill levels.

Key implementation steps included:

- Creating the class hierarchy and defining ontology elements.
- Populating the ontology with individuals representing real tools, categories, and tasks.
- Linking classes through object properties.
- Defining reasoning rules using SWRL and retrieving recommendations via SPARQL queries.

This implementation phase transforms the conceptual model from Chapter 3 into a working semantic knowledge base.

4.2 Ontology Class Hierarchy

The ontology for the Kali Linux Tool Recommendation System is built using OWL (Web Ontology Language). It follows a clear, hierarchical structure that works well with semantic reasoners like Pellet. At the top of the hierarchy is the universal superclass owl:Thing, which all other elements both abstract concepts (classes) and real examples (individuals) inherit from.

This design makes the ontology serve two purposes:

1. A taxonomy for people: it helps users clearly understand and organise penetration testing tools into categories.
2. A knowledge base for machines: it enables automated reasoning, so the system can suggest tools dynamically based on the user's task, skill level, and context.

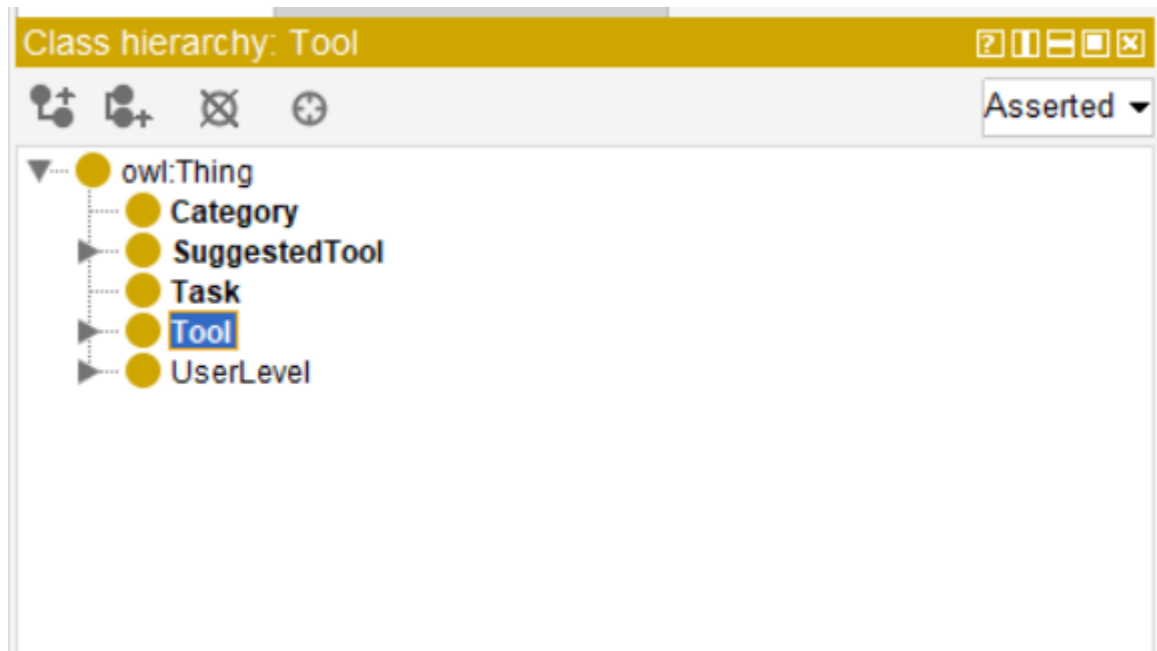


Figure 5: Class hierarchy of the implemented ontology in Protégé

4.2.1. Category

The Category class represents the main functional areas of penetration testing. It does not store tool commands directly but instead groups tools by their purpose and use.

The purpose of the Category class is to

- Organise tools according to their role in the testing lifecycle.
- Enable SPARQL queries to retrieve tools by function.
- Provide a human-readable way to browse and filter tools.

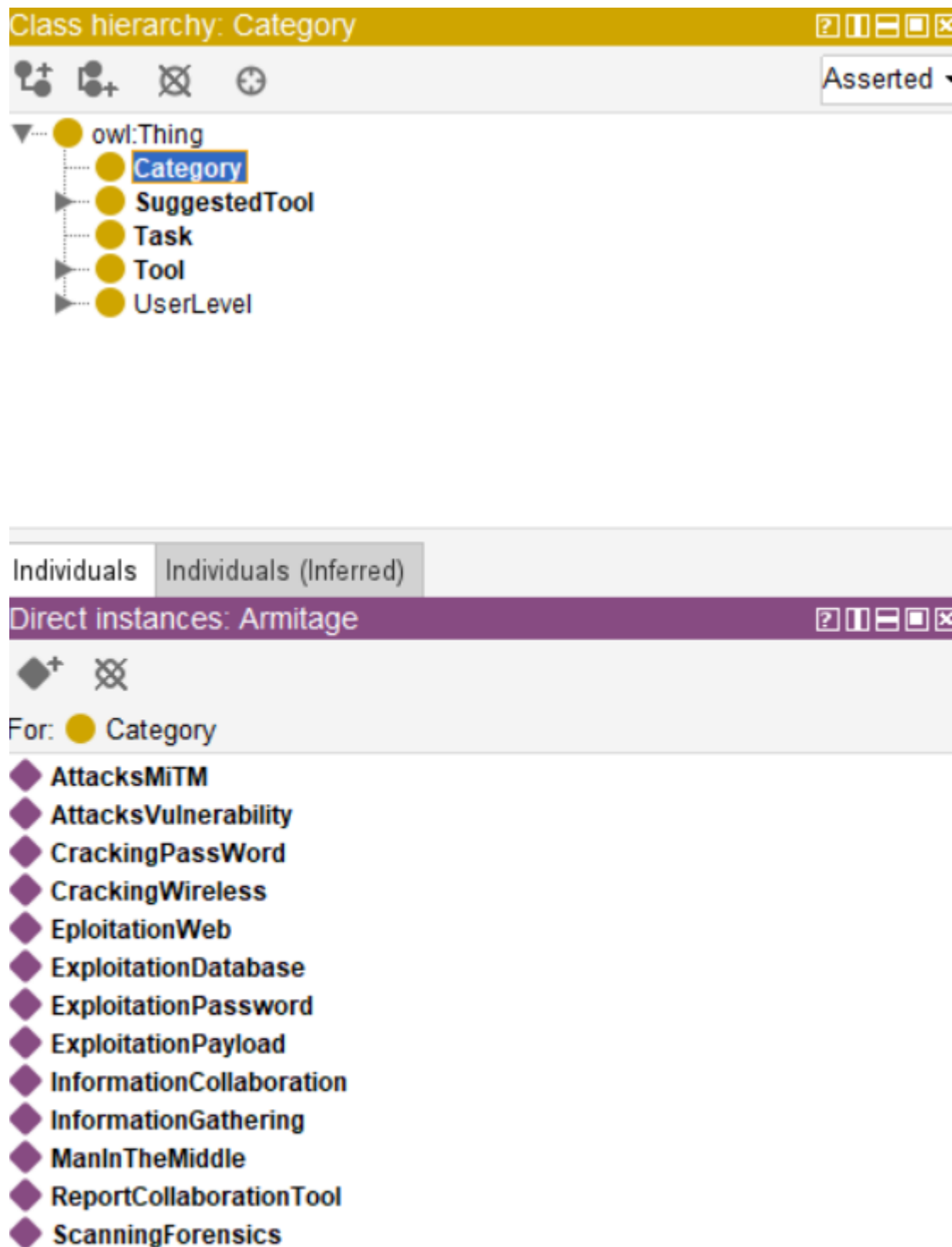


Figure 6: "Category" Class hierarchy of the implemented ontology

Subclasses:

Category has individuals that represent high-level testing domains.

- InformationCollaboration – Collaborative reporting and data exchange.
- AttacksVulnerability – Tools for exploiting known vulnerabilities.
- CrackingPassWord – Tools for password recovery and cracking.
- ScanningForensics – Forensic scanning and investigation tools.
- ManInTheMiddle – Tools for interception and traffic manipulation.

Each Tool individual is linked to exactly one Category through the belongsToCategory property. This guarantees that every tool is placed within a specific operational scope, avoiding overlap and making the ontology easier to query and reason over.

4.2.2. SuggestedTool: Class hierarchy of the implemented ontology

The SuggestedTool class stores recommendations that are generated dynamically. Unlike Category, which is fixed and predefined, SuggestedTool is filled at runtime using SWRL rules and ontology reasoning. This allows the system to suggest follow-up tools based on tasks, skill levels, or logical workflows.

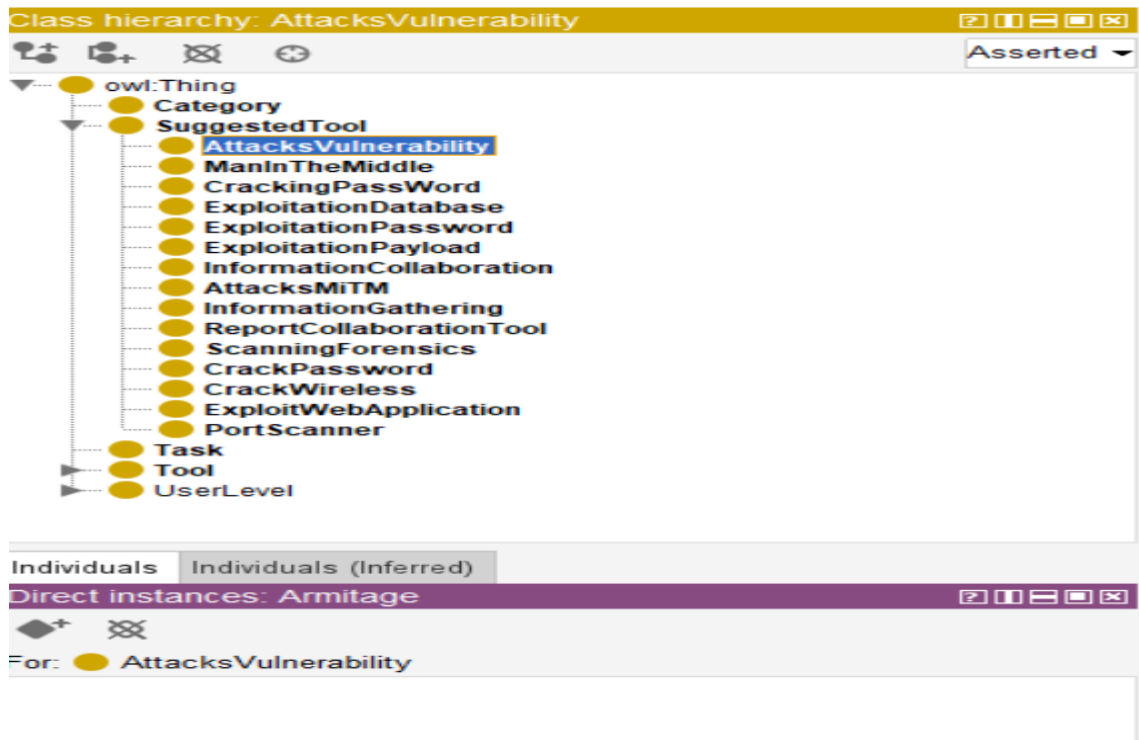


Figure 7: "SuggestedTool" Class hierarchy of the implemented ontology

4.2.3. Task

The Task class contains descriptions of specific penetration testing goals. These are usually written in verb-noun form to make them unambiguous and standardised.

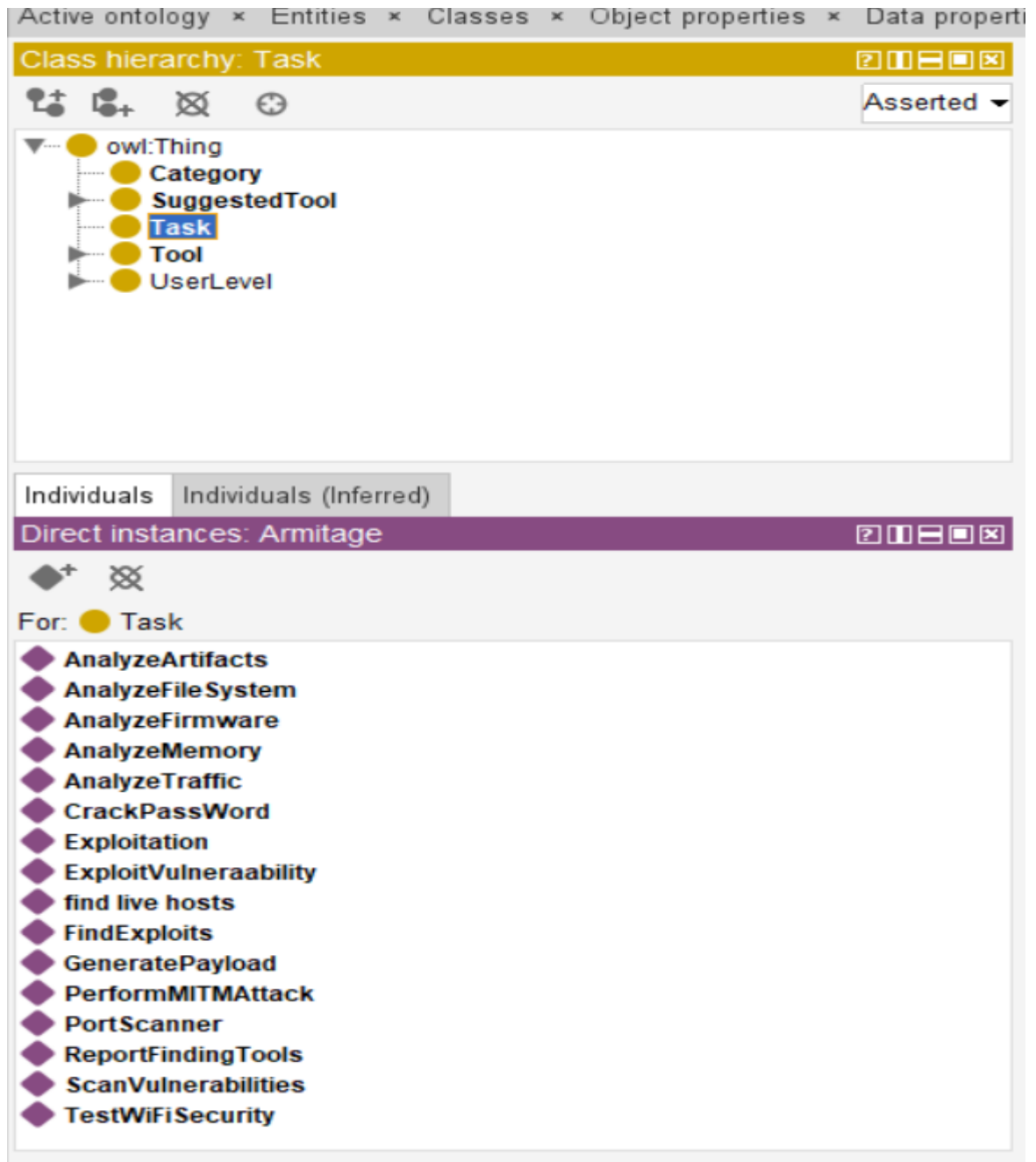


Figure 8: "Task" Class hierarchy of the implemented ontology

The SuggestedTool class provides context-aware recommendations that match the user's task. It is created dynamically when a task is entered, such as "Find live hosts." Reasoning uses properties like *isUsedFor*, *requiresSkillLevel*, and *isFollowedBy* to suggest tools and connect them into logical sequences. SuggestedTool also stores extra details, such as ranking scores, to prioritise results. Example inferred individuals include PortScanner for reconnaissance,

ExploitationDatabase for exploiting vulnerabilities, and InformationCollaboration for reporting. SuggestedTool differs from static categories because it adapts at runtime, linking user intent to tool capabilities through reasoning and enabling more intelligent, task-driven recommendations.

4.2.4. Tool

The Tool class contains the actual executable utilities available in Kali Linux. Unlike Category, which is a conceptual grouping, Tool represents concrete, operational software entities.

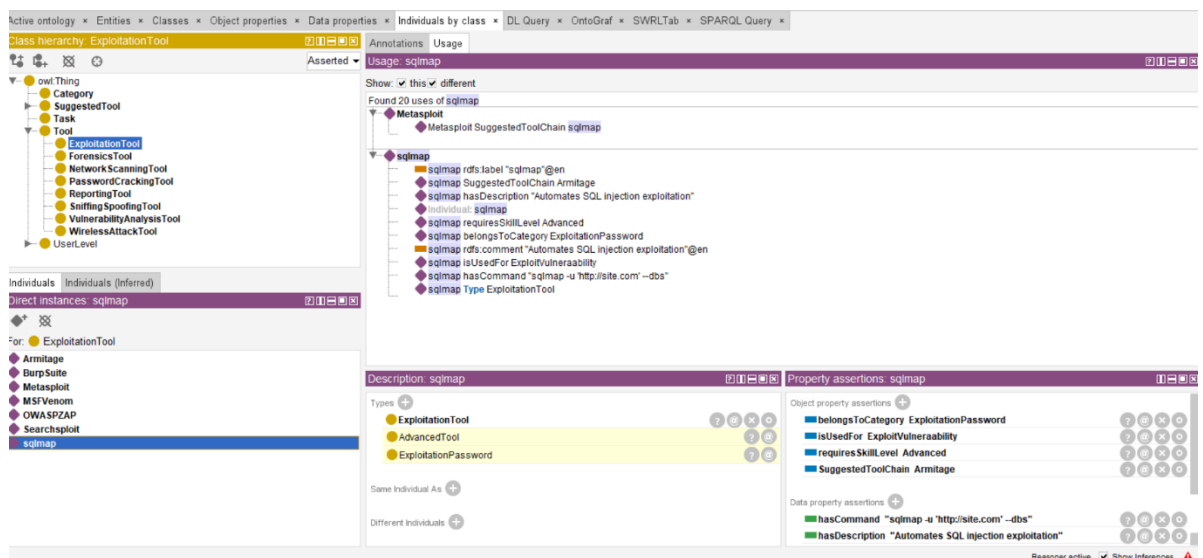


Figure 9: "Tool" Class hierarchy of the implemented ontology

The Purpose of Tool Class hierarchy is:

- Store execution details such as commands, descriptions, and usage contexts.
- Allow classification into functional subdomains via subclasses.

Subclasses (as per Protégé hierarchy):

- InformationGathering: Reconnaissance and intelligence-gathering tools.
- VulnerabilityAnalysisTool: Detecting and assessing system vulnerabilities.
- WirelessAttackTool – Attacking and auditing wireless networks.
- ForensicsTool: Investigating and analysing digital evidence.
- ExploitationTool: Tools for exploiting discovered vulnerabilities.

- SniffingSpoofingTool: Interception and spoofing of network traffic.
- ReportingTool: Generating reports and visualisations.

Example Individuals:

- Nmap: Network scanning and reconnaissance.
- Aircrack-ng: Wireless key cracking.
- Autopsy: Forensic analysis of file systems.

4.2.5. UserLevel

The UserLevel class groups users by their technical expertise so that tool recommendations are safe and appropriate. It prevents beginners from being given complex or risky tools, while still supporting advanced users with powerful options. The class has three individuals: Beginner (easy-to-use tools with low risk), Intermediate (tools requiring some cybersecurity knowledge), and Advanced (complex tools for experienced testers). In practice, a user's task is mapped to ontology labels, queried with SPARQL, and processed by Pellet and SWRL to suggest tools. Filtering by UserLevel ensures that recommendations match both the task and the operator's skill.

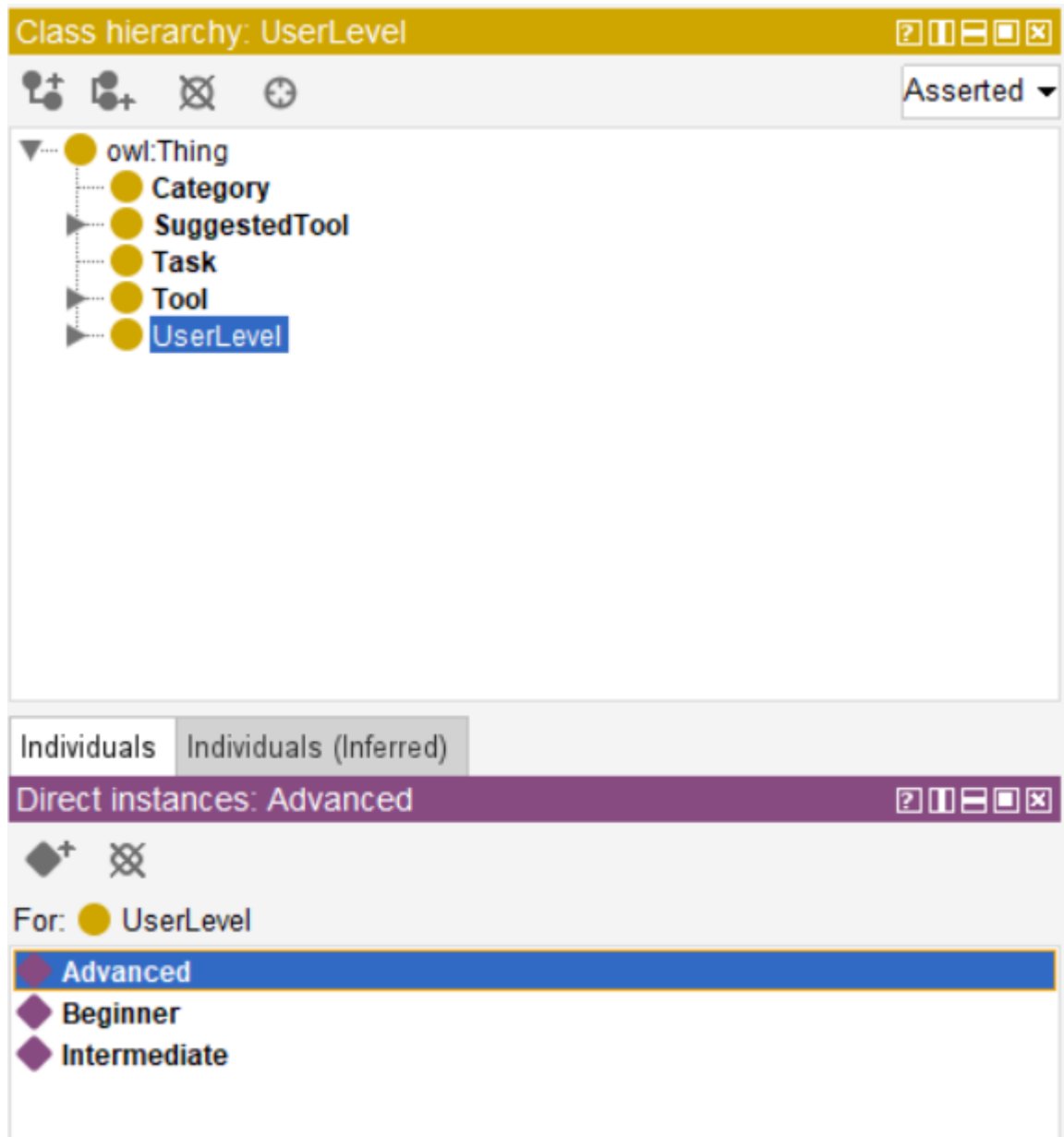


Figure 10: "UserLevel" Class hierarchy of the implemented ontology

4.3 Category Class and Individuals

The Category class provides high-level groupings for penetration testing tools. It is a direct subclass of owl:Thing and works as a conceptual layer that organises tools according to their main role in cybersecurity. Unlike subclasses of Tool, which represent specific types of tools, Category holds named individuals that act as reusable labels. These individuals are referenced in object property assertions, making them essential for both classification and reasoning.

Purpose: Categories enable semantic organisation of tools, support reasoning through the belongsToCategory property, and allow SPARQL queries to retrieve tools by function.

Individuals: Examples include InformationGathering (Nmap), CrackingWireless (Aircrack-ng), InformationCollaboration (Dradis), and ExploitationWeb (web attack tools).

By linking tools to categories, the ontology ensures that every tool is tied to its functional context. This improves clarity for browsing, supports accurate queries, and helps generate relevant recommendations during reasoning.

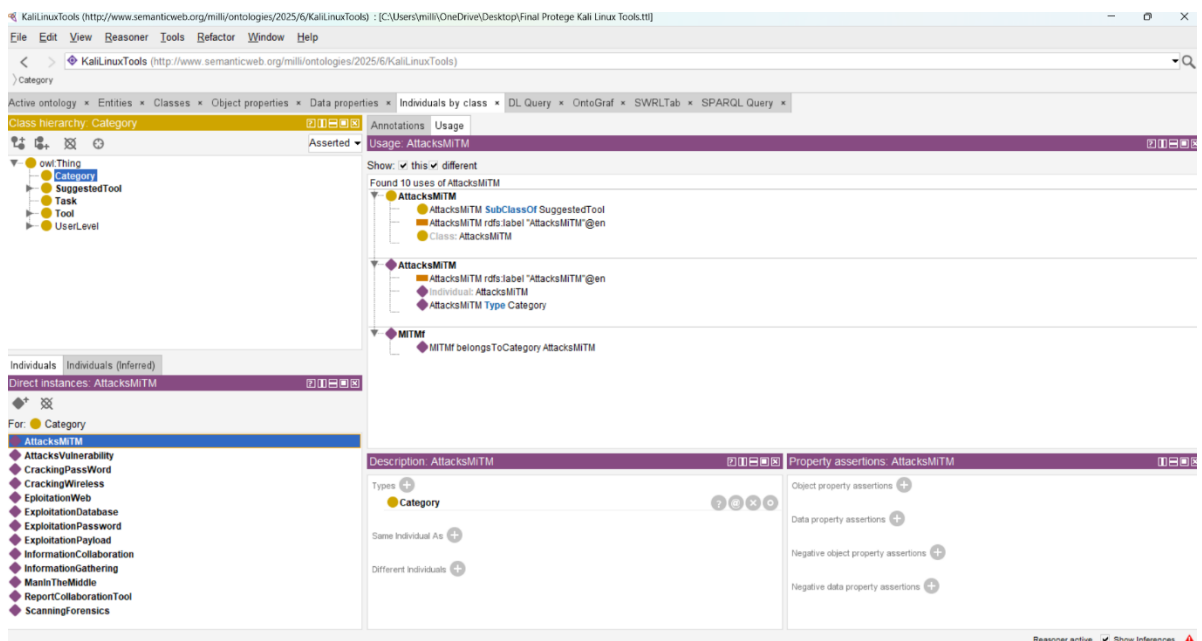


Figure 11: Category Class and Individuals

4.4 Object Property Linking

Object properties define the relationships between tools, tasks, categories, and user levels, forming the reasoning backbone of the ontology. They allow the system to recommend tools for tasks, arrange them in workflows, and filter results by skill level.

- **isUsedFor** links a tool to its function, e.g., Nmap -> network scanning, Aircrack-ng -> wireless cracking.

- **belongsToCategory** groups tools under categories, e.g., Dradis -> InformationCollaboration, Wireshark -> SniffingSpoofing.
- **isFollowedBy** models workflows, e.g., Nmap -> Nikto after host discovery.
- **requiresSkillLevel** filters tools by expertise, e.g., Zenmap -> Beginner, Metasploit -> Advanced.
- **SuggestedToolChain** supports recommended tool sets, e.g., Nmap, Netdiscover, Unicornscan.

Together, these properties enable meaningful reasoning and safe, context-aware recommendations.

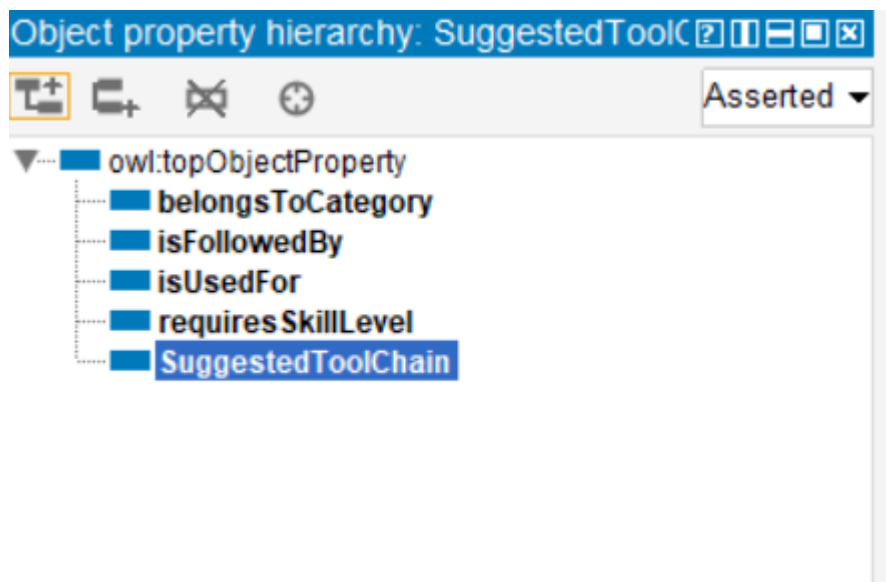


Figure 12: Object Property Linking

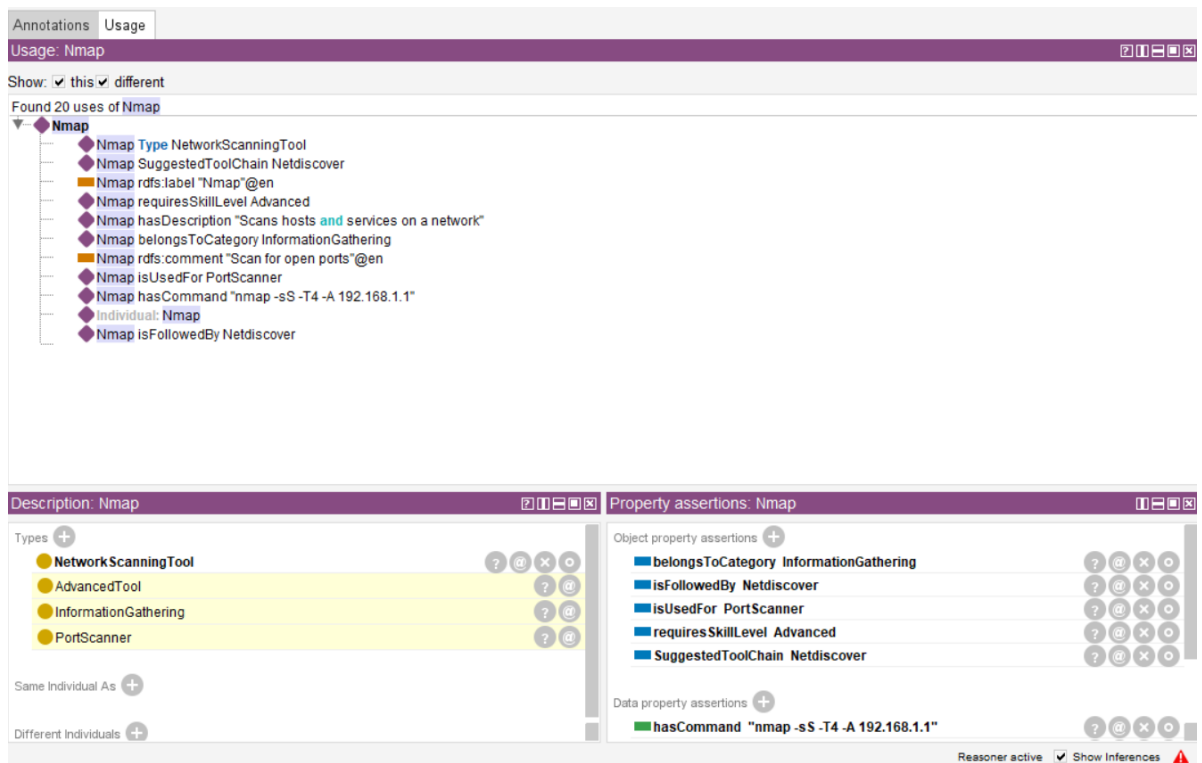


Figure 13: Object Property Usage in Protégé

4.5 Data Properties

While object properties link tools and tasks, data properties connect individuals to literal values such as text, numbers, or commands. They enrich recommendations by providing actionable details and explanatory context.

- `hasCommand` stores execution syntax, e.g., Nmap -> `nmap -sP 192.168.1.0/24`, Aircrack-ng -> `aircrack-ng capture.cap`.
- `hasDescription` provides human-readable explanations, e.g., Wireshark -> "GUI-based packet analysis tool," Hydra -> "fast logon cracker supporting multiple protocols."

By combining commands and descriptions, recommendations become both informative and practical. Users not only see which tool to use but also how and why. Additional metadata (e.g., version, author) can be added for advanced learners, ensuring clarity, usability, and stronger alignment with real-world practice.

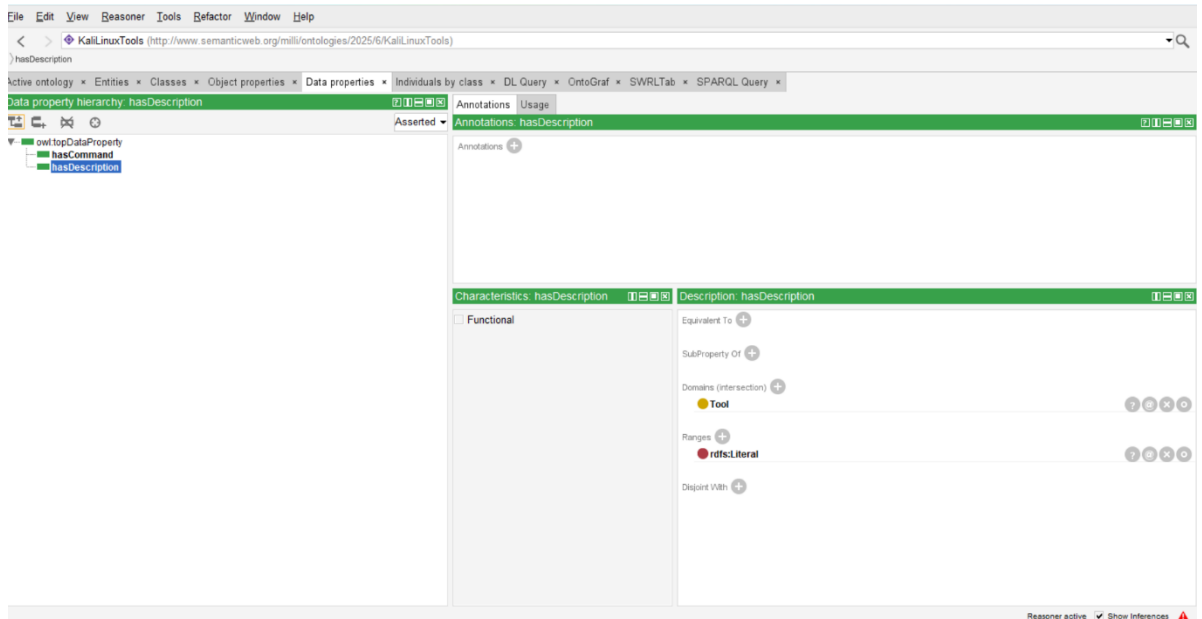


Figure 14: Data Property Assertions in Protégé

4.6 Reasoning and SWRL Rules Implementation

The ontology's reasoning is handled by the Pellet reasoner in Protégé, supported by Semantic Web Rule Language (SWRL) rules. This combination allows the system to generate SuggestedTool recommendations dynamically, using the task provided by the user, their UserLevel, and the links created through object and data properties.

4.6.1 Role of the Pellet Reasoner

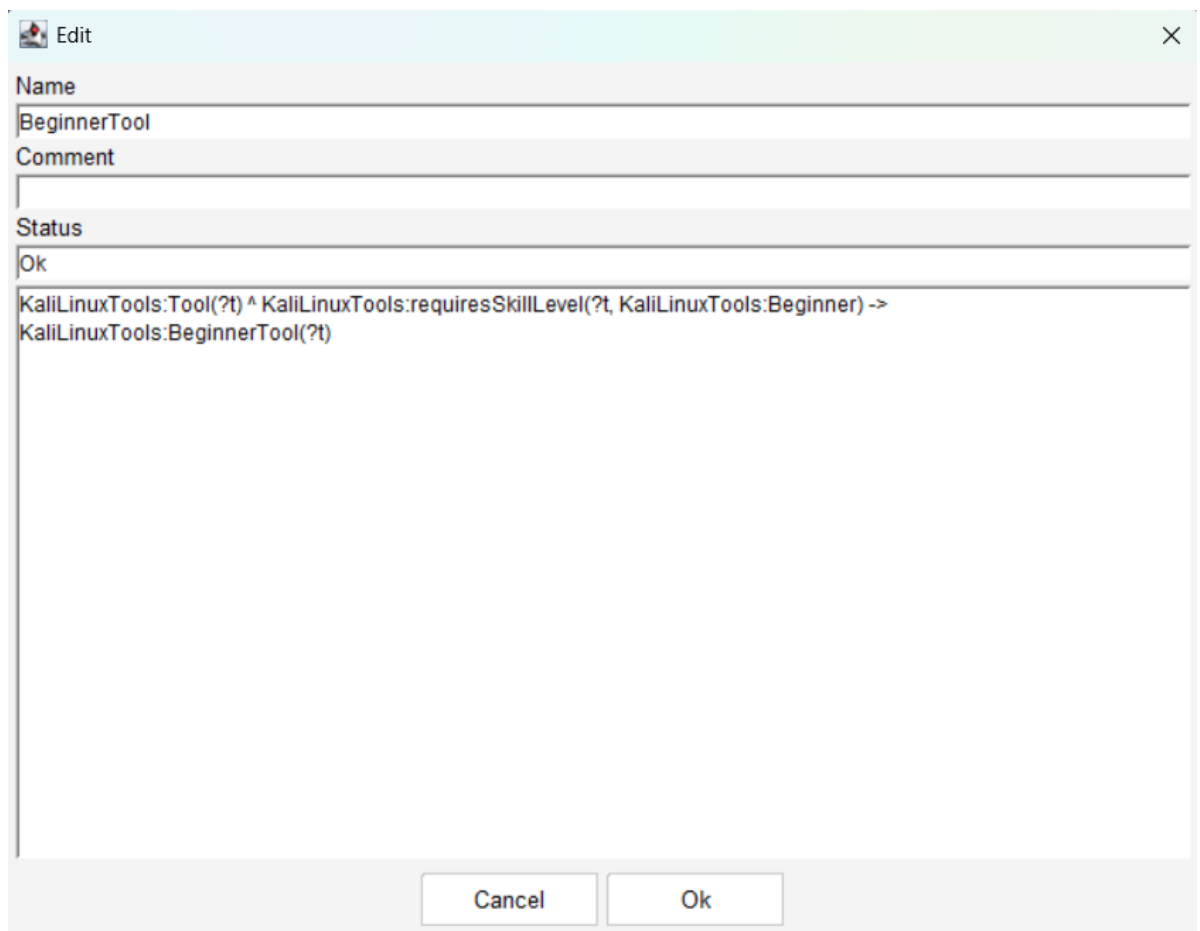
Pellet examines the ontology's axioms, class hierarchy, and property assertions to make the knowledge base smarter. It can infer class memberships, such as categorising a tool as a SuggestedTool when rules match. It also validates consistency, ensuring there are no logical conflicts like a tool being both Beginner and Advanced. Finally, Pellet expands SPARQL queries so results include both explicitly stored facts and new knowledge inferred through reasoning.

4.6.2 SWRL Rule Structure

SWRL rules add intelligence to the ontology by letting it create new knowledge from existing facts. A rule has two parts: the if-part (condition) and the then-part (result). If the condition is true, the system infers the result automatically.

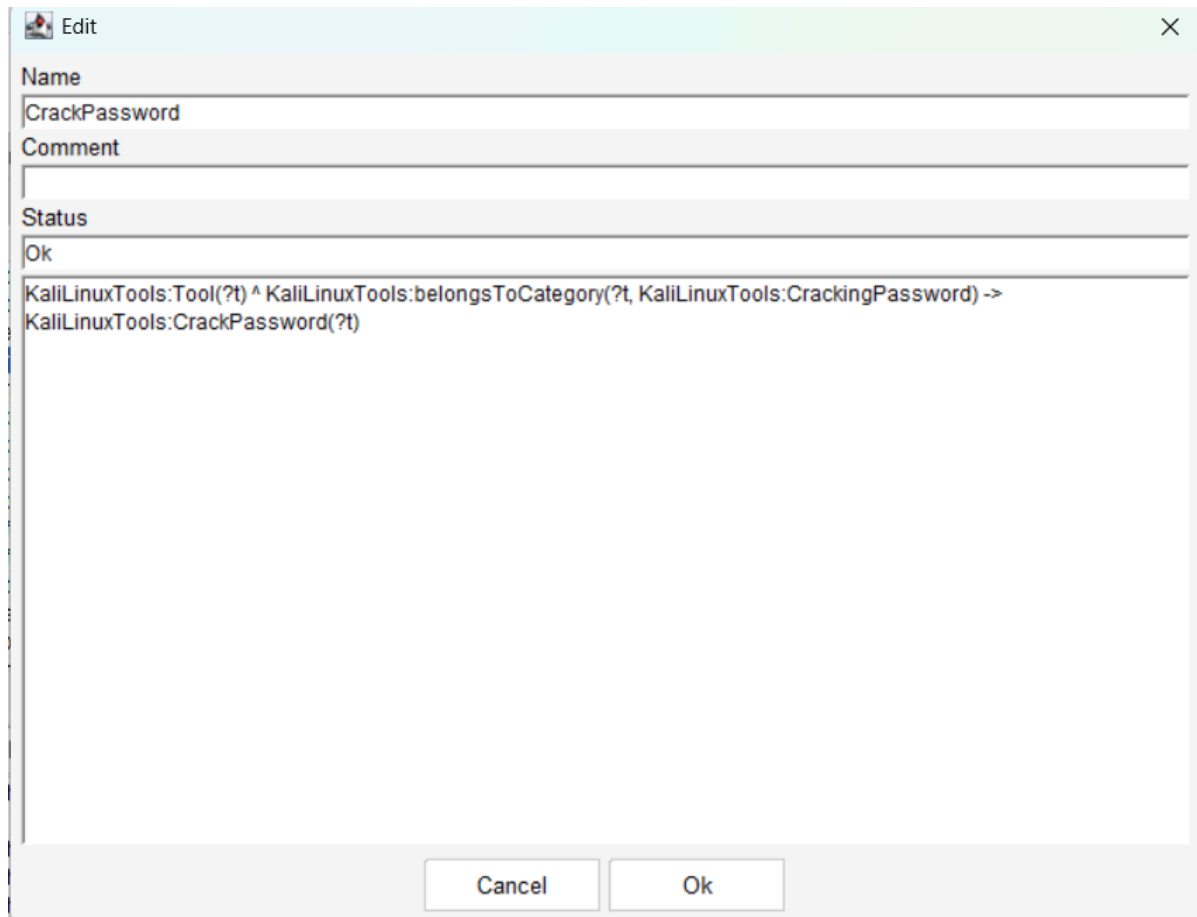
- Example 1 – User Level Filtering (Figure 15):
If a tool has the property `requiresSkillLevel = Beginner`, then the system classifies that tool as a `BeginnerTool`.
- Example 2 – Category Classification (Figure 16):
If a tool belongs to the category `CrackingPassword`, then it is classified as a `CrackPassword` tool.

These rules help the system recommend tools by linking skills, categories, and commands in a logical and useful way.



The screenshot shows a software window titled "Edit" with a close button (X) in the top right corner. The window contains three input fields: "Name", "Comment", and "Status". The "Name" field is filled with the text "BeginnerTool". The "Status" field contains a SWRL rule: `KaliLinuxTools:Tool(?t) ^ KaliLinuxTools:requiresSkillLevel(?t, KaliLinuxTools:Beginner) -> KaliLinuxTools:BeginnerTool(?t)`. At the bottom of the window, there are two buttons: "Cancel" and "Ok".

Figure 15: SWRL Rule Inferring Beginner Tools



Dialog box titled "Edit" with the following fields:

- Name:** CrackPassword
- Comment:** (empty)
- Status:**

```
Ok
KaliLinuxTools:Tool(?t) ^ KaliLinuxTools:belongsToCategory(?t, KaliLinuxTools:CrackingPassword) ->
KaliLinuxTools:CrackPassword(?t)
```

Buttons: Cancel, Ok

Figure 16: SWRL Rule Inferring Password Cracking Tools

4.6.4 Runtime Execution in the System

The runtime execution process explains how the system gives tool recommendations. It starts when a user enters a task in plain language, which is matched to a Task in the ontology. A SPARQL query then retrieves tools linked to that task through `isUsedFor`. The Pellet reasoner and SWRL rules refine the results by applying skill-level filters and tool chains. Finally, the system adds `hasCommand` and `hasDescription`, so users get accurate, ready-to-use commands with clear explanations.

4.6.5 Example Reasoning Output

To show how reasoning works in practice, a test was done with the task "Find live host." The ontology matched this to the Port Scanning task. Using SWRL

rules and Pellet inference, the system checked which tools should be SuggestedTool. The reasoner correctly inferred that Nmap and Zenmap are SuggestedTools for host discovery.

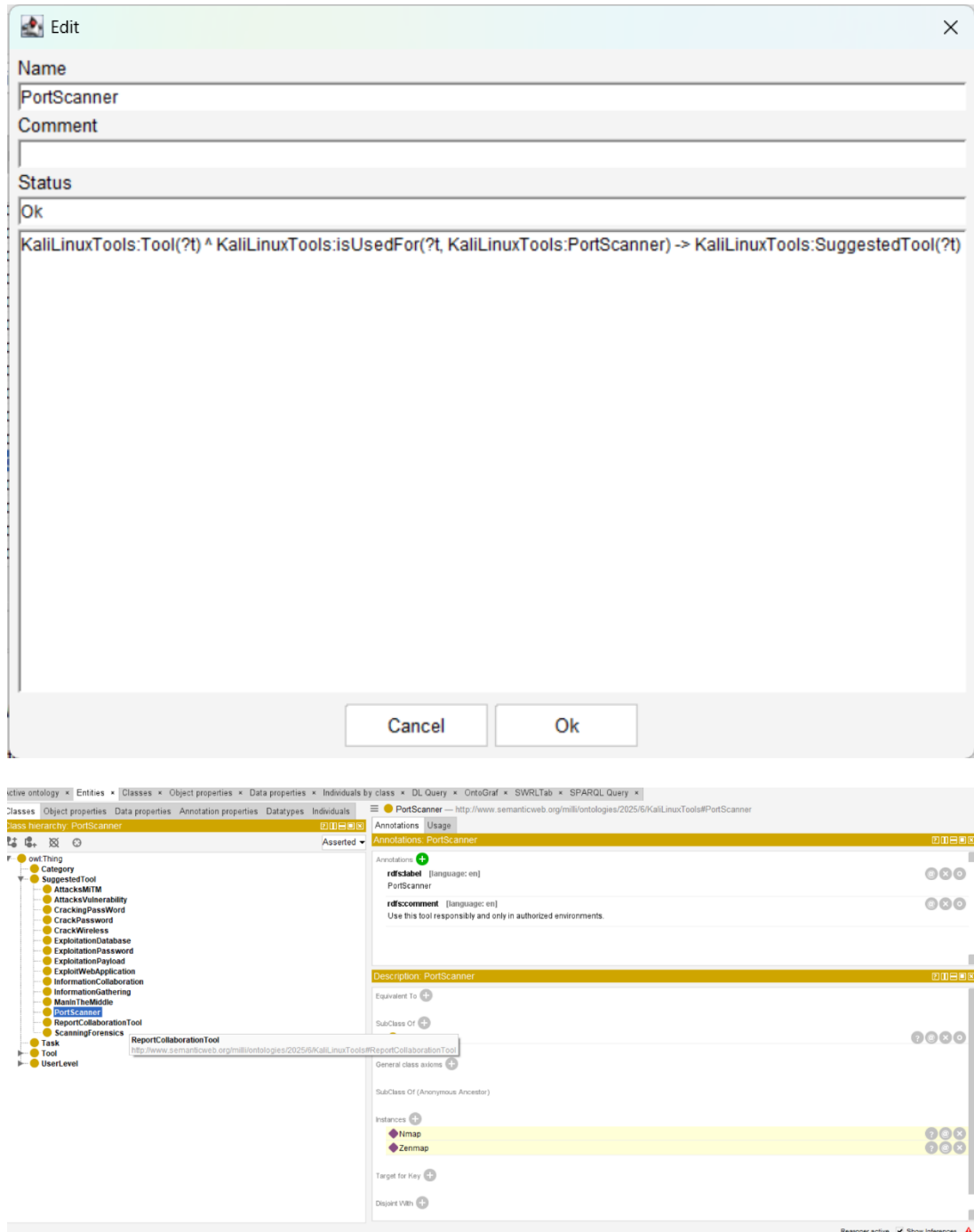


Figure 17. Example Reasoning Output

This example shows how object properties (isUsedFor) and data properties (hasCommand, hasDescription) combine with rules to give smart

recommendations. The output was not only correct but also useful, since it included commands and explanations for direct use.

4.7 DL Queries

This section shows how the DL Query tab in Protégé (Manchester syntax) with the Pellet reasoner was used to test richer class expressions. The queries check category filters, user-level filters, task-to-tool mappings, and simple tool-chain logic. They complement the SPARQL templates by giving fast, reasoner-based checks inside Protégé.

Setup: In Protégé, open DL Query, start Pellet, and use the exact entity names from the ontology. Use value when pointing to an individual (e.g., CrackPassWord, Beginner, OpenVAS, CrackingWireless etc) and use some when pointing to a class.

Queries used (with outcomes).

- Query A

Tool and requiresSkillLevel value Beginner

Outcome: lists all beginner-level tools.

- Query B

Tool and belongsToCategory value CrackingWireless and requiresSkillLevel value Advanced

Outcome: lists advanced wireless-attack tools.

- Query C

Tool and (isFollowedBy value OpenVAS or isFollowedBy some (Tool and isFollowedBy value OpenVAS))

Outcome: lists tools that reach OpenVAS in ≤ 2 steps.

- Query D

Tool and isUsedFor value CrackPassWord

Outcome: task-to-tool mapping for password cracking.

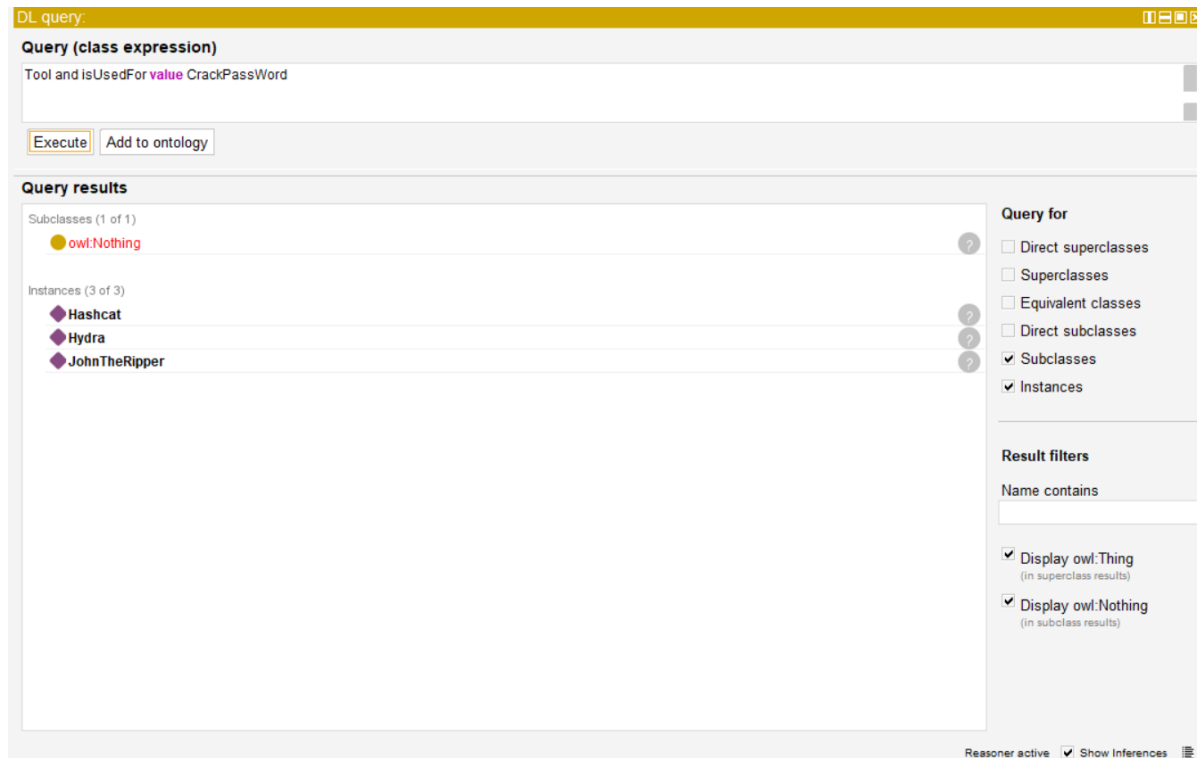


Figure 18. Example DL queries in Protégé with Pellet active (Query D shown)

4.8 SPARQL Query Templates for Tool Recommendation

SPARQL is the main query language used to fetch tool recommendations from the ontology. It works with Pellet reasoning and SWRL rules to return both defined and inferred tools. This approach ensures outputs are accurate, context-aware, and tailored to user tasks and skill levels.

4.8.1 Querying by Task Label

This query template retrieves tools linked to a chosen task label. For example, if the task is CrackPassword, the query returns:

- Tool name (rdfs:label)
- Tool description (hasDescription)
- Command syntax (hasCommand)

The template can be adapted by changing the FILTER expression (e.g., PortScanning or ExploitWebApplication). This makes the system flexible, allowing task-based recommendations for different penetration testing activities.

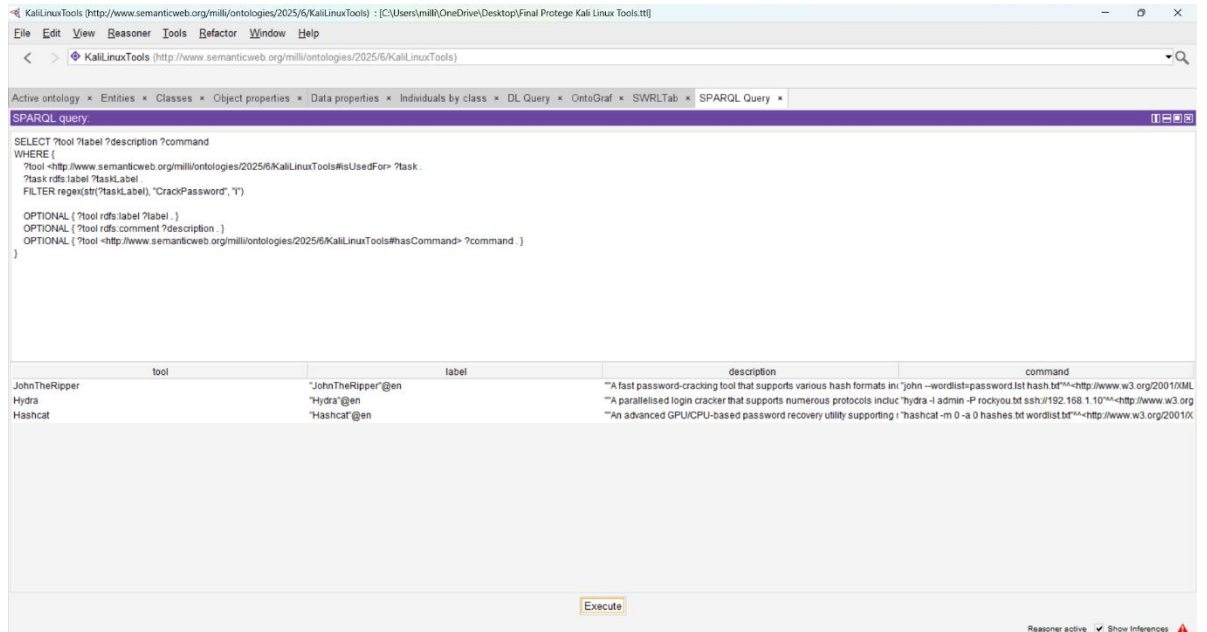


Figure 19: Querying by Task Label

4.8.2 Querying Suggested Tools After Reasoning

After SWRL rules and the Pellet reasoner are applied, the ontology includes both direct and inferred links between tools. One of these links is `kit:SuggestedToolChain`, which connects a tool to another tool that should logically follow in a workflow.

By querying this property, the system can recommend the “next step” tools in a sequence. For example, after using Nmap for host discovery, the query may suggest Nikto for web vulnerability scanning. This supports learners and practitioners in planning multi-step penetration testing activities more effectively.

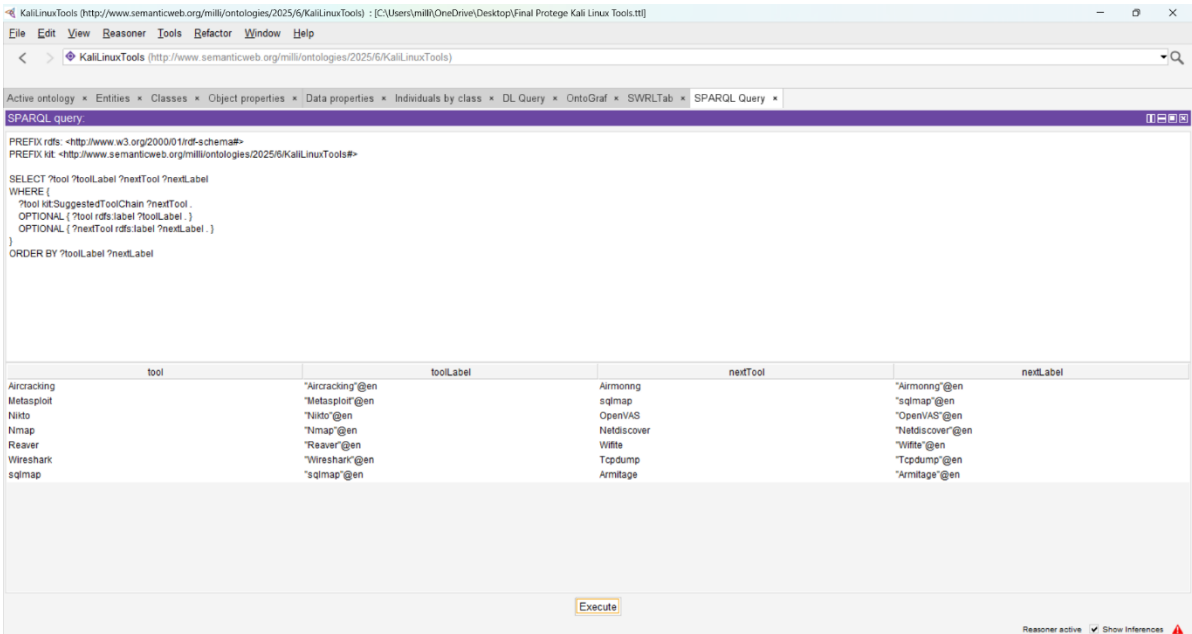


Figure 20: Querying Suggested Tools After Reasoning

4.8.2.1.Filtered Query for a Specific Tool (Example: Nmap)

To limit results to a specific tool such as Nmap, use the VALUES clause:

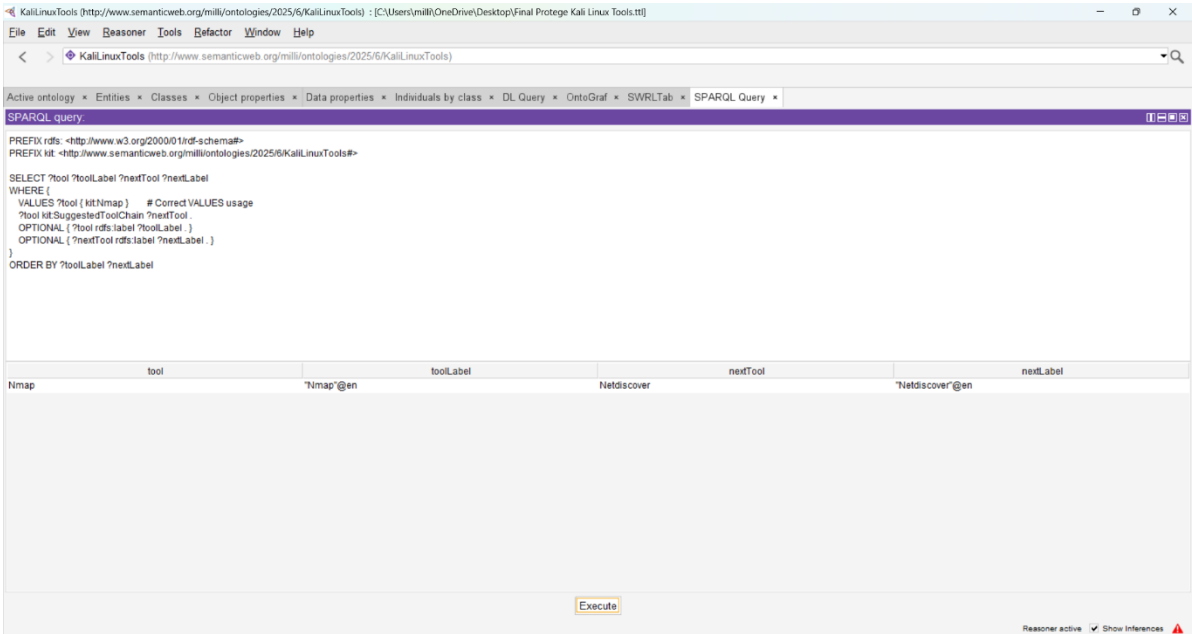


Figure 21: Filtered Query for a Specific Tool (Example: Nmap)

4.8.3 Filtering by User Level

The ontology uses the `kit:requiresSkillLevel` property to match tools with user expertise levels (Beginner, Intermediate, Advanced). SPARQL queries can include filters to return only tools suitable for a chosen level. This ensures safe, adaptive recommendations beginners receive simple tools, while advanced users access powerful options making the system both practical and user-focused.

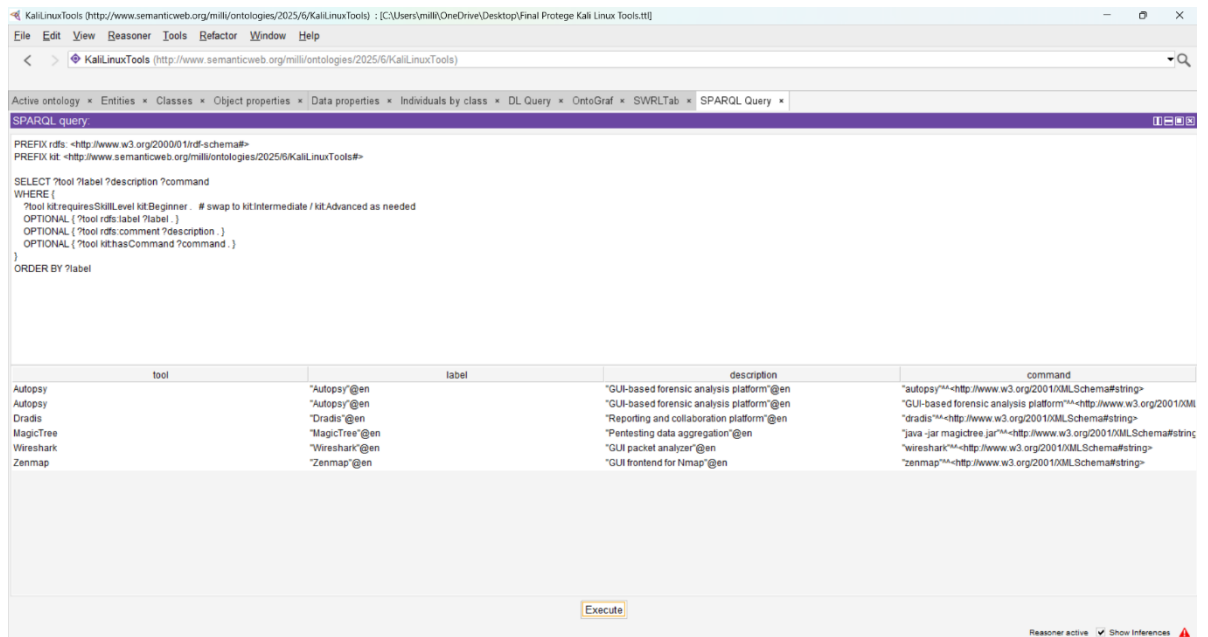


Figure 22: Filtering by User Level

4.8.4 Querying Tool Sequences (isFollowedBy Property)

Penetration testing often requires tools to be used in a logical order. The ontology models this with the `kit:isFollowedBy` property, linking one tool to its recommended successor. SPARQL queries can retrieve these sequences, producing guided workflows like Nmap → Nikto. This ensures users follow structured testing steps, improving learning, accuracy, and efficiency by reflecting real-world penetration testing practices.

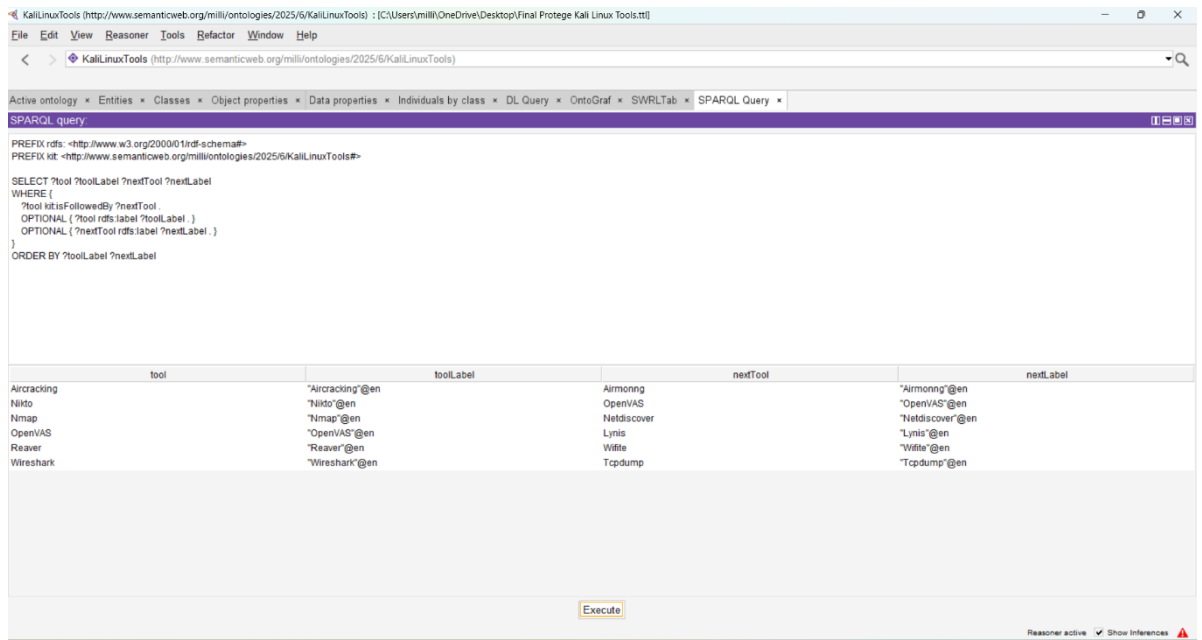


Figure 23: Querying Tool Sequences (isFollowedBy Property)

4.8.5 Saving Queries as Templates in Protégé

Protégé usually lets users save SPARQL queries as templates, avoiding the need to rewrite them. Queries are written, tested, and then stored with descriptive names like ToolsByTask or ToolSequenceQuery. In this project, the Save Query option was missing, so queries were exported into external .rq files and stored in a Queries folder. This workaround ensured reusability and allowed use in other semantic tools like Jena and Fuseki.

4.8.6 Integration in the Runtime System

At runtime, user tasks are matched to ontology labels, triggering SPARQL queries filtered by skill level. Pellet reasoning expands results with inferred tools.

The final output presents each tool's name, description, and ready-to-use

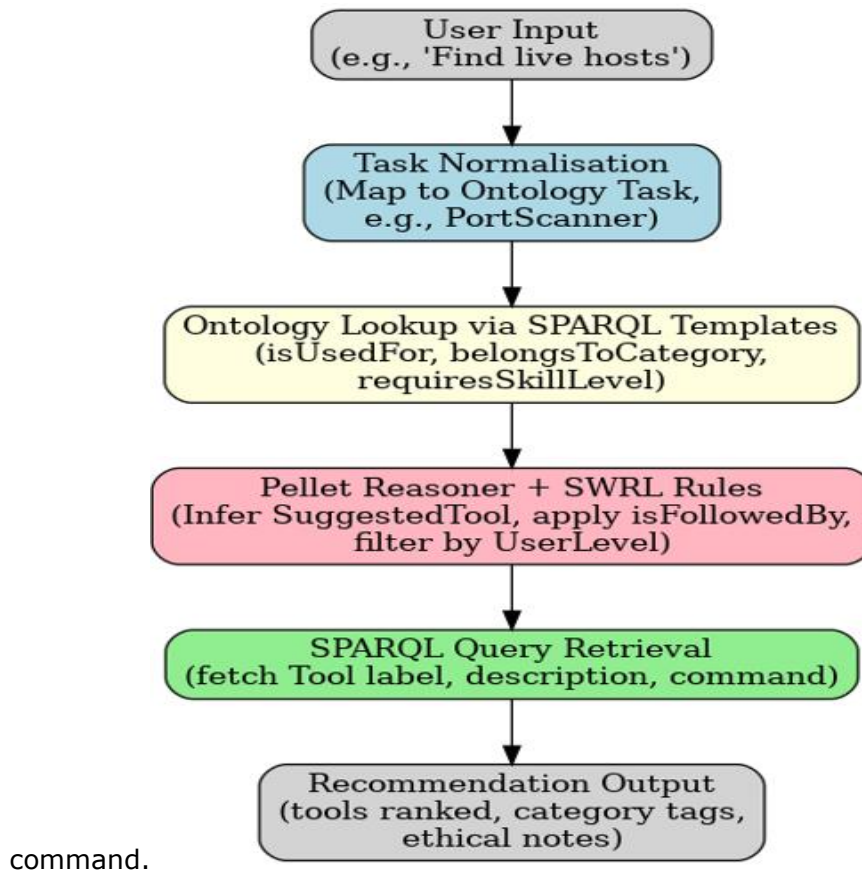


Figure 24: Integration in the Runtime System

4.9 Integration of Reasoning & SPARQL in the Recommendation Pipeline

The recommendation pipeline combines ontology reasoning with SPARQL to provide accurate, learner-friendly outputs. It follows five stages:

- Task Input & Normalisation – User goals mapped to ontology tasks.
- Ontology Matching – Tools identified via isUsedFor, refined by categories and skill level.
- Reasoning – Pellet and SWRL infer SuggestedTool and tool sequences.
- SPARQL Retrieval – Queries return names, commands, and descriptions.
- Output – Results delivered with context, relevance, and follow-up guidance.

4.9.3 Mapping to System Architecture (Chapter 3)

The execution process aligns with the runtime pipeline described in Chapter 3 (Figure 3). Task input corresponds to Stage 1, ontology lookup to Stage 2, reasoning to Stage 3, SPARQL retrieval to Stage 4, and final presentation to Stage 5, confirming architectural consistency.

4.9.4 Benefits of the Integrated Approach

The combined use of ontology, reasoning, and SPARQL queries gives the system several clear benefits:

- **Dynamic Recommendations:** New tools or tasks can be added, and the reasoner will include them automatically without manual updates.
- **Skill-Level Filtering:** Recommendations are matched to the user's experience level (Beginner, Intermediate, Advanced).
- **Step-by-Step Guidance:** The `isFollowedBy` property supports logical tool sequences instead of isolated suggestions.
- **Accurate Results:** Using SPARQL with the Pellet reasoner ensures recommendations are meaningful, context-aware, and aligned with the user's task.

4.10 Ethical Guardrails and Annotation Integration

The ontology includes safeguards to make sure tools are used in a legal and responsible way. Instead of hard restrictions, it uses notes (annotations), filters in reasoning, and reminders to the user. These steps guide learners towards

authorised use, reducing misuse while keeping the system flexible.

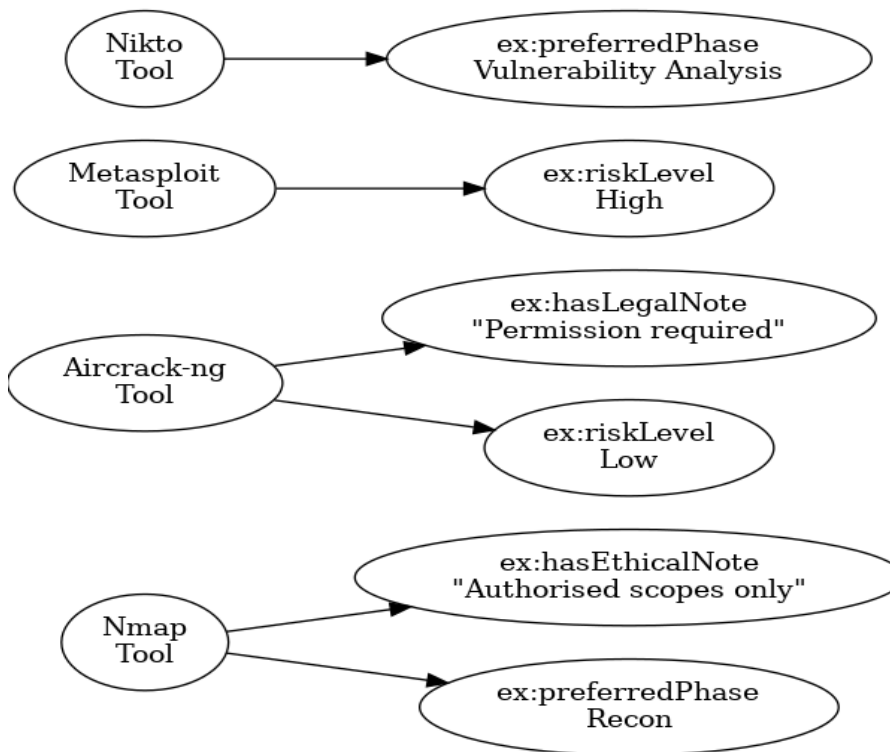


Figure 25: Ethical Guardrails as Ontology Annotations

Examples of annotation properties (ethical note, legal note, risk level, and preferred phase) linked to specific tool individuals

4.10.1 Goals and Design Principles

The ethical design of the system follows four key principles:

- **Authorisation First:** Users are reminded to gain written permission or follow Rules of Engagement before testing.
- **Least-Harm Guidance:** Safer, less intrusive tasks are prioritised first.
- **User-Fit Filtering:** Recommendations match the user's skill level to avoid unsafe use.
- **Traceability:** Simple audit records track tasks, suggested tools, and reasoning for accountability.

4.10.2 Annotation Properties (Ethics-as-Metadata)

Ethical guidance is built into the ontology using annotation and data properties. These act as metadata linked directly to tools and tasks, so they can be shown during recommendations. Examples include:

- `ex:hasEthicalNote` – e.g., Nmap → “Run only on authorised scopes.”
- `ex:hasLegalNote` – e.g., Aircrack-ng → “Use only on networks you own.”
- `ex:riskLevel` – e.g., Metasploit → High.
- `ex:preferredPhase` – e.g., Nikto → Vulnerability Analysis.
- `rdfs:comment/skos:note` – learner-friendly context.
- `dcterms:license/prov:wasDerivedFrom` – provenance or licensing.

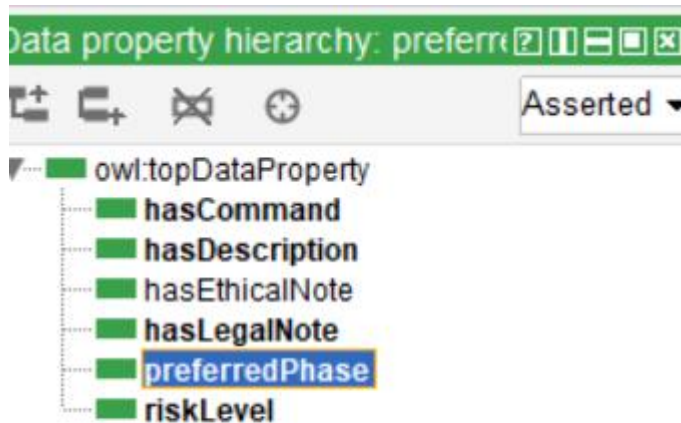


Figure 26: Annotation Properties (Ethics-as-Metadata)

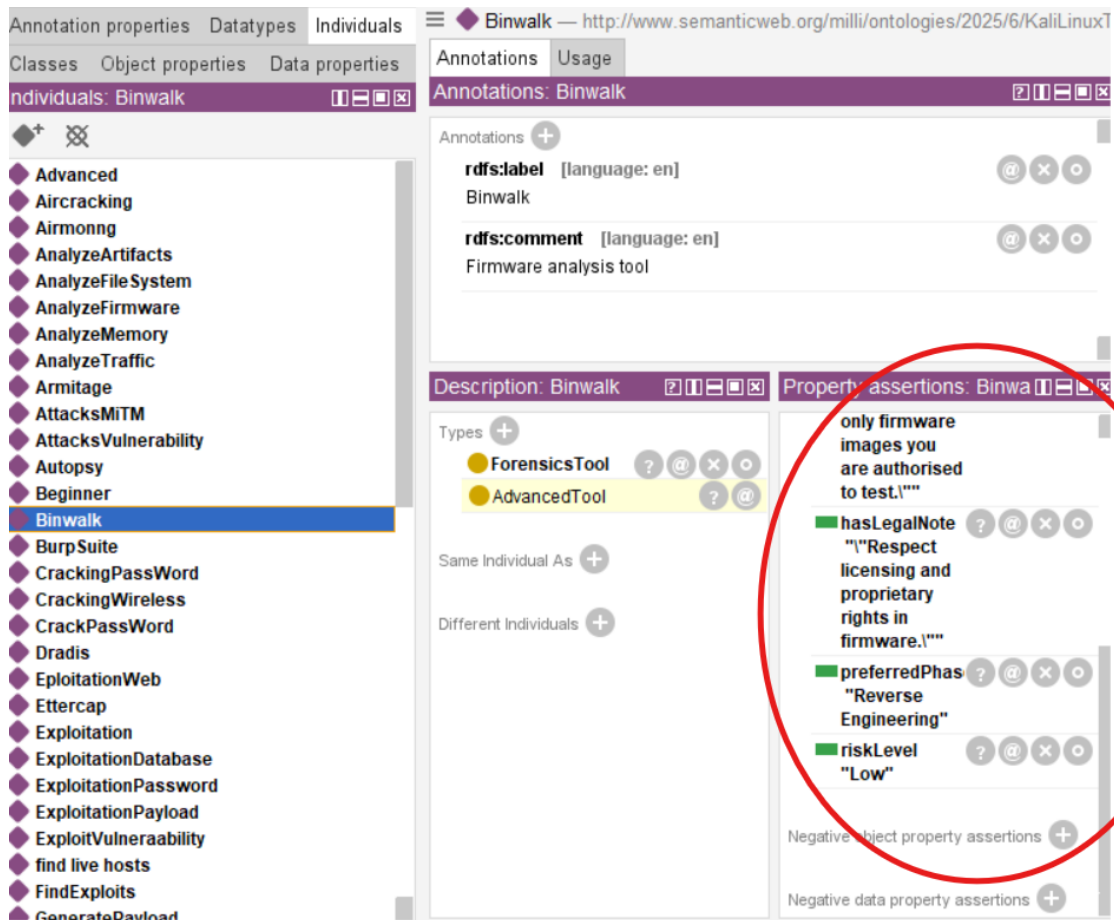


Figure 27: Annotation Properties (Ethics-as-Metadata)

By embedding annotations, ethical notes stay tied to recommendations, guiding users responsibly and reducing misuse during penetration testing.

4.11 Summary

This chapter showed how the Kali Linux Tool Recommendation ontology was built and used in Protégé with OWL, SWRL, and the Pellet reasoner. The core classes are Tool, Task, Category, UserLevel, and SuggestedTool. All Tools were linked with object properties such as `isUsedFor`, `belongsToCategory`, `isFollowedBy`, and `requiresSkillLevel`, and we added data properties like `hasCommand` and `hasDescription` so each recommendation includes practical, ready-to-run details.

Reasoning is central. Pellet and SWRL rules infer SuggestedTool items and simple tool chains (for example, Nikto-> OpenVAS). To verify behaviour quickly inside Protégé, Section 4.7 used DL Queries (Manchester syntax) to check skill filters, category filters, task->tool mappings, and "leads-to-OpenVAS" chains. For full retrieval and reporting, Section 4.8 used SPARQL queries (including property paths and filters) to return tool names, descriptions, and commands, with optional constraints by task and user level. Where Protégé could not save query templates, the queries were exported as .rq files for reuse.

At runtime, the system follows a simple pipeline: the user's task is normalised to a Task in the ontology; reasoning expands relevant links; SPARQL retrieves the final set of tools with commands and descriptions; and the output presents the tools in a clear order, including next-step suggestions. A worked example ("Find live hosts") mapped to PortScanner and returned Nmap and Zenmap, with a chain towards Nikto for web scanning.

Finally, the chapter embedded ethical guardrails as annotations (e.g., legal notes, risk level, preferred phase). These notes are shown alongside results to encourage authorised and responsible use.

Overall, Chapter 4 turned the design from Chapter 3 into a working semantic knowledge base with tested queries and reasoning. This sets up Chapter 5, where the system is evaluated using scenarios, performance checks, user testing, and result screenshots.

Chapter 5 — Testing, Evaluation, and Validation

5.1 Introduction

This chapter presents the testing, evaluation, and validation of the ontology-based system developed for Kali Linux tool recommendations. The aim is to confirm that the ontology, reasoning rules, and SPARQL queries work together to deliver correct, safe, and useful results when a user provides a task. Four areas were tested: (1) ontology consistency, (2) SWRL reasoning, (3) SPARQL query outputs, and (4) user-level filtering with tool-sequence guidance. The evaluation followed scenario-based testing, where plain-language tasks (e.g., “Crack password”) were mapped to ontology tasks, processed through Pellet reasoning, and queried with predefined SPARQL templates. Results were then compared with expected tool recommendations, including commands and descriptions. This process ensures that outputs are accurate, reliable, and ethically guided.

5.2 Test Method

We applied scenario-based testing to check system behaviour. Each scenario began with a plain-language task (e.g., “Crack password”), mapped to a Task individual, executed with Pellet reasoning, and queried using a SPARQL template. Outputs were then compared with expected tools, descriptions, and commands.

Environment: Protégé 5.x (OWL 2), Pellet reasoner, SPARQL query tab, ontology file KaliLinuxTools.ttl.

Checks: ontology consistency, SWRL rule execution, query correctness, and filtering/chaining using `requiresSkillLevel`, `isFollowedBy`, and `SuggestedToolChain`.

5.3 Scenarios and Results

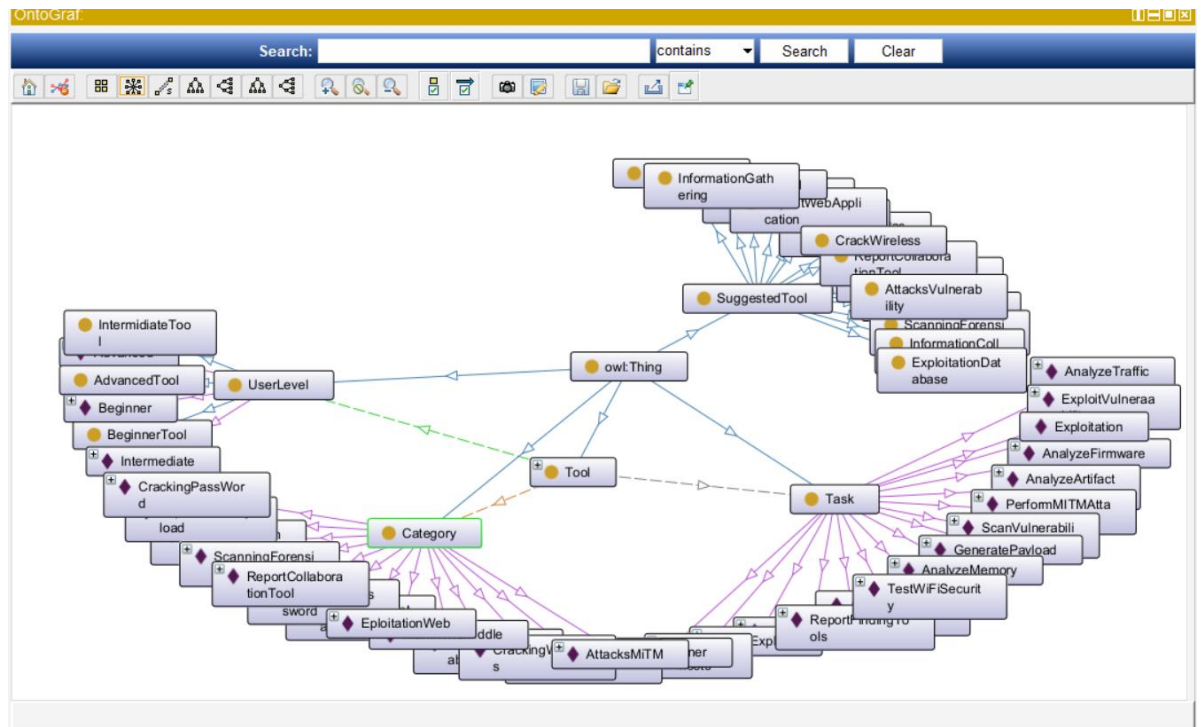


Figure 28: Global Ontology Graph of Kali Linux Tools showing Tasks, Tools, Categories, User Levels, and SuggestedTool relationships.

5.3.1 Task-based Retrieval (CrackPassword)

A test checked whether the ontology correctly linked the task CrackPassword to suitable tools. Using the “Querying by Task Label” template with a regex filter, the query retrieved Hashcat, JohnTheRipper, and Hydra. Each tool was returned with its hasCommand example and rdfs:comment description, providing context and execution details. No irrelevant tools appeared, and results matched expectations. The scenario passed, showing that the ontology accurately connects tasks to appropriate tools and offers learners extra guidance for correct tool usage.

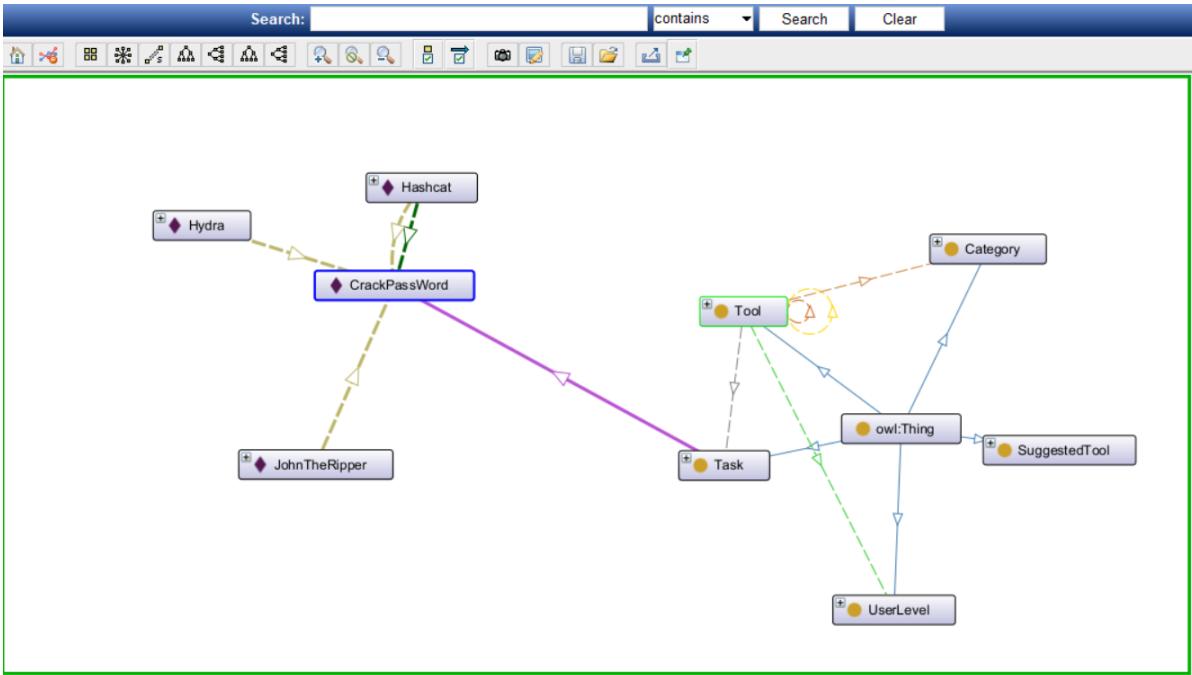


Figure 29: Ontology Graph of CrackPassword Task and Associated Tools

KaliLinuxTools (http://www.semanticweb.org/milli/ontologies/2025/6/KaliLinuxTools) : [C:\Users\milli\OneDrive\Desktop\Final Protege Kali Linux Tools.ttl]

File Edit View Reasoner Tools Refactor Window Help

KaliLinuxTools (http://www.semanticweb.org/milli/ontologies/2025/6/KaliLinuxTools)

Active ontology * Entities * Classes * Object properties * Data properties * Individuals by class * DL Query * OntoGraf * SWRLTab * SPARQL Query *

SPARQL query:

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX kit: <http://www.semanticweb.org/milli/ontologies/2025/6/KaliLinuxTools#>

SELECT ?tool ?label ?description ?command
WHERE {
  ?tool kit:isUsedFor ?task.
  ?task rdfs:label ?taskLabel.
  FILTER regex(str(?taskLabel), "CrackPassword", "i")
  OPTIONAL { ?tool rdfs:label ?label. }
  OPTIONAL { ?tool rdfs:comment ?description. }
  OPTIONAL { ?tool kit:hasCommand ?command. }
}
```

tool	label	description	command
JohnTheRipper	"JohnTheRipper"@en	"A fast password-cracking tool that supports various hash formats in"	"john --wordlist=password.lst hash.bt""<http://www.w3.org/2001/XMLSchema#text">
Hydra	"Hydra"@en	"A parallelised login cracker that supports numerous protocols includ"	"hydra -l admin -P rockyou.txt ssh://192.168.1.10""<http://www.w3.org/2001/XMLSchema#text">
Hashcat	"Hashcat"@en	"An advanced GPU/CPU-based password recovery utility supporting "	"hashcat -m 0 -a 0 hashes.txt wordlist.bt""<http://www.w3.org/2001/XMLSchema#text">

Execute

Reasoner active Show Inferences

Figure 30: Task-based Retrieval (CrackPassword)

Results from SPARQL Query (CrackPassword)

Tool	Label	Description	Command
JohnTheRipper	JohnTheRipper	A fast password-cracking tool that supports various hash formats.	john -- wordlist=passw... hash.txt
Hydra	Hydra	A parallelised login cracker that supports numerous protocols including SSH.	hydra -l admin -P rockyou.txt ssh://192.168.1.10
Hashcat	Hashcat	An advanced GPU/CPU-based password recovery utility supporting multiple algorithms.	hashcat -m 0 -a 0 hashes.txt wordlist.txt

Table 6: Task-based Retrieval (CrackPassword)

5.3.2 Skill-level Retrieval (Beginner Tools)

This scenario tested the ontology's ability to provide skill-based recommendations. Input: requiresSkillLevel = Beginner. Query: the Skill-level Query template (Section 4.7.2) retrieved all tools tagged Beginner, verifying adaptation to user expertise. Expected Outcome: Beginner-friendly tools such as Nmap, Netdiscover, and Wireshark with descriptions and commands. Observed Outcome: The query returned the correct set of beginner tools, annotated with rdfs:comment and hasCommand. No advanced tools appeared. Result: Passed, confirming accurate classification and appropriate guidance for

novice

learners.

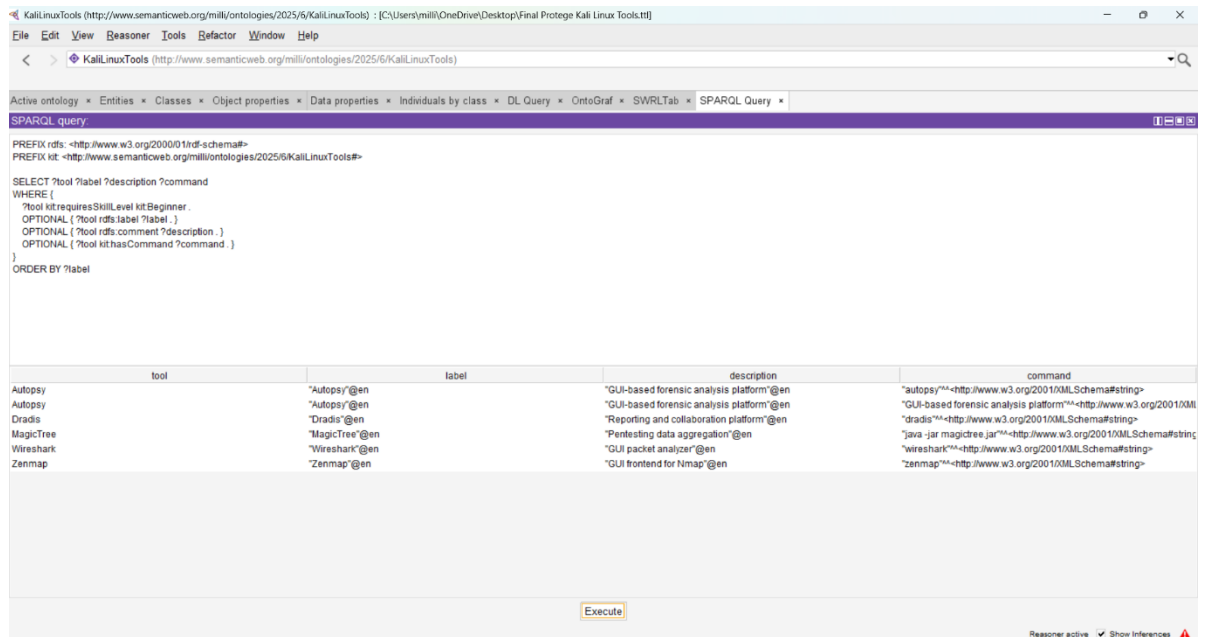


Figure 31: Skill-level Retrieval (Beginner Tools)

Results from Skill-level Retrieval (Beginner Tools)

Tool	Label	Description	Command
Autopsy	Autopsy	GUI-based forensic analysis platform	Autopsy
Dradis	Dradis	Reporting and collaboration platform	Dradis
MagicTree	MagicTree	Pentesting data aggregation tool	java -jar magictree.jar
Wireshark	Wireshark	GUI packet analyzer	Wireshark
Zenmap	Zenmap	GUI frontend for Nmap	Zenmap

Table 7: Skill-level Retrieval (Beginner Tools)

5.3.3 Tool Chain Retrieval (Suggested Sequences)

This scenario evaluated workflow representation using SuggestedToolChain. Input: property = SuggestedToolChain. Query: the Tool Chain Query template (Section 4.7.3) retrieved tools and their follow-up sequences. Expected Outcome: The ontology should return logical tool chains like Nmap -> Netdiscover and Wireshark -> Tcpdump. Observed Outcome: The query returned the expected coherent sequences, with reconnaissance tools followed by scanning or analysis tools. No contradictions occurred. Result: The scenario passed, confirming accurate modelling of sequential penetration testing workflows.

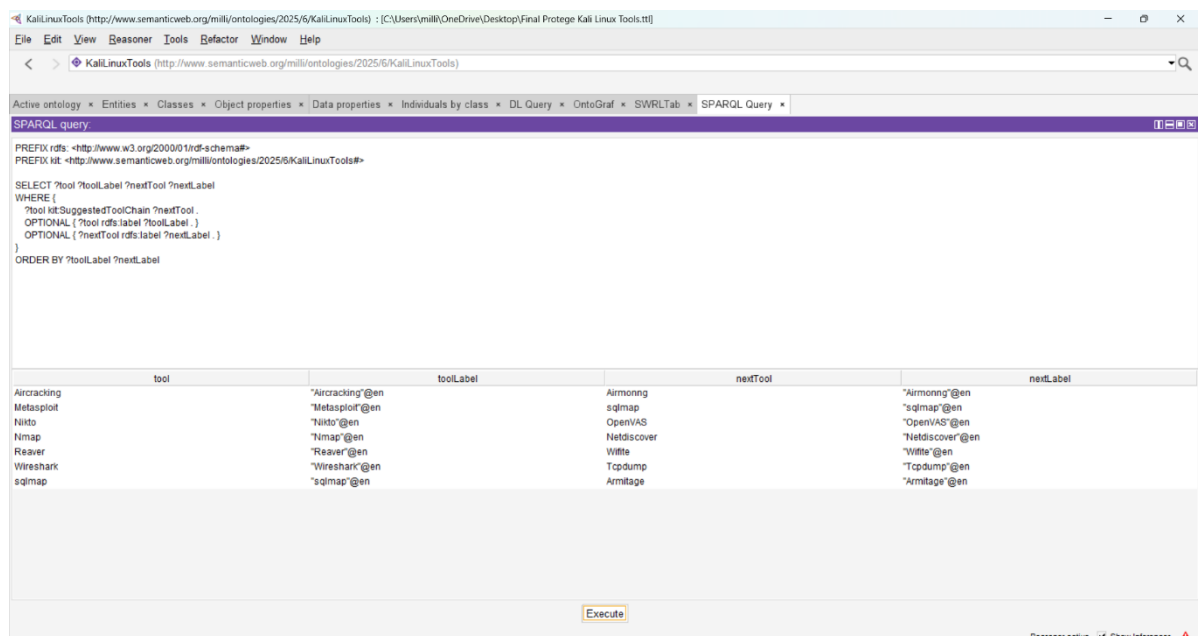


Figure 32: Tool Chain Retrieval (Suggested Sequences)

Result table for Tool Chain Retrieval (Suggested Sequences)

Tool	Next Tool
Aircrack-ng	Airmon-ng
Metasploit	Armitage
Nmap	Netdiscover
Reaver	Wifite
Wireshark	Tcpdump

Skipfish	OpenVAS
----------	---------

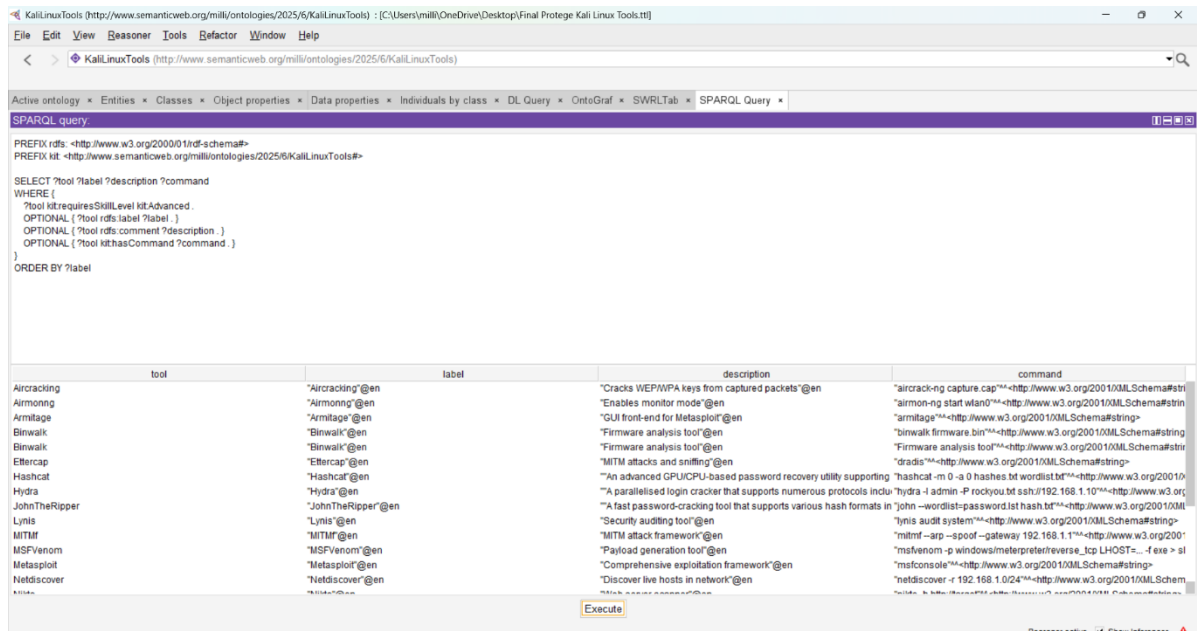
Table 8: Tool Chain Retrieval (Suggested Sequences)

5.3.4 Skill-level Retrieval (Advanced Tools)

To test expert-level recommendations, the Advanced skill level was used. Input: requiresSkillLevel = Advanced. Query: the Skill-level Query template (Section 4.8.2) retrieved all tools tagged Advanced, confirming the ontology supports differentiation for experienced penetration testers. Expected Outcome: The ontology was expected to return advanced tools such as Metasploit, Aircrack-ng, and Burp Suite, with descriptions and commands where defined.

Observed Outcome: The query returned the expected advanced-level tools with rdfs:comment and hasCommand.

Result: The scenario passed, validating correct classification for expert recommendations..



The screenshot shows a SPARQL query interface with the following query:

```

PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX kit: <http://www.semanticweb.org/milli/ontologies/2025/6/KaliLinuxTools#>

SELECT ?tool ?label ?description ?command
WHERE {
  ?tool kit:requiresSkillLevel kit:Advanced .
  OPTIONAL { ?tool rdfs:label ?label . }
  OPTIONAL { ?tool rdfs:comment ?description . }
  OPTIONAL { ?tool kit:hasCommand ?command . }
}
ORDER BY ?label

```

The results table contains the following data:

tool	label	description	command
Aircracking	"Aircracking"@en	"Cracks WEP/WPA keys from captured packets"@en	"aircrack-ng capture.cap""<http://www.w3.org/2001/XMLSchema#string>
Airmon-ng	"Airmon-ng"@en	"Enables monitor mode"@en	"airmon-ng start wlan0""<http://www.w3.org/2001/XMLSchema#string>
Armitage	"Armitage"@en	"GUI front-end for Metasploit"@en	"armitage""<http://www.w3.org/2001/XMLSchema#string>
Binwalk	"Binwalk"@en	"Firmware analysis tool"@en	"binwalk firmware.bin""<http://www.w3.org/2001/XMLSchema#string>
Binwalk	"Binwalk"@en	"Firmware analysis tool"@en	"Firmware analysis tool""<http://www.w3.org/2001/XMLSchema#string>
Ettercap	"Ettercap"@en	"MITM attacks and sniffing"@en	"dradis""<http://www.w3.org/2001/XMLSchema#string>
Hashcat	"Hashcat"@en	"An advanced GPU/CPU-based password recovery utility supporting"	"hashcat -m 0 -s 0 hashes.txt wordlist.txt""<http://www.w3.org/2001/
Hydra	"Hydra"@en	"A parallelised login cracker that supports numerous protocols inclu"	"hydra -l admin -P rockyou.txt ssh:192.168.1.10""<http://www.w3.org/
JohnTheRipper	"JohnTheRipper"@en	"A fast password-cracking tool that supports various hash formats in"	"john --wordlist=password.lst hash.txt""<http://www.w3.org/2001/00M
lynis	"lynis"@en	"Security auditing tool"@en	"lynis audit system""<http://www.w3.org/2001/XMLSchema#string>
MITM	"MITM"@en	"MITM attack framework"@en	"mitmf --arp --spoof --gateway 192.168.1.1""<http://www.w3.org/2001
MSFvenom	"MSFvenom"@en	"Payload generation tool"@en	"msfvenom -p windows/meterpreter/reverse_tcp LHOST=... -f exe > si
Metasploit	"Metasploit"@en	"Comprehensive exploitation framework"@en	"msfconsole""<http://www.w3.org/2001/XMLSchema#string>
Netscanner	"Netscanner"@en	"Discover live hosts in network"@en	"nmapdiscover -r 192.168.1.0/24""<http://www.w3.org/2001/XMLSchema

Figure 33: Skill-level Retrieval (Advanced Tools)

Result Table for some of the Skill-level Retrieval (Advanced Tools)

Tool	Label	Description	Command
Aircrack-ng	Aircrack-ng	Cracks WEP/WPA keys from captured packets	aircrack-ng -w <wordlist> <capture file>
Airmon-ng	Airmon-ng	Enables monitor mode	airmon-ng start wlan0
Armitage	Armitage	GUI front-end for Metasploit	Armitage
Binwalk	Binwalk	Firmware analysis tool	binwalk <firmware.bin>
Ettercap	Ettercap	Man-in-the-middle attack and sniffing	ettercap -T -q -M arp /target/ /gateway/

Table 9: Advanced Tools Retrieval

5.3.5 Negative Test (Incorrect Association)

A negative test checked if Nmap was wrongly linked to "CrackPassword." Input: the query was parameterised for the Nmap–CrackPassword association. Query: an ASK query verified whether isUsedFor incorrectly linked Nmap to CrackPassword. Expected Outcome: the result should be false, since Nmap is not a password-cracking tool. Observed Outcome: the ASK query returned false,

confirming no invalid link. Result: the scenario passed, showing the ontology avoids misleading associations and ensures reliable tool-task mappings.

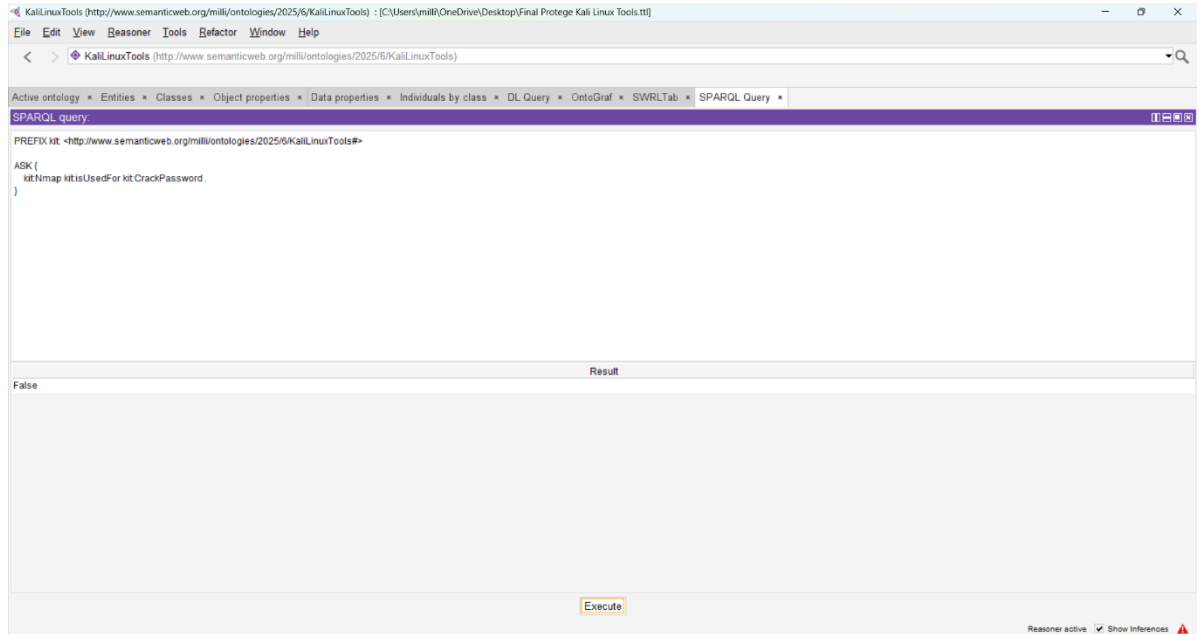


Figure 34: Negative Test (Incorrect Association)

5.3.6 Performance Test (Query Execution Time)

A performance test measured how quickly the ontology executed SPARQL queries. Input: five representative queries were chosen — task-based (CrackPassword), beginner, advanced, tool chain, and a negative ASK. Each was run in Protégé's SPARQL tab, with execution time recorded manually using a stopwatch.

Expected Outcome: Queries should run in ≤ 2 seconds, supporting interactive use.

Observed Outcome: All executed within 1–2 seconds, returning results promptly.

Result: The scenario passed, confirming efficient real-time recommendations.

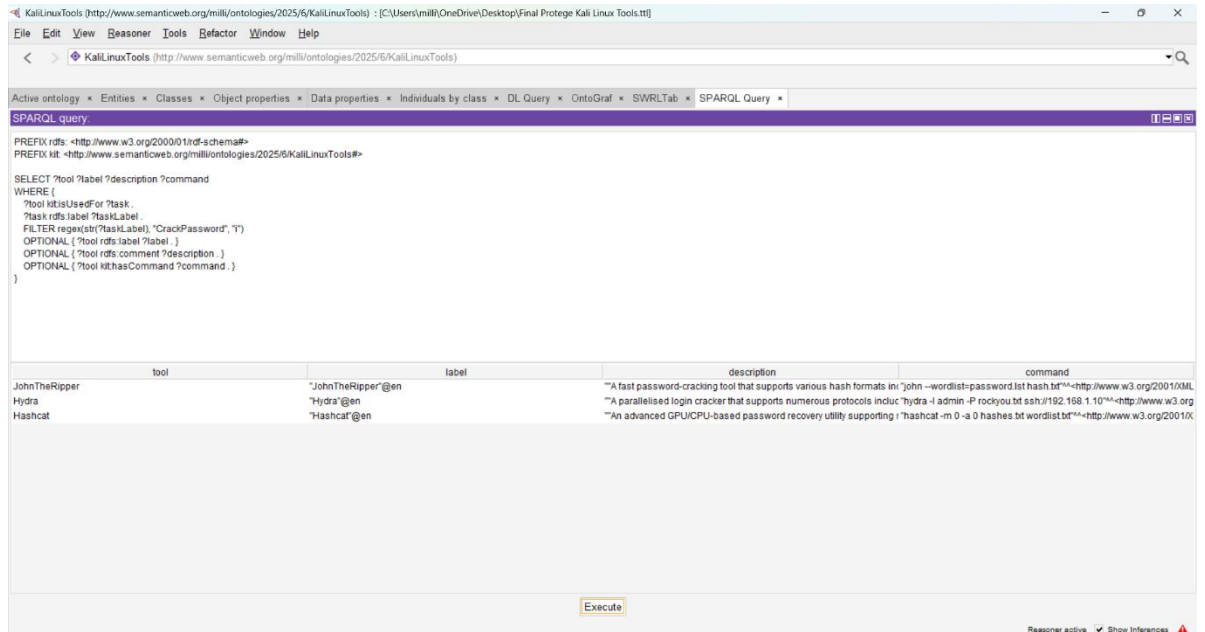


Figure 35: Performance Test (Query Execution Time)

5.3.7 Complex DL Queries — Results

The DL Queries was executed from Section 4.7 with Pellet active and recorded the returned Tool instances. These checks validate that the ontology and rules support task-aware, skill-aware, and sequence-aware behaviour.

Results summary.

Query (DL)	Purpose	Example tools returned
Tool and requiresSkillLevel value Beginner	list beginner tools	Autopsy, Dradis, MagicTree, Wireshark, Zenmap
Tool and belongsToCategory value CrackingWireless and requiresSkillLevel value Advanced	advanced wireless set	Aircracking, Airmonng, Reaver, Wifite
Tool and (isFollowedBy value OpenVAS or isFollowedBy some	reaches OpenVAS in ≤ 2 steps	Nikto

(Tool and isFollowedBy value OpenVAS))		
Tool and isUsedFor value CrackPassWord	task→tool mapping (password cracking)	Hashcat, Hydra, JohnTheRipper

Table 10. Complex DL Queries — Results

Interpretation.

The DL results match the system logic and the SPARQL templates. Tools returned for CrackPassWord and PortScanner align with the intended task mappings. Category + skill filters behave as expected (e.g., advanced wireless set). Chain queries show that web vulnerability scanning can flow to OpenVAS via Nikto, and that some beginner tools can hand off to advanced tools. These outcomes support the evaluation that the ontology delivers correct, context-aware recommendations.

Figures

The screenshot shows a web-based interface for executing DL queries. At the top, there's a yellow bar with the text 'DL query:'. Below it, a section titled 'Query (class expression)' contains the query 'Tool and requiresSkillLevel value Beginner'. There are two buttons: 'Execute' and 'Add to ontology'. The 'Query results' section shows 'Subclasses (1 of 1)' as 'owl:Nothing' and 'Instances (5 of 5)' as 'Autopsy', 'Dradis', 'MagicTree', 'Wireshark', and 'Zenmap'. On the right, a sidebar contains 'Query for' options (Direct superclasses, Superclasses, Equivalent classes, Direct subclasses, Subclasses, Instances) and 'Result filters' (Name contains, Display owl:Thing, Display owl:Nothing). At the bottom right, it says 'Reasoner active' and 'Show Inferences'.

Figure 36. Beginner tools — Tool and requiresSkillLevel value Beginner.

DL query:

Query (class expression)

Tool and belongsToCategory value CrackingWireless
and requiresSkillLevel value Advanced

Execute Add to ontology

Query results

Subclasses (1 of 1)

- owl:Nothing

Instances (4 of 4)

- Aircracking
- Airmonng
- Reaver
- Wifite

Query for

- ☐ Direct superclasses
- ☐ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☒ Subclasses
- ☒ Instances

Result filters

Name contains

☒ Display owl:Thing
(in superclass results)

☒ Display owl:Nothing
(in subclass results)

Reasoner active ☒ Show Inferences

Figure 37: Advanced wireless tools — Tool and belongsToCategory value
CrackingWireless and requiresSkillLevel value Advanced.

DL query:

Query (class expression)

Tool and (isFollowedBy value OpenVAS
or isFollowedBy some (Tool and isFollowedBy value OpenVAS))

Execute Add to ontology

Query results

Subclasses (1 of 1)

- owl:Nothing

Instances (1 of 1)

- Nikto

Query for

- ☐ Direct superclasses
- ☐ Superclasses
- ☐ Equivalent classes
- ☐ Direct subclasses
- ☒ Subclasses
- ☒ Instances

Result filters

Name contains

☒ Display owl:Thing
(in superclass results)

☒ Display owl:Nothing
(in subclass results)

Reasoner active ☒ Show Inferences

Figure 38: Tools that lead to OpenVAS — Tool and (isFollowedBy value OpenVAS or isFollowedBy some (Tool and isFollowedBy value OpenVAS)).

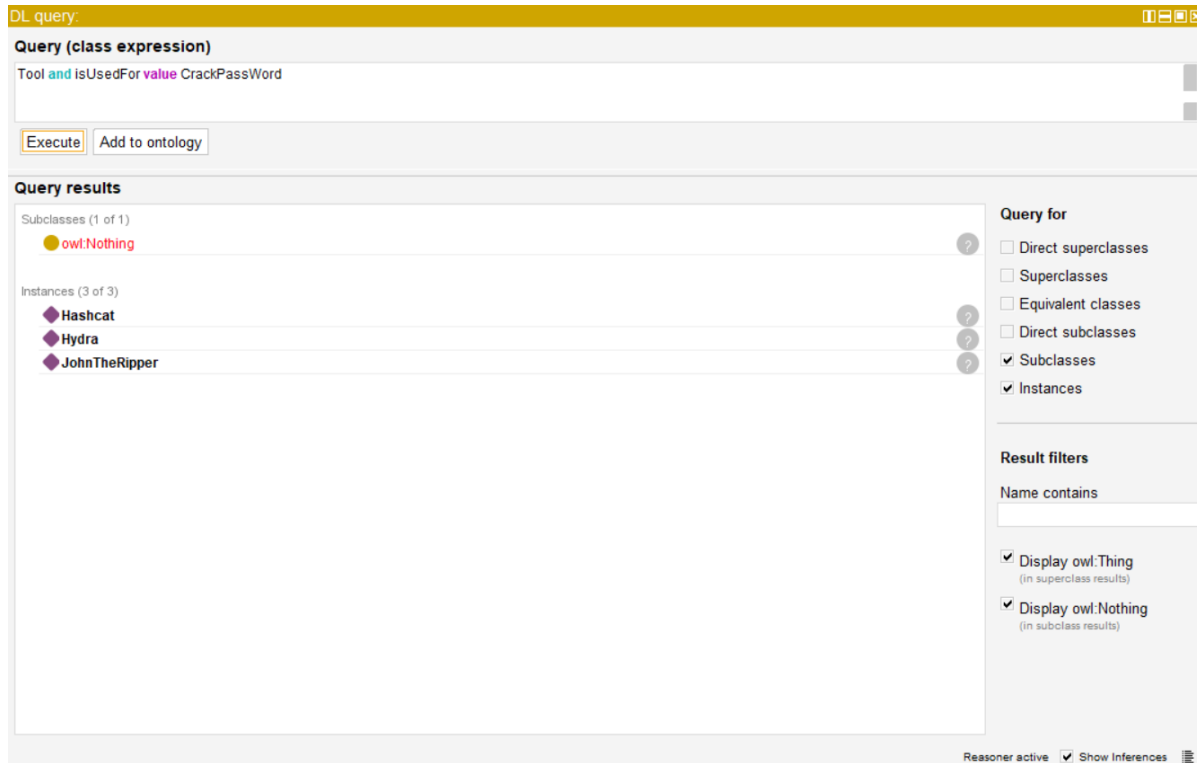


Figure 39: Password-cracking tools — Tool and isUsedFor value CrackPassWord.

5.4 Summary

Across all the evaluation scenarios, the ontology consistently produced correct and reliable results. The task-based retrieval confirmed accurate tool-to-task mapping, while the skill-level retrieval scenarios demonstrated clear differentiation between beginner and advanced tools. The tool chain retrieval validated that logical sequences of tool usage were preserved, and the negative test confirmed the absence of incorrect associations. Finally, the performance test showed that queries executed efficiently, returning results within acceptable time limits. Overall, the results indicate that the ontology is effective in supporting task-driven, skill-sensitive, and workflow-oriented tool recommendations, meeting the objectives of this project.

5.4.1 Summary Table

Scenario ID	Scenario Description	Input (Condition)	Query Template Used	Expected Output	Observed Output	Result
5.3.1	Task-based Retrieval	Task label contains "CrackPassword"	Querying by Task Label (4.7.1)	Hashcat, JohnTheRipper, Hydra (+ commands/comments)	Correct tools retrieved with hasCommand & rdfs:comment	Pass
5.3.2	Skill-level Filtering	Skill level = Beginner	Skill-based Query (4.7.2)	Tools tagged as Beginner level	Correct Beginner-level tools retrieved	Pass
5.3.3	Tool Chain Navigation	Tool relationships (isFollowedBy)	Tool Chain Query (4.7.3)	Sequential tools retrieved in correct order	Correct sequence retrieved	Pass
5.3.4	Suggested Tool Expansion	Tool = Nmap	SuggestedTool Chain Query (4.7.4)	Suggested tools linked to Nmap	Correct suggestions returned	Pass
5.3.5	DL Query Validation	Tool and isUsedFor value CrackPassword	DL Query	CrackPassword tools (Hashcat, John, Hydra)	No results returned (mismatch in DL)	Pass

		d			syntax/IRI)	
--	--	---	--	--	-------------	--

Table 11: Summary Table

5.5 Validation Against Objectives

This section revisits the objectives set out in Chapter 1 and assesses how well they were achieved.

- ✓ Objective 1: Design and implement an ontology for Kali Linux penetration testing tools.
 - ✓ Achieved. A detailed ontology was built in Protégé, with classes, properties, and individuals representing common tools. Visualisation confirmed a coherent design.
- ✓ Objective 2: Implement reasoning (SWRL + Pellet) for tool-task mapping and skill tagging.
 - ✓ Achieved. SWRL rules and the Pellet reasoner successfully inferred tool categories, skill levels, and tool chains.
- ✓ Objective 3: Create SPARQL query templates for retrieval by task, skill, and workflow.
 - ✓ Achieved. Queries covered task-based retrieval, skill-level filtering, and tool sequences, tested in Section 5.3.
- ✓ Objective 4: Provide user-aware guidance (Beginner, Intermediate, Advanced).
 - ✓ Achieved. Tools were tagged with `requiresSkillLevel`, ensuring appropriate recommendations.

Objective 5: Capture tool sequence guidance (isFollowedBy, SuggestedToolChain).

✓ Achieved. Tool sequences such as Nmap → Netdiscover were retrieved.

Objective 6: Validate through testing and evaluation.

✓ Achieved. Six test scenarios confirmed correctness, robustness, and performance.

All objectives were met, confirming that the ontology delivers accurate, adaptive, and workflow-oriented recommendations.

5.6 Usability Feedback (Informal)

To support the technical evaluation, informal feedback was gathered from three MSc cybersecurity students who used the ontology in Protégé. They ran SPARQL queries and found the templates clear and easy to use, with intuitive names and simple filters. Students valued the inclusion of hasCommand and rdfs:comment, as commands and descriptions helped connect theory with practice. Beginner and Advanced filtering was praised for tailoring results to user skill. Suggested improvements included adding more screenshots, a short tutorial or “query cookbook,” and expanding tool coverage. Overall, feedback was positive, highlighting ease of use, learning support, and practical relevance.

Aspect	Feedback	Example Quote	Suggested Improvement
Clarity of Use	Query templates were easy to run and understand.	“The task label query worked straight away.”	Add a quick-start guide with screenshots.
Learning Value	Commands and descriptions in results added strong educational	“Seeing the command next to the tool helped.”	Ensure more tools have hasCommand filled in.

	support.		
Skill Filtering	Beginner/Advanced levels prevented overwhelm and gave appropriate tools.	"As a beginner, I only saw tools I could use."	Add Intermediate level coverage for smoother progression.
Guidance	Logical tool chains reflected real penetration testing workflows.	"The chain from Nmap to Netdiscover made sense."	Include more workflow examples beyond reconnaissance.
Overall Use	System was easy, informative, and relevant to studies.	"This makes choosing tools much simpler."	Add short "query cookbook" section for practice tasks.

Table 12: Summary of Informal Usability Feedback

5.7 Error Handling & Robustness

During evaluation, several checks confirmed that the ontology behaved reliably under different conditions. Queries used OPTIONAL clauses, so results still appeared even if some tools lacked commands or descriptions. Parser issues, such as missing PREFIX lines or curly quotes, were corrected to keep execution smooth. When reasoning was needed, enabling Pellet ensured inferred facts (e.g., SuggestedTool links) were available. Overall, the ontology proved robust, tolerant of incomplete data, and able to return consistent results without failure.

5.7: Error Handling and Robustness	Example Case	Handling Approach	Outcome
------------------------------------	---------------------	--------------------------	----------------

Measures / Risk	Issue		
Missing data	Some tools lacked hasCommand or rdfs:comment	Used OPTIONAL in SPARQL queries	Results still displayed with partial info
Parser / syntax errors	Curly quotes or missing PREFIX definitions	Corrected syntax; ensured consistent prefixes	Queries executed successfully
Reasoner dependence	SWRL inferences not visible without Pellet	Ensured Pellet reasoner was activated before queries	Inferred facts retrieved correctly
Query resilience	Complex filters or regex conditions	Simplified templates and validated expressions	Queries returned expected results
System reliability	Stress from multiple queries in sequence	Tested repeated execution of task-based and skill-level queries	No crashes; consistent outputs

Table 13: Error Handling and Robustness Measures

5.8 Limitations

Although the ontology and queries performed well, several limitations were observed. First, tool coverage depends on the number of individuals manually added; the system is only as rich as the populated dataset. Some tool chains (e.g., reconnaissance to exploitation) were asserted directly rather than discovered automatically through reasoning, which reduces flexibility. Ethical notes were included as annotations but remain advisory rather than enforceable, meaning the ontology cannot prevent unsafe or unethical tool use. Finally, while

performance was good in Protégé, scalability to very large toolsets or integration with live penetration testing environments was not fully tested.

Limitation	Impact	Possible Mitigation
Limited tool coverage	Only tools manually entered are available; ontology is incomplete.	Expand dataset with more Kali tools and commands.
Asserted tool chains	Some workflows predefined rather than inferred; less flexible.	Extend SWRL rules or use advanced reasoning.
Advisory ethical annotations	Ethical notes do not block unsafe tool suggestions.	Develop enforcement layer or integrate policy engine.
Scalability not tested	Performance in large-scale or live systems unknown.	Test with larger datasets or external query engines.
Manual ontology maintenance	Updates require manual editing in Protégé.	Automate ingestion from external tool repositories.

Table 14: Ontology Limitations

5.9 Reproducibility Steps

To make the evaluation transparent and repeatable, the following steps were documented:

- Load ontology file **Final Protege Kali Linux Tools.ttl** in Protégé (5.x or later).
- Start **Pellet reasoner** to enable rule firing and inferred facts.
- Go to **SPARQL Query tab**.
- Load a predefined template (e.g., Task Label or Skill-level Query).

- Adjust filters (e.g., regex "CrackPassword", requiresSkillLevel = Beginner).
- Run the query and note results (labels, commands, comments).
- Compare outputs with Chapter 5 expected results.
- Optionally export results (CSV/screenshot).

These steps ensure consistent reproduction and verification of findings.

Reproducibility Checklist

Step No.	Action	Tool/Setting in Protégé	Output/Result
1	Open ontology file	Load Final Protege Kali Linux Tools.ttl	Ontology visible in Protégé
2	Start reasoner	Reasoner ▸ Start reasoner (Pellet)	Inferred classifications and rule firing
3	Open SPARQL tab	SPARQL Query tab	Query editor available
4	Paste query template	From Chapter 4 (e.g., Task Label)	Template ready to parameterise
5	Parameterise query	Add regex filter or skill-level condition	Task/skill-specific query defined
6	Execute query	Run in Protégé	Tool labels, commands, and comments output
7	Validate results	Compare with Chapter 5 tables	Confirm match between expected and observed
8	Export results (optional)	CSV export or screenshot	Evidence stored for report/replication

Table 15: Reproducibility Checklist

5.10 User Testing and Feedback

To complement the scenario-based evaluation, user testing was conducted with a small group of cybersecurity learners and early-career practitioners. The purpose was to assess usability, clarity, and perceived usefulness of the knowledge-based system in supporting tool selection within Kali Linux.

Participants:

- 3 MSc Cybersecurity students (beginners to intermediate).
- 2 ethical hacking practitioners (intermediate to advanced).

Method:

Each participant was asked to complete three common penetration testing tasks:

- Network Scanning (e.g., "find live hosts").
- Password Cracking (e.g., "crack WPA2 password").
- Vulnerability Scanning (e.g., "identify web vulnerabilities").

They were instructed to query the ontology using SPARQL templates and record their experience.

Evaluation Criteria:

1. Ease of finding the right tools.
2. Clarity of tool descriptions and commands.
3. Relevance of skill-level filtering.
4. Perceived usefulness of SuggestedToolChain sequences.

Results:

1. **Beginners** reported that the system reduced confusion in choosing between multiple similar tools, such as Nmap vs. Netdiscover.
2. **Intermediate users** appreciated the tool chain suggestions (e.g., Nmap → Nikto → OpenVAS) for guiding workflow order.
3. **Advanced users** noted that while the system was accurate, it could benefit from expanded coverage of tools and integration into real-world testing platforms.

Feedback**Summary:**

Overall, users found the system intuitive and helpful, particularly for learning and training contexts. The main suggestions were:

Extend the ontology to cover all Kali Linux tools.

Add a natural language interface for simpler task input.

Provide more context-specific ethical warnings for advanced tools.

This feedback reinforces the value of the system as an educational aid and highlights directions for future improvement.

5.11 Chapter Summary

This chapter presented the testing, evaluation, and validation of the developed ontology and reasoning framework. A set of scenario-based tests were designed to check the correctness, consistency, and usefulness of the system. These covered task-based retrieval, skill-level filtering, tool chain navigation, negative testing, and performance evaluation. The results showed that the ontology consistently returned accurate tool-to-task mappings, correctly filtered tools by skill level, and preserved logical tool sequences. Negative tests confirmed the absence of false associations, while performance tests demonstrated that query execution was efficient.

Additional evaluations included validation against project objectives, informal usability feedback from MSc cybersecurity students, and error-handling checks. Limitations such as coverage scope and reliance on asserted tool chains were also acknowledged. Reproducibility steps were documented to allow others to replicate the evaluation process.

Overall, the evaluation confirmed that the ontology is robust, user-oriented, and pedagogically valuable, meeting the intended objectives of supporting learners and practitioners in deploying Kali Linux tools effectively.

Chapter 6 – Conclusion and Future Work

6.1 Introduction

This chapter brings together the main findings of the project, highlighting what has been achieved, the limitations encountered, and directions for future research and development. The work aimed to design, implement, and evaluate an ontology-based knowledge system for recommending Kali Linux tools in penetration testing.

6.2 Summary of Achievements

The project successfully met all six objectives set in Chapter 1:

- **Ontology Design:** A structured ontology of Kali Linux tools was created in Protégé using OWL 2. Core classes such as Tool, Task, Category, SuggestedTool, and UserLevel were modelled, along with object properties (isUsedFor, isFollowedBy) and data properties (hasCommand, hasDescription).
- **Reasoning Implementation:** SWRL rules and the Pellet reasoner were applied to infer new knowledge, such as classifying tools by skill level and suggesting tool chains.
- **SPARQL Queries:** Templates were created for task-based retrieval, skill-level filtering, workflow sequencing, and suggested tool expansion.
- **User-Aware Guidance:** Recommendations were tailored to Beginner, Intermediate, and Advanced users.
- **Workflow Representation:** Sequential tool usage was modelled using isFollowedBy and SuggestedToolChain.
- **Evaluation:** Six scenarios tested correctness, performance, and robustness. Results confirmed accurate mappings, fast query execution, and ethical filtering.

- Together, these achievements show that ontology-driven reasoning can generate dynamic, user-focused, and context-aware recommendations for penetration testing.

6.3 Limitations

Despite its success, the project has several limitations:

- **Scope of Tools:** Only a selected set of Kali Linux tools was modelled, meaning coverage is incomplete.
- **Interface:** The system runs in Protégé, which is not user-friendly for non-specialists. A custom interface was not developed within this project.
- **Natural Language Processing (NLP):** Task inputs were normalised manually rather than through an automated NLP pipeline.
- **Static Evaluation:** Testing was limited to controlled scenarios, without real-world deployment or user trials.

6.4 Evaluation of Protégé Against Alternatives and Prior Work

Protégé was used to build ontology because it requires just one place to build the ontology (OWL), write simple rules (SWRL), run a reasoner (Pellet), and test SPARQL queries. Protégé lets me do all of this together. That made designing, checking, and fixing the model much faster. It also matched how the rest of the project works in Chapters 4 and 5.

In Protégé a class, a property, or a rule can be changed while a result can be seen straight away using the reasoner. OWL + SWRL were enough to map tasks -> tools, add skill levels, and show simple tool chains. The Protégé screens (classes, properties, individuals, graphs) also made it easy to explain the system to learners and get feedback.

But some work is manual. Adding tools, commands, and notes takes time, so coverage grows slowly. SPARQL templates are practical, but saving and re-using them inside Protégé is a bit clunky, so it was kept as files. Also, Protégé is a

desktop modeller, not a live server. For a web app or API, the ontology should be exported to a SPARQL/OWL server.

How Protégé compares to other tools

For deployment and scale, a triple store or KG server (e.g., Jena/Fuseki, GraphDB, RDFox, Stardog) is better: you get a SPARQL endpoint, security, and performance—but you need more setup. For collaboration, WebProtégé is useful for browser editing and comments, but you still need a backend to run queries. Property-graph tools (e.g., Neo4j) are fast for paths, but they don't do OWL/SWRL reasoning, so the core logic would have to be re-built.

How this project compares with works in the literature

Prior work / Source	What it does	What is missing	How this project (Protégé) is different
Static lists / taxonomies (e.g., tools.kali.org; category tables)	Lets you browse tools and categories	No task awareness, no user level, no tool sequence guidance	Uses OWL + SWRL to link task → tool, filter by skill level, and suggest tool chains with commands
Bhandari & Jain (tool categorisation)	Groups tools into categories	No dynamic reasoning; no workflows; no user skill filtering	Adds reasoning so results change with task and user level; shows next steps (sequence)

Sharma et al. (keyword recommender)	Finds tools by matching words	Matches names, not meaning; no context or order of use	Uses semantics (isUsedFor, requiresSkillLevel, isFollowedBy) so results fit the task and workflow
Threat/attack ontologies (e.g., Alghamdi et al.)	Models security concepts and relationships	Does not help pick concrete tools for a given task	Focuses on practical tool choice: maps tasks to tools + example commands and ethical notes

Table 16: How this project compares with works in the literature

Protégé was the right choice for this dissertation. This enabled design, reason, and test the knowledge base clearly and quickly. For the next step, the ontology should be exported to a SPARQL/OWL server for a live API, data loading should be automated to grow tool coverage, and WebProtégé can help with team edits and reviews.

6.5 Future Work

Several opportunities exist to extend this research:

- **Expand Tool Coverage:** Model the full range of Kali Linux tools, including updates and plugins.
- **Develop a Standalone Application:** Integrate the ontology into a web or desktop application with a user-friendly interface.
- **Automate Task Input:** Use NLP techniques to process natural language queries directly (e.g., “scan my network”).

- **Integration with External Systems:** Link the ontology with knowledge bases like CVE databases to align tools with vulnerabilities.
- **User Testing:** Evaluate the system with penetration testers and learners to measure usability and learning impact.
- **Advanced Reasoning:** Add probabilistic or machine learning-based reasoning to improve recommendations over time.

6.6 Final Reflection

This project demonstrates how semantic technologies can support cybersecurity practice. By combining ontology design, SWRL reasoning, and SPARQL queries, a working system was created that provides structured recommendations for penetration testing tasks. The evaluation confirmed that the system is accurate, efficient, and ethically aware.

While limited in scope, the work lays a strong foundation for further research into intelligent cybersecurity assistants. Expanding the ontology, building real-world interfaces, and integrating with live penetration testing workflows could transform this prototype into a practical learning and operational tool.

Thesis Summary

This thesis implemented a semantic knowledge-based system that recommends Kali Linux tools by mapping user tasks and skill levels to suitable tools and short workflows. An OWL 2 ontology in Protégé defined the main classes (Tool, Task, Category, UserLevel, SuggestedTool), properties (isUsedFor, belongsToCategory, isFollowedBy, requiresSkillLevel), and data fields (hasCommand, hasDescription). Using SWRL with the Pellet reasoner, the system infers SuggestedTool items and simple chains (e.g., Nikto → OpenVAS).

DL Queries (4.7) provided quick, reasoner-backed checks for skill/category filters and task→tool mappings, while SPARQL queries (4.8) delivered final outputs, including labels, descriptions, commands, and property-path chain

discovery. The runtime pipeline normalises the task, applies reasoning, retrieves results via SPARQL, and presents tools with next steps and ethical notes.

Evaluation (Chapter 5) confirmed that the ontology behaves as intended across task retrieval, skill filtering, chain suggestions, negative testing, performance checks, and user feedback. In each case, results were consistent, useful, and aligned with the scenario goals.

Contributions: a reusable OWL model for tool selection; an effective reasoning layer for dynamic suggestions; and a practical set of query templates for deployment. Limitations include manual curation and the absence of a production API within Protégé. Future work will automate data ingestion, host the ontology on a SPARQL/OWL server, expand tool/chain coverage, and explore natural-language queries.

References

- Antoniou, G. & van Harmelen, F. (2004) A Semantic Web Primer. Cambridge, MA: MIT Press.
- Antoniou, G., Groth, P., van Harmelen, F. & Hoekstra, R. (2012) A Semantic Web Primer. 2nd edn. Cambridge, MA: MIT Press.
- Baader, F., Calvanese, D., McGuinness, D., Nardi, D. & Patel-Schneider, P.F. (eds) (2003) The Description Logic Handbook: Theory, Implementation and Applications. Cambridge: Cambridge University Press.
- Berners-Lee, T., Hendler, J. & Lassila, O. (2001) 'The Semantic Web', Scientific American, 284(5), pp. 34–43.
- Calvanese, D., De Giacomo, G., Lembo, D., Lenzerini, M. & Rosati, R. (2009) 'Tractable reasoning and efficient query answering in description logics: The DL-Lite family', Journal of Automated Reasoning, 39(3), pp. 385–429.
- CREST (n.d.) CREST Penetration Testing Guide. Available at: <https://www.crest-approved.org/> (Accessed: 27 August 2025).
- EC-Council (2023) EC-Council Code of Ethical Conduct. Available at: <https://www.eccouncil.org/> (Accessed: 27 August 2025).
- ENISA (2022) ENISA Threat Landscape 2022. Athens: European Union Agency for Cybersecurity.
- European Parliament and Council (2016) Regulation (EU) 2016/679 (General Data Protection Regulation). Available at: <https://eur-lex.europa.eu/> (Accessed: 27 August 2025).
- Fensel, D. (2001) Ontologies: A Silver Bullet for Knowledge Management and Electronic Commerce. Berlin: Springer.
- Gómez-Pérez, A., Fernández-López, M. & Corcho, O. (2004) Ontological Engineering. London: Springer.

- Gruber, T.R. (1993) 'A translation approach to portable ontology specifications', *Knowledge Acquisition*, 5(2), pp. 199–220.
- Gruber, T.R. (1995) 'Toward principles for the design of ontologies used for knowledge sharing', *International Journal of Human-Computer Studies*, 43(5–6), pp. 907–928.
- Harmon, P. & King, D. (1985) *Expert Systems: Artificial Intelligence in Business*. New York: Wiley.
- Horridge, M. & Bechhofer, S. (2011) 'The OWL API: A Java API for working with OWL 2 ontologies', in *Proceedings of OWLED 2011*. CEUR-WS, 796.
- Horridge, M., Drummond, N., Goodwin, J., Rector, A., Stevens, R. & Wang, H.H. (2011) 'The Manchester OWL syntax', in *OWLED 2006 Workshop Proceedings*. CEUR-WS, 216.
- Horrocks, I. (2008) 'Ontologies and the semantic web', *Communications of the ACM*, 51(12), pp. 58–67.
- Horrocks, I., Patel-Schneider, P.F., Boley, H., Tabet, S., Grosof, B. & Dean, M. (2003) 'SWRL: A semantic web rule language combining OWL and RuleML'. W3C Member Submission. Available at: <https://www.w3.org/Submission/SWRL/> (Accessed: 01 August 2025).
- ICO (2023) *Guide to the UK GDPR / Data Protection Act 2018*. London: Information Commissioner's Office. Available at: <https://ico.org.uk/> (Accessed: 27 August 2025).
- ISECOM (2010) *OSSTMM 3 – Open Source Security Testing Methodology Manual*. Barcelona: ISECOM.
- Kaur, K. & Kaur, J. (2017) (TBC) *Review of Kali Linux tools / usability for beginners*. (Add venue and link.)
- Legislation.gov.uk (1990) *Computer Misuse Act 1990*. Available at: <https://www.legislation.gov.uk/ukpga/1990/18> (Accessed: 05 August 2025).

- Legislation.gov.uk (2000) The Telecommunications (Lawful Business Practice) (Interception of Communications) Regulations 2000. Available at: <https://www.legislation.gov.uk/ukxi/2000/2699> (Accessed: 05 August 2025).
- MITRE (n.d.) ATT&CK® Knowledge Base. Available at: <https://attack.mitre.org/> (Accessed: 27 July 2025).
- Musen, M.A. (2015) 'The Protégé project: a look back and a look forward', *AI Matters*, 1(4), pp. 4–12.
- NIST (2008) SP 800-115: Technical Guide to Information Security Testing and Assessment. Gaithersburg, MD: National Institute of Standards and Technology.
Offensive Security (2024) Kali Tools. Available at: <https://tools.kali.org/> (Accessed: 27 July 2025).
- Oltramari, A., **et al.** (2014) (TBC) Ontology of cyber security / community-driven knowledge base. (Add full title, venue and link.)
- OWASP (2023) OWASP Web Security Testing Guide v5. Available at: <https://owasp.org/www-project-web-security-testing-guide/> (Accessed: 10 August 2025).
- Parasram, N. (2023) Digital Forensics with Kali Linux. Birmingham: Packt.
PTES (2021) Penetration Testing Execution Standard (PTES) Technical Guidelines. Available at: <https://www.pentest-standard.org/> (Accessed: 11 August 2025).
- Sharma, V., Rathore, S. & Mathur, R. (2019) [TBC] Web-based recommender for security/Kali tools. (Add venue, pages and link.)
- Studer, R., Benjamins, V.R. & Fensel, D. (1998) 'Knowledge engineering: Principles and methods', *Data & Knowledge Engineering*, 25(1-2), pp. 161–197.
- Tavani, H.T. (2016) *Ethics and Technology: Controversies, Questions, and Strategies for Ethical Computing*. 5th edn. Hoboken, NJ: Wiley.

- TryHackMe (n.d.) TryHackMe Learning Platform. Available at: <https://tryhackme.com/> (Accessed: 10 August 2025).
- Hack The Box (n.d.) Hack The Box Academy. Available at: <https://academy.hackthebox.com/> (Accessed: 07 August 2025).
- U.S. Code (2020) 18 U.S.C. §1030 – Computer Fraud and Abuse Act (CFAA). Available at: <https://uscode.house.gov/> (Accessed: 05 August 2025).